



Final Exam Review

CS 400 – Programming III

Matt Schwennesen

August 05, 2026

Outline

Final Exam Logistics

Final Exam Review

Practice Exam



Final Exam Logistics

Final Exam Logistics

- ◆ Final exam released Thursday, August 06, 2026 at 12:00:00 AM Central Time
- ◆ Final exam due Friday, August 07, 2026 at 11:59:59 PM Central Time
- ◆ Two hour time limit.
- ◆ 30 questions, no written code questions.
- ◆ Scratch paper allowed and recommended.
- ◆ Honorlock required.
 - ◇ Google chrome based browser with Honorlock extension

Final Exam Topics

- ◆ Graphs
- ◆ Graph traversals
- ◆ Minimum Spanning Trees
- ◆ Shortest Path
- ◆ Tries
- ◆ Hashtables
- ◆ Linear Sorts
- ◆ Skip Lists
- ◆ Regular Expressions
- ◆ HTML + CSS
- ◆ Web Servers
- ◆ JavaScript
- ◆ GUI Programming
 - ◇ Signals
 - ◇ JSX
 - ◇ Effects & Resources
- ◆ Streams & Pipes



Final Exam Review

Graphs

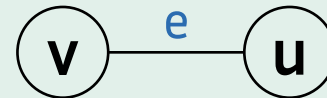
Definition (Node) A *node* or *vertex* is an atomic element of a graph.

$$v \in \mathcal{V}$$



Definition (Edge) An *edge* or *link* is the connection joining a pair of nodes.

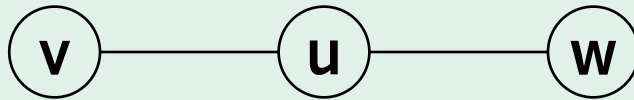
$$e = \{v_1, v_2\} \in \mathcal{E}$$



Graphs

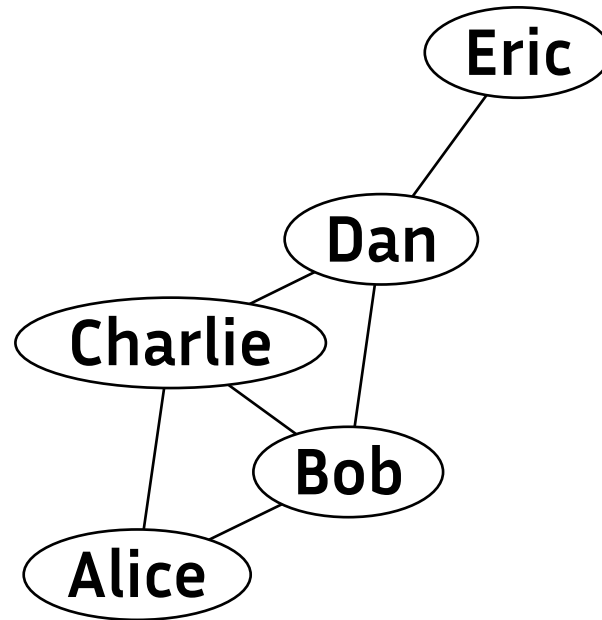
Definition (Graph) A *graph* is a set of **nodes** and a set of **edges**.

$$G = (\mathcal{V}, \mathcal{E})$$



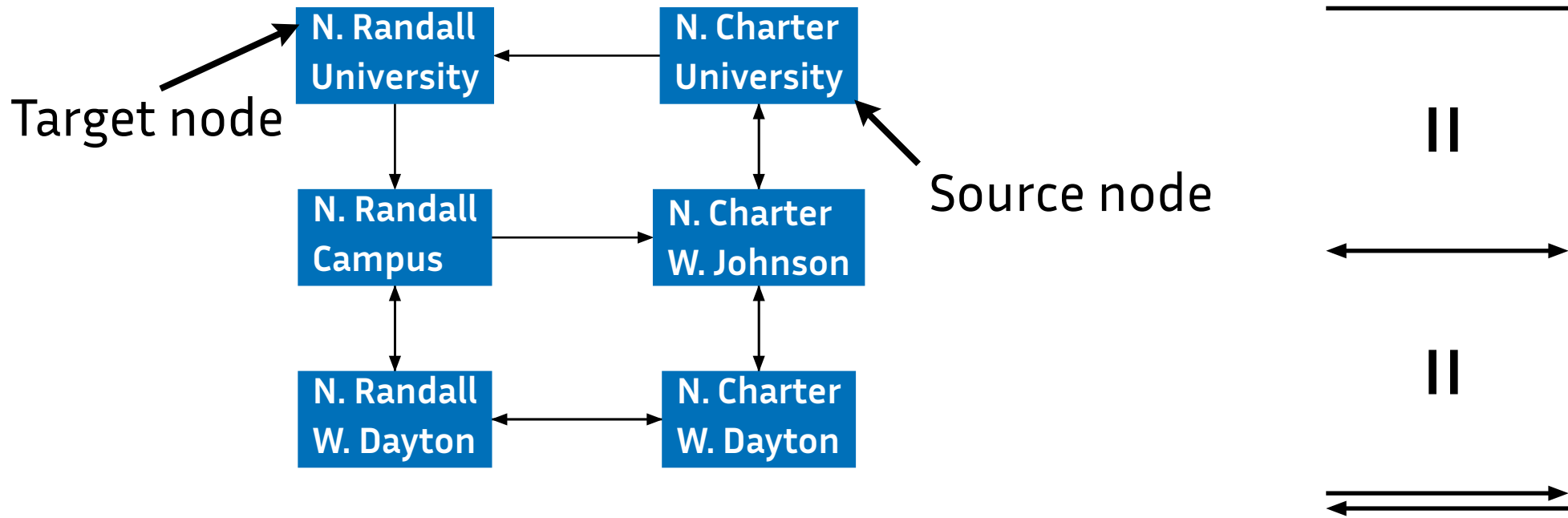
Undirected Graph

- ✦ Edges are **directionless** → Can go in either direction.
- ✦ Example: Friendship network



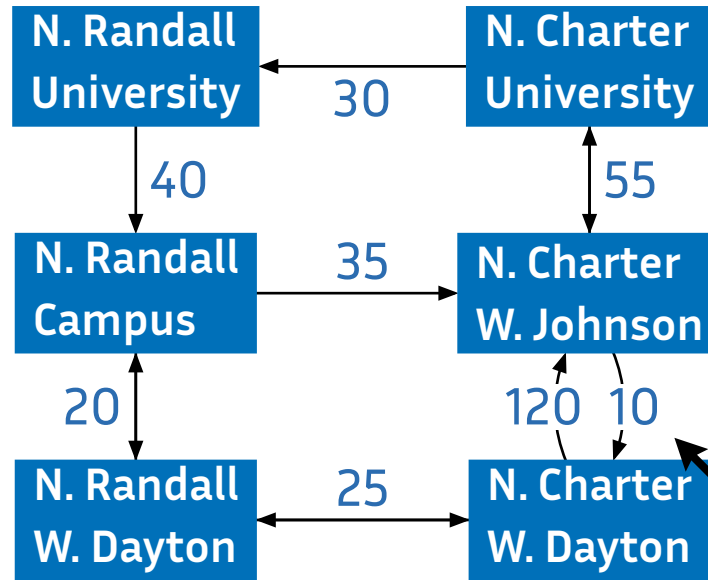
Directed Graph

- ◆ Edges are **directed** → They go in a specified direction
- ◆ Example: Street map



Weighted Graph

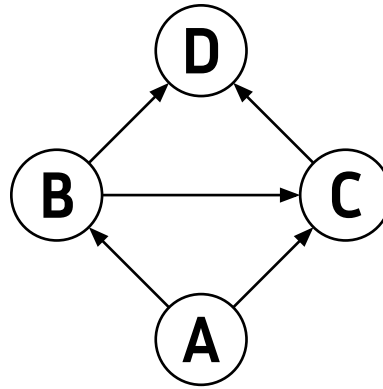
- ◆ Edges can contain **weights / labels**.
- ◆ Example: Travel time



Each direction can have a separate weight!

Graph Representations

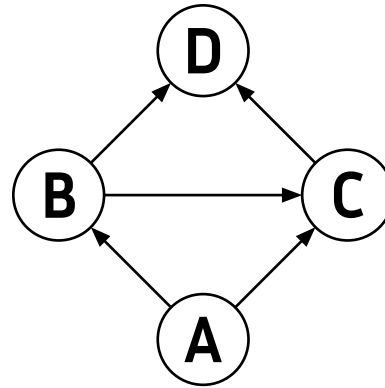
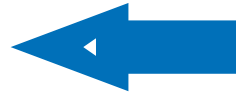
There are multiple ways to **represent** a graph; each is more useful for **different algorithms** or with a different **set of graphs**.



Graph Representations

There are multiple ways to **represent** a graph; each is more useful for **different algorithms** or with a different **set of graphs**.

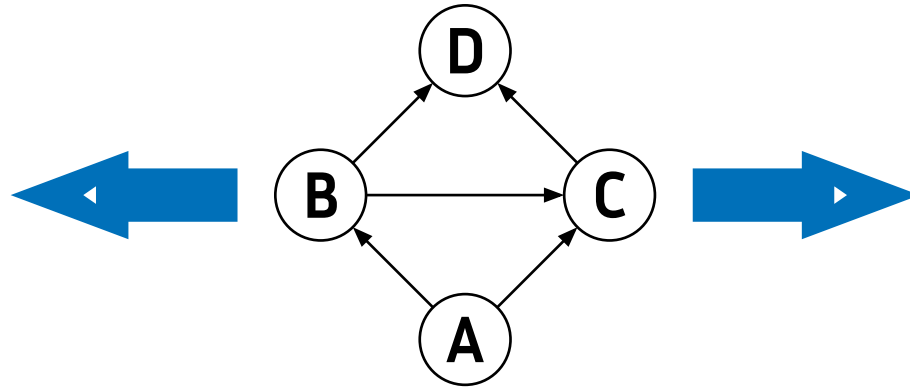
	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	1
D	0	0	0	0



Graph Representations

There are multiple ways to **represent** a graph; each is more useful for **different algorithms** or with a different **set of graphs**.

	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	1
D	0	0	0	0



A	B, C
B	C, D
C	D
D	—

Graph Representation Complexity

Operation	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge Query	$O(1)$	$O(\deg(x))$
All Neighbors	$O(V)$	$O(\deg(x))$

- ✦ Adjacency matrices take up more space but have constant edge query time.
- ✦ Adjacency lists take up less space for **sparse graphs**. Operation speeds also depend on sparsity.

Graph Traversals

- ✦ **Depth-first** traversal
 - ◇ Greedily follow the **first unvisited neighbor**.
 - ◇ Similar to in-order, pre-order or post-order tree traversals.
- ✦ **Breadth-first** traversal
 - ◇ Explore all nodes 1 edge away from the start, then all nodes 2 edges away, 3 edges away, etc...
 - ◇ Similar to level-order tree traversals.

Graph Traversals – DFS

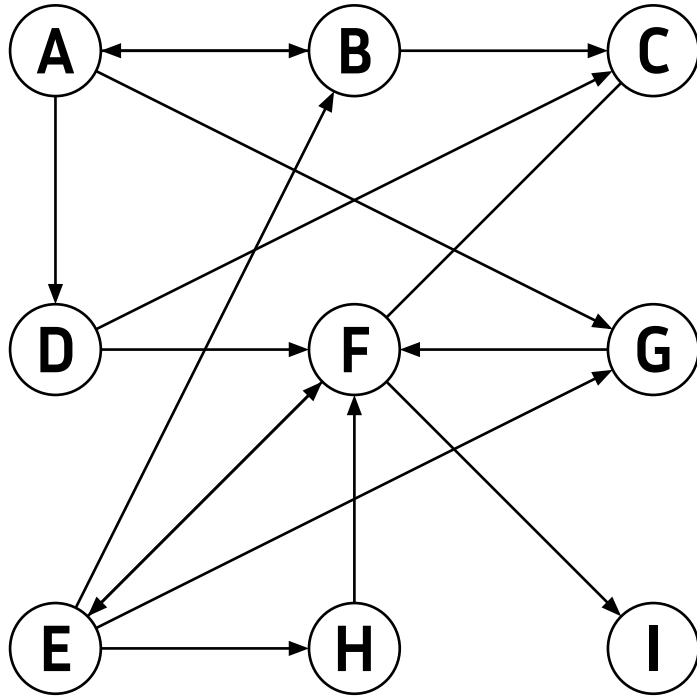
DFT(v):

```
1 |  $s \leftarrow \text{stack}$ 
2 |  $s.\text{push}(v)$ 
3 | while  $s$  is not empty do
4 |    $v \leftarrow s.\text{pop}()$ 
5 |   mark  $v$  as visited
6 |   for each neighbor  $u$  of  $v$  do
7 |     if  $v$  is unvisited then
8 |        $s.\text{push}(u)$ 
```

DFT(v):

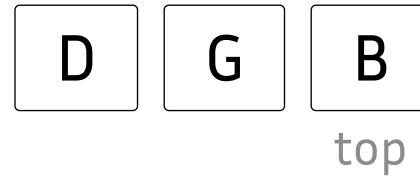
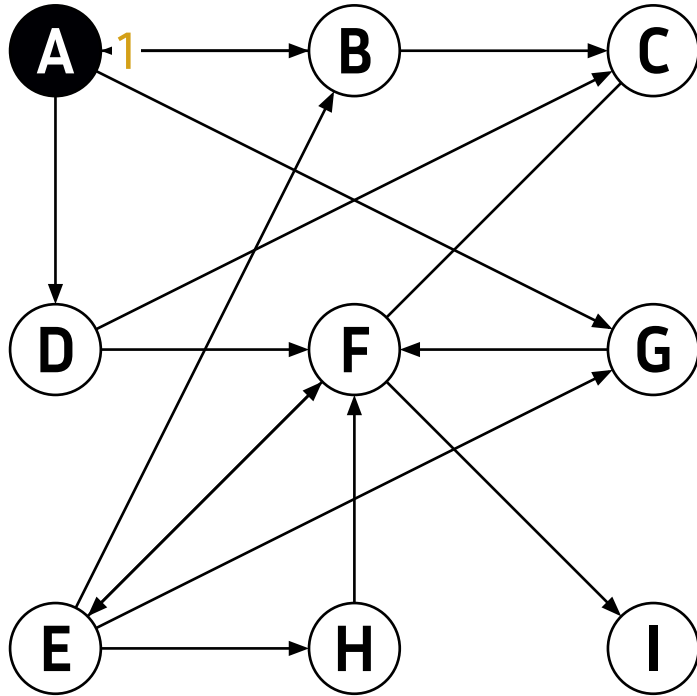
```
1 | mark  $v$  as visited
2 | for each neighbor  $u$  of  $v$  do
3 |   if  $u$  is unvisited:
4 |     |  $\text{DFT}(u)$ 
```

Graph Traversals – DFS Example



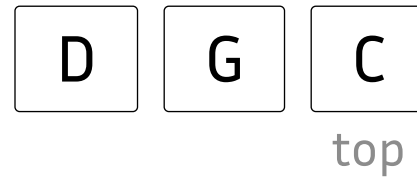
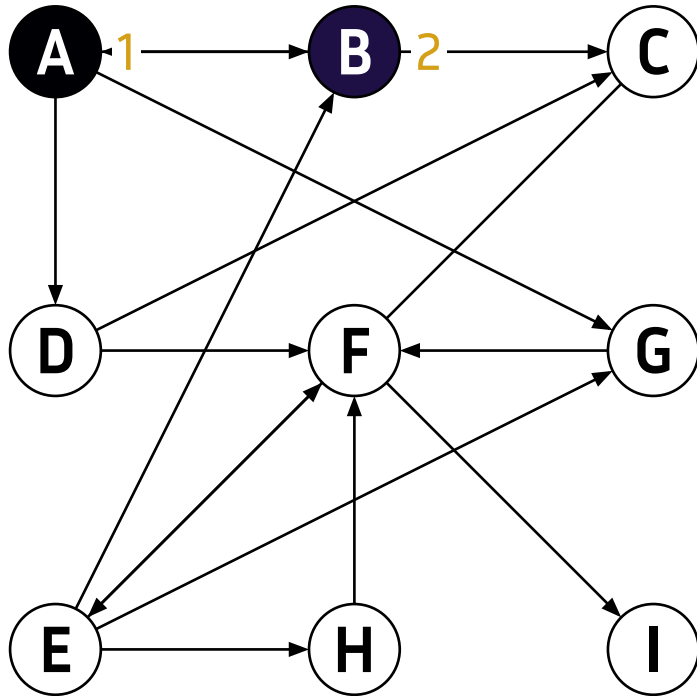
A
top

Graph Traversals – DFS Example



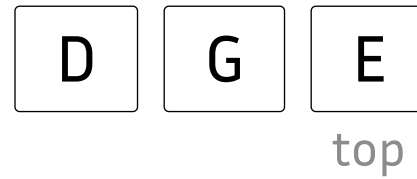
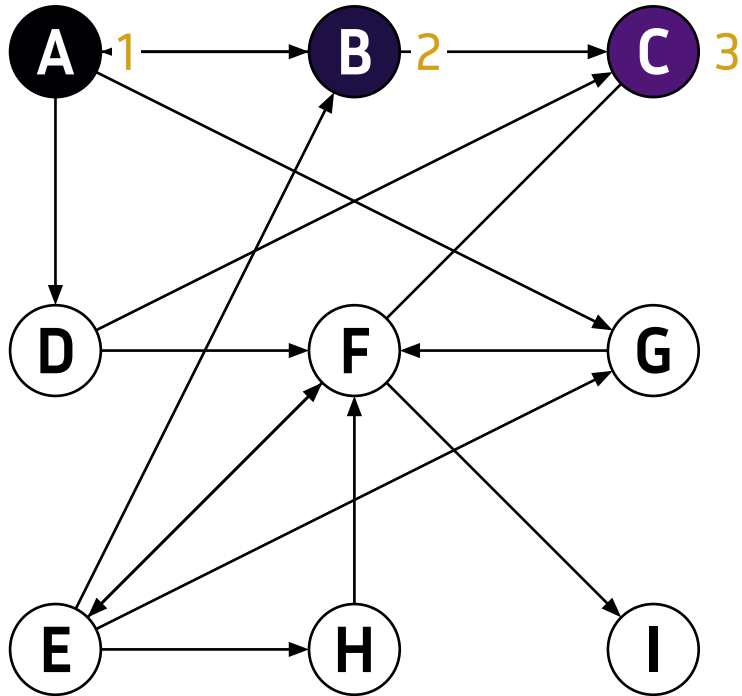
Visited: [A]

Graph Traversals – DFS Example



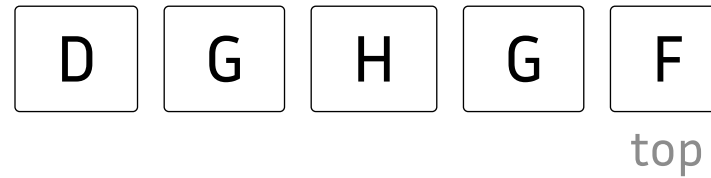
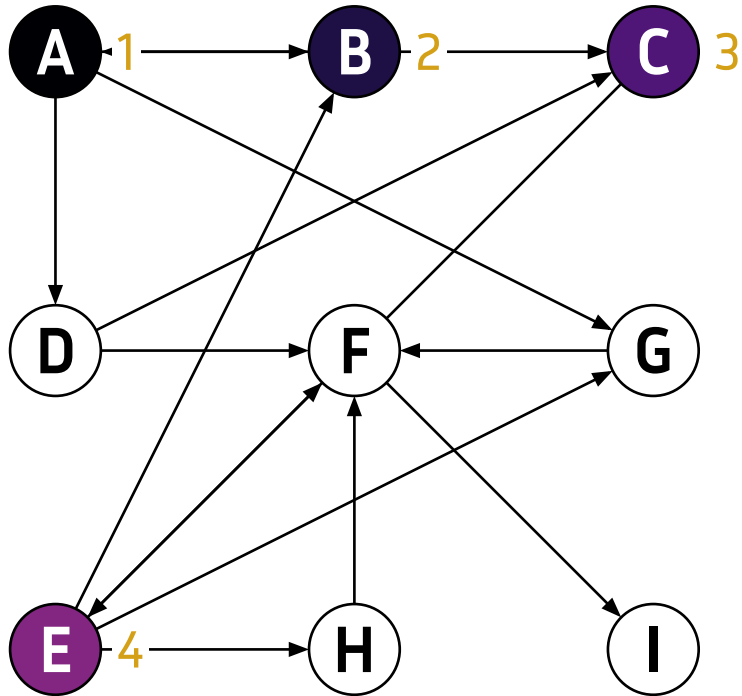
Visited: [A, B]

Graph Traversals – DFS Example



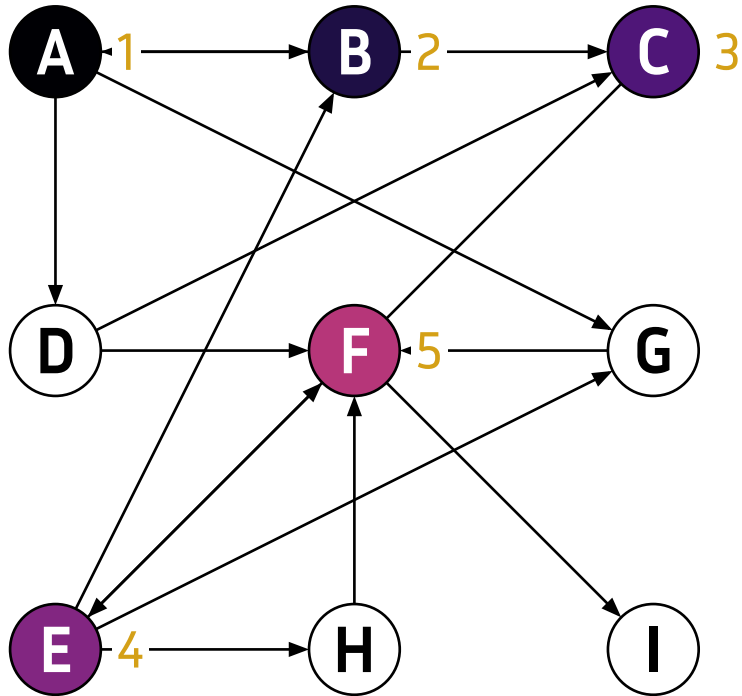
Visited: [A, B, C]

Graph Traversals – DFS Example



Visited: [A, B, C, E]

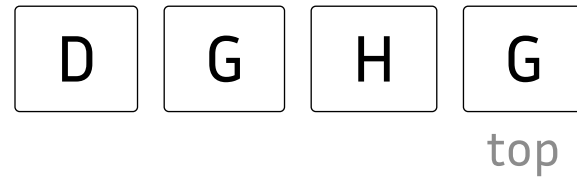
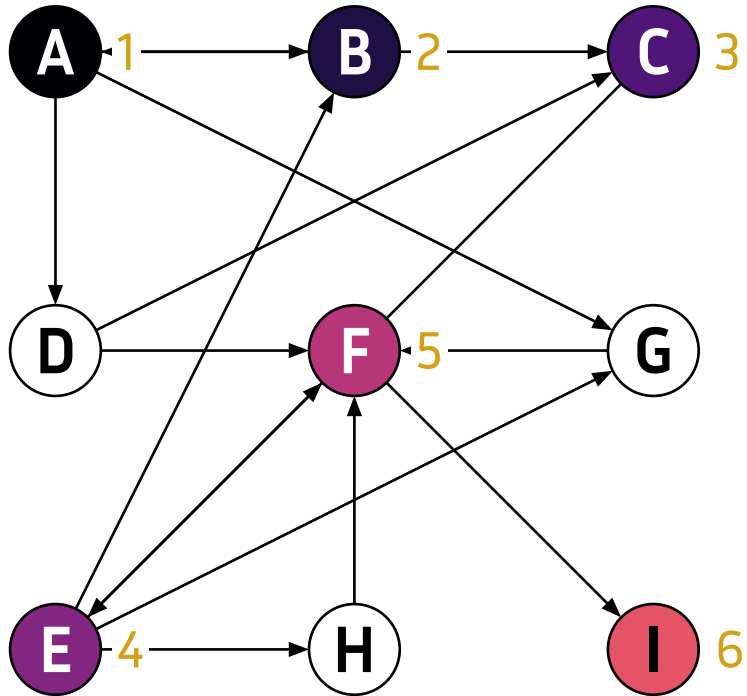
Graph Traversals – DFS Example



D G H G I
top

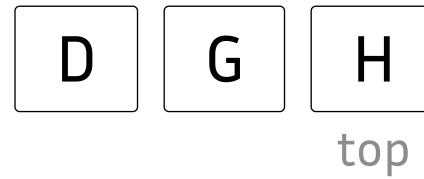
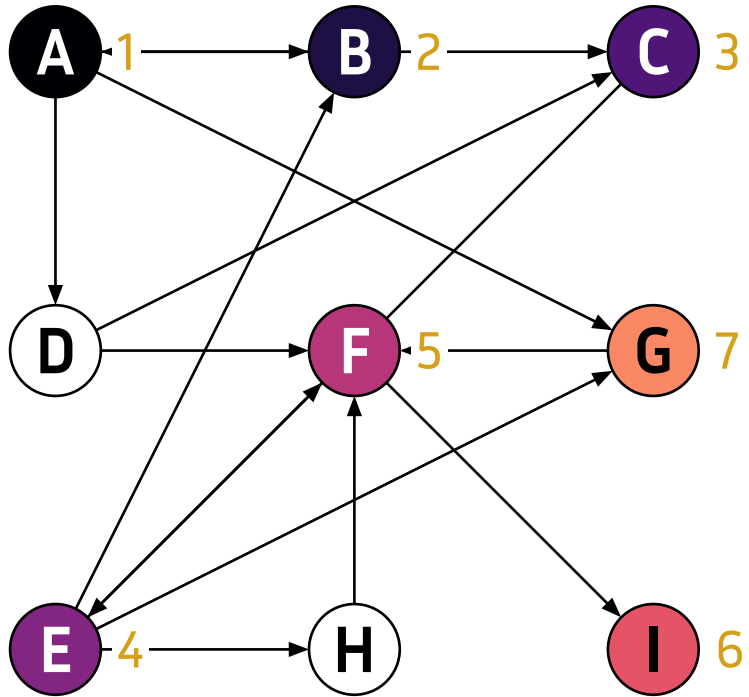
Visited: [A, B, C, E, F]

Graph Traversals – DFS Example



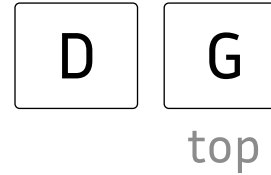
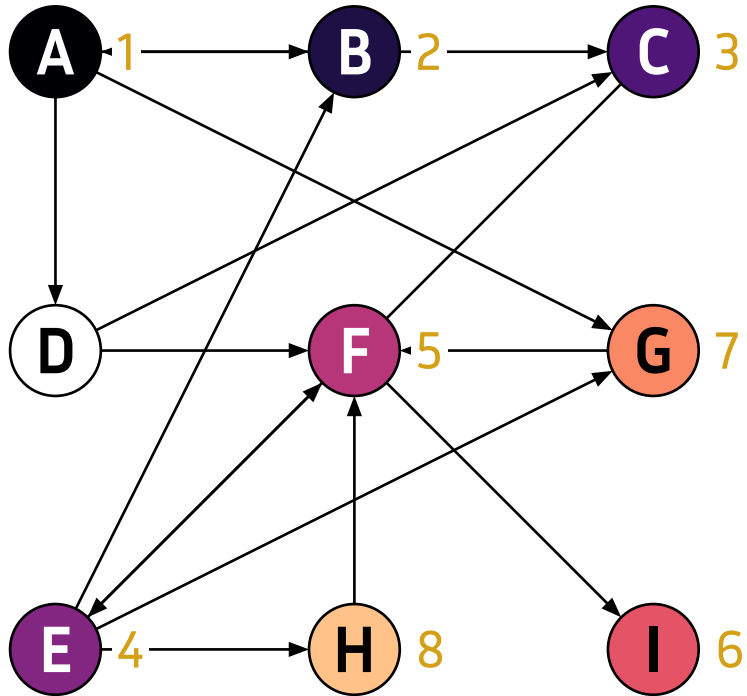
Visited: [A, B, C, E, F, I]

Graph Traversals – DFS Example



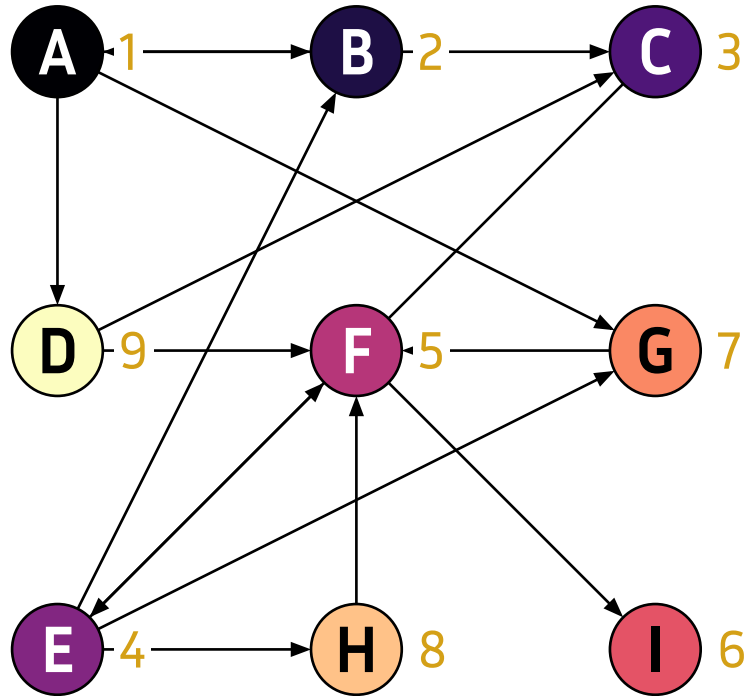
Visited: [A, B, C, E, F, I, G]

Graph Traversals – DFS Example



Visited: [A, B, C, E, F, I, G, H]

Graph Traversals – DFS Example



(empty)

top

Visited: [A, B, C, E, F, I, G, H, D]

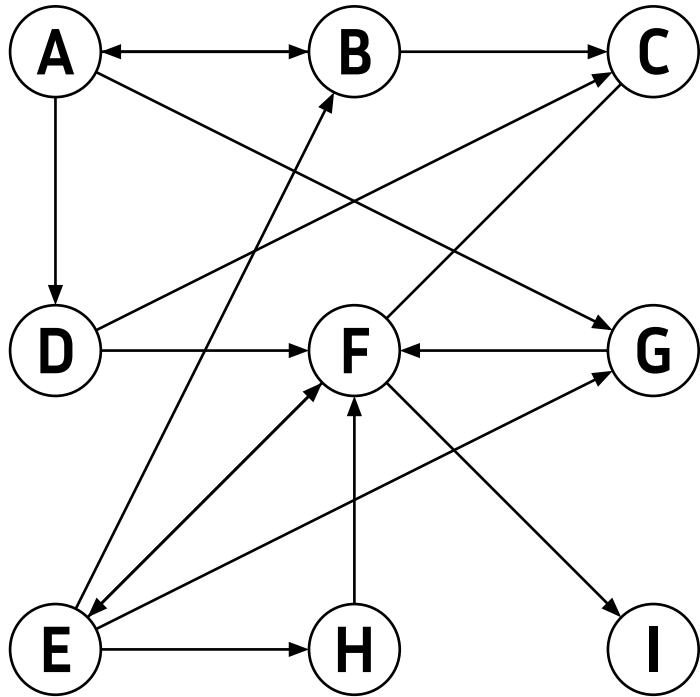
Graph Traversal – BFS

BFT(v):

```
1   $q \leftarrow$  queue
2  enqueue  $v$  and mark  $v$  visited
3  while  $q$  is not empty do
4      dequeue the next node  $w$ 
5      for each neighbor  $u$  of  $w$  do
6          if  $u$  is unvisited
7              mark  $u$  visited
8              enqueue  $u$ 
```

↖ Different order of neighbors lead to different visitation orders

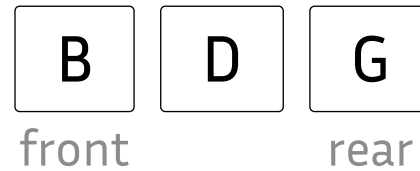
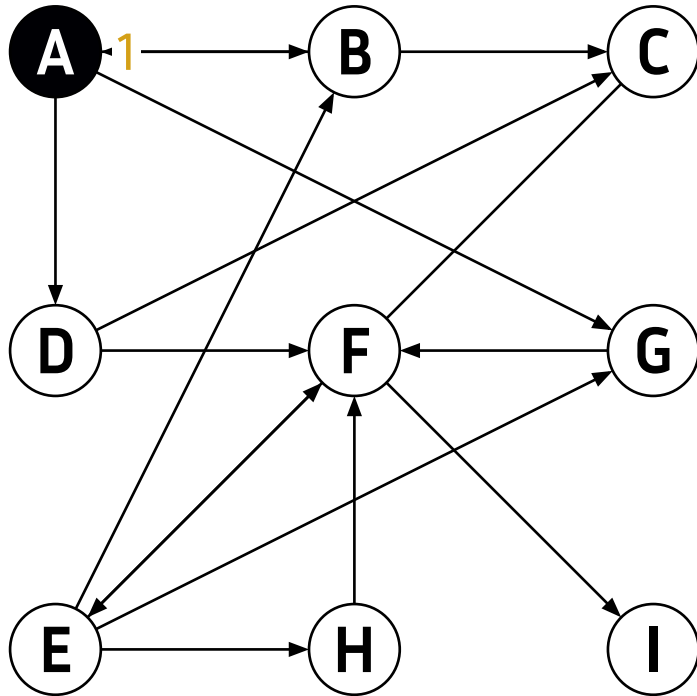
Graph Traversal – BFS Example



A

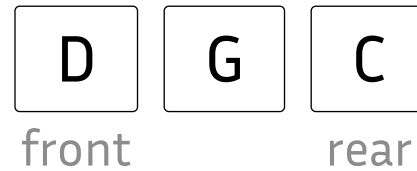
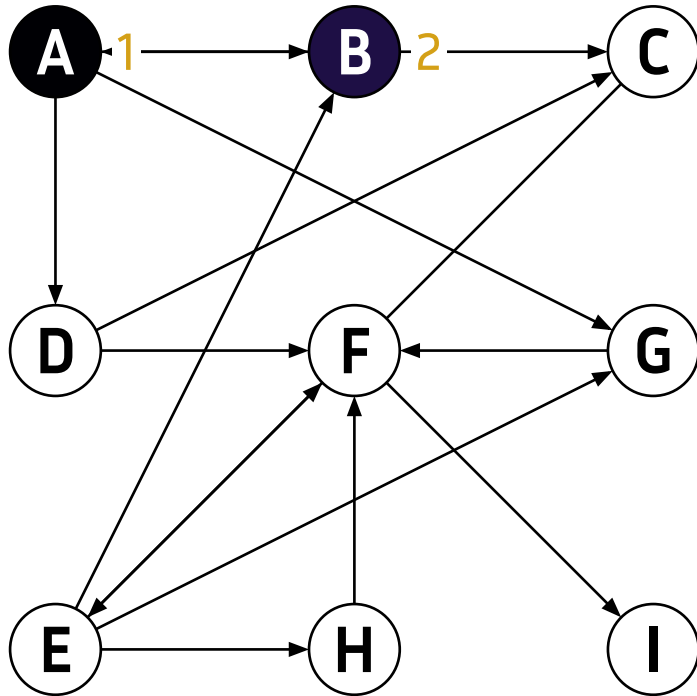
front / rear

Graph Traversal – BFS Example



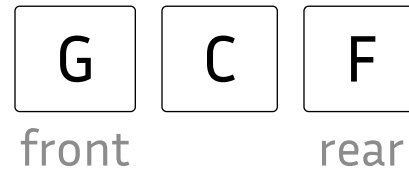
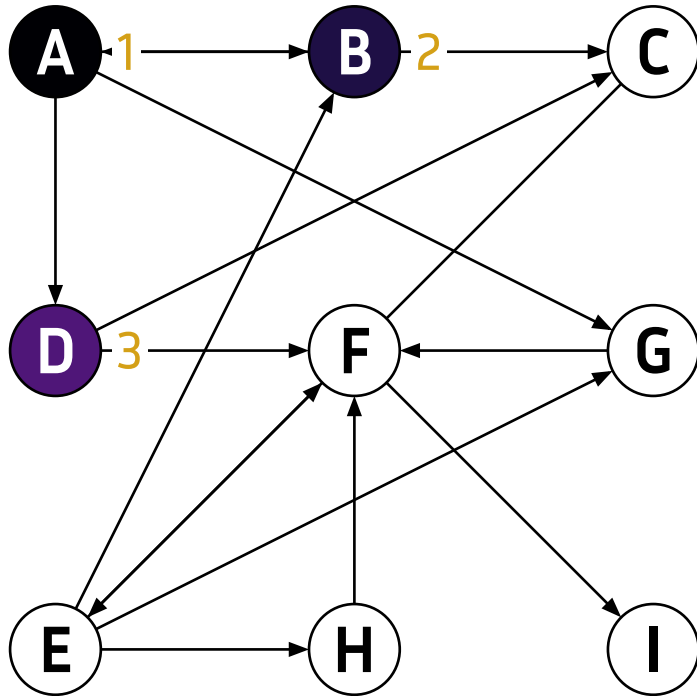
Visited: [A]

Graph Traversal – BFS Example



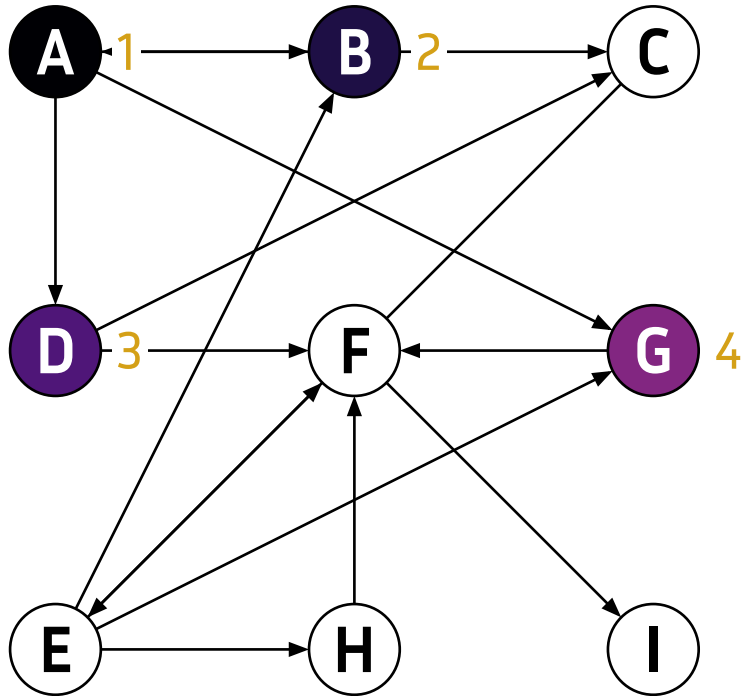
Visited: [A, B]

Graph Traversal – BFS Example



Visited: [A, B, D]

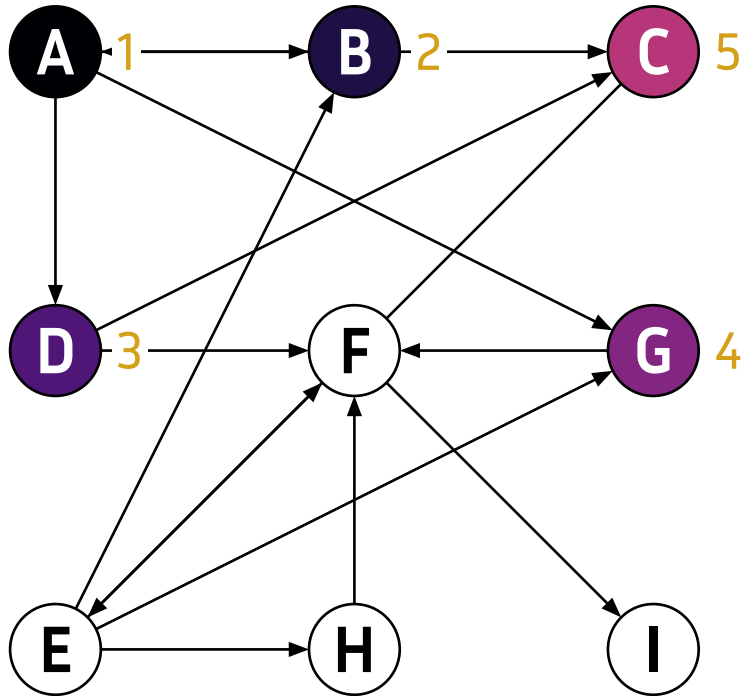
Graph Traversal – BFS Example



C	F
front	rear

Visited: [A, B, D, G]

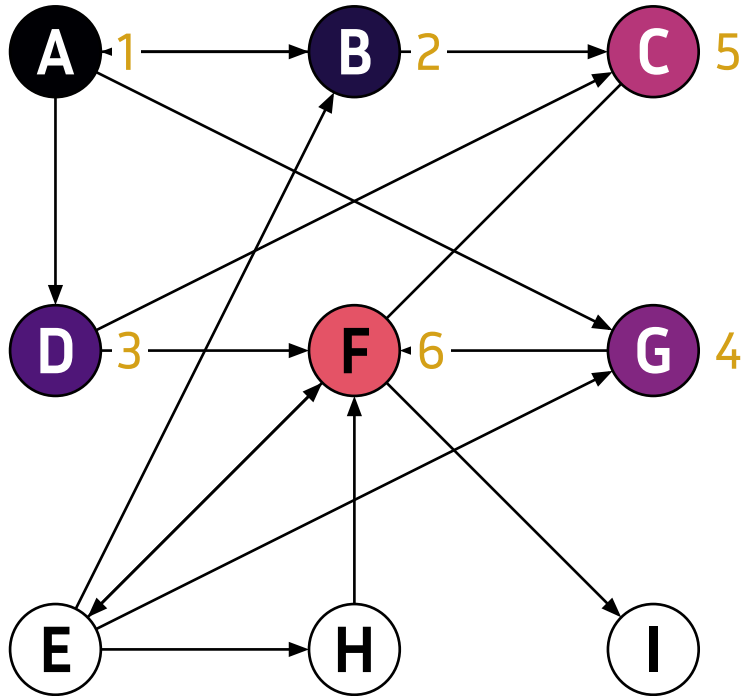
Graph Traversal – BFS Example



F	E
front	rear

Visited: [A, B, D, G, C]

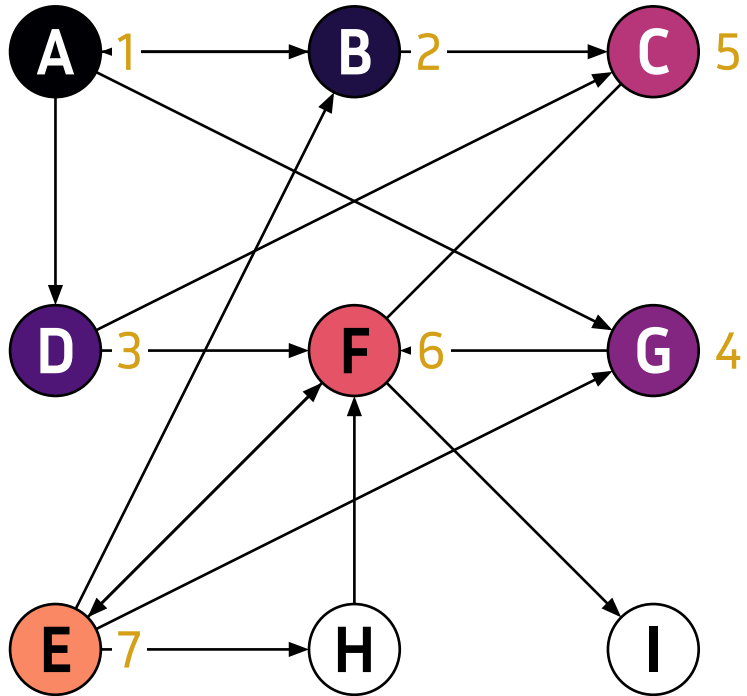
Graph Traversal – BFS Example



E	I
front	rear

Visited: [A, B, D, G, C, F]

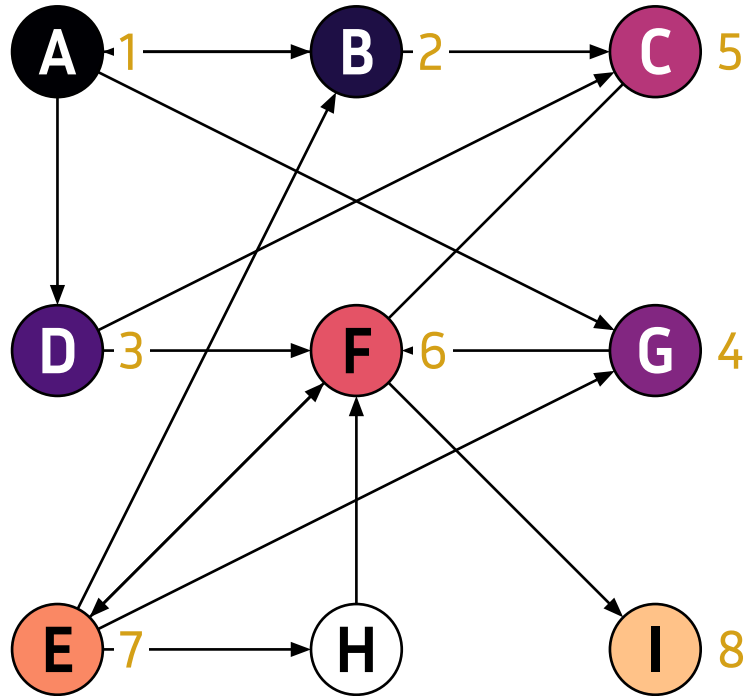
Graph Traversal – BFS Example



I	H
front	rear

Visited: [A, B, D, G, C, F, E]

Graph Traversal – BFS Example

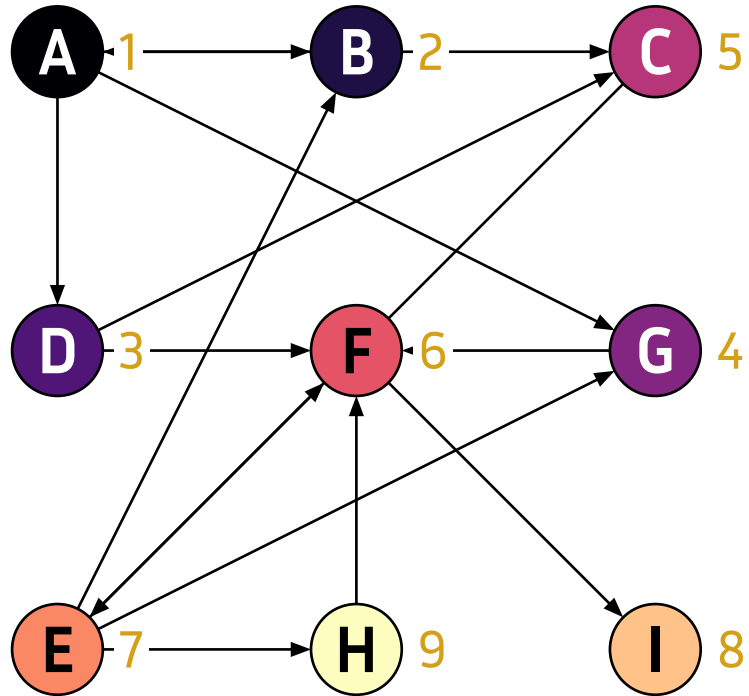


H

front / rear

Visited: [A, B, D, G, C, F, E, I]

Graph Traversal – BFS Example



(empty)

front

Visited: [A, B, D, G, C, F, E, I, H]

Graph Traversal – Complexity

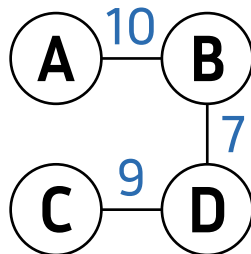
Traversal	Adjacency List	Adjacency Matrix
DFS	$O(V + E)$	$O(V^2)$
BFS	$O(V + E)$	$O(V^2)$

- ✦ Finding all neighbors in adjacency list is $O(\deg(x))$.
- ✦ Finding all neighbors in adjacency matrix is $O(V)$.

Minimum Spanning Trees

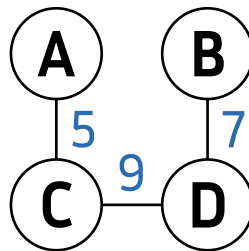
Definition (Minimum Spanning Tree) A minimum spanning tree T of a graph G is the spanning tree with the smallest sum of edge weights.

Option 1



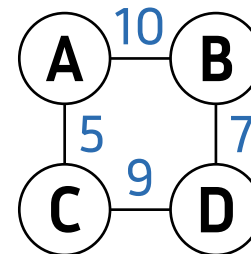
Cost: 26

Option 2



Cost: 21

Room Layout



MST – Prim's Algorithm

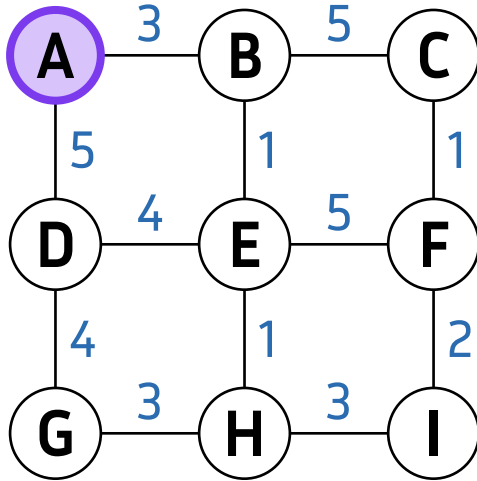
prim(v)

Implement ADT with heap



```
1   $q \leftarrow$  new priority queue
2  insert all edges of  $v$  in  $q$  and mark  $v$  as visited
3  while  $q$  is not empty:
4      remove minimum edge  $\{w, u\}$  from  $q$ 
5      if  $u$  is unvisited
6          add  $\{w, u\}$  to the spanning tree
7          insert all edges of  $u$  into  $q$ 
8          mark  $u$  as visited
```

MST – Prim's Example



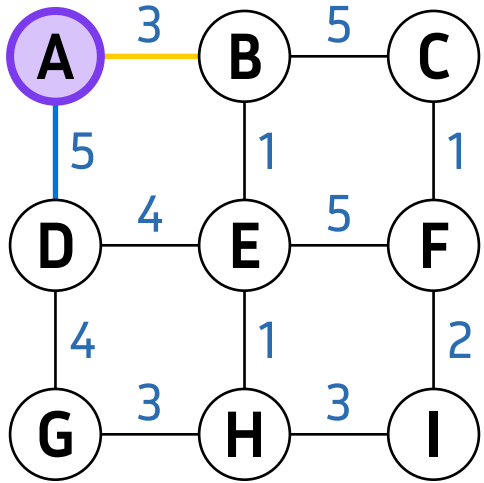
Start at A

min

A-B (3)

A-D (5)

MST – Prim's Example



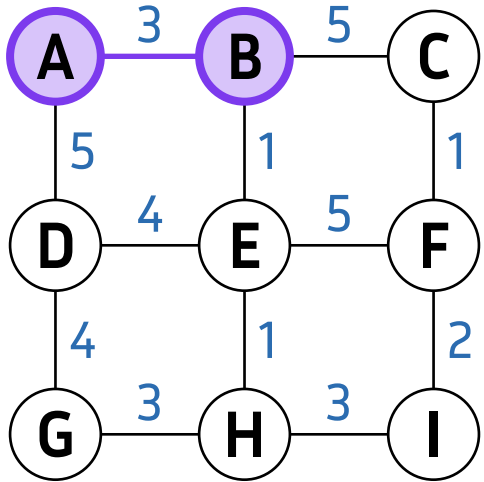
min

A–B (3)

A–D (5)

Min crossing edge: A–B (3)

MST – Prim's Example



Add A–B tree weight 3

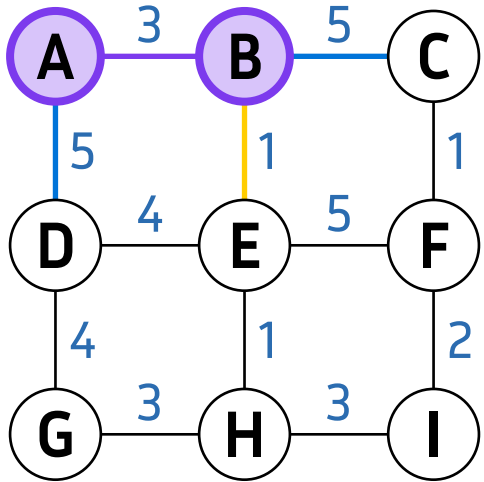
min

B–E (1)

A–D (5)

B–C (5)

MST – Prim's Example



min

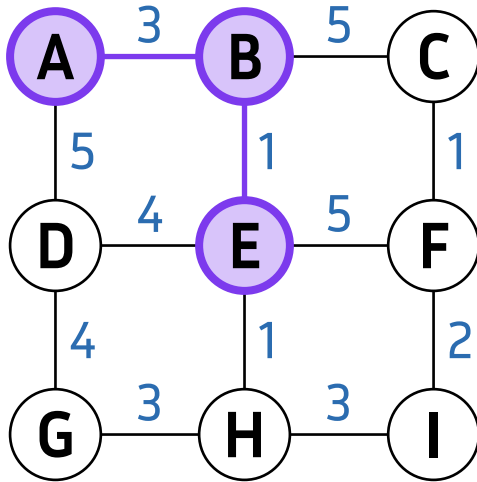
B-E (1)

A-D (5)

B-C (5)

Min crossing edge: B-E (1)

MST – Prim's Example



Add B–E tree weight 4

min

E–H (1)

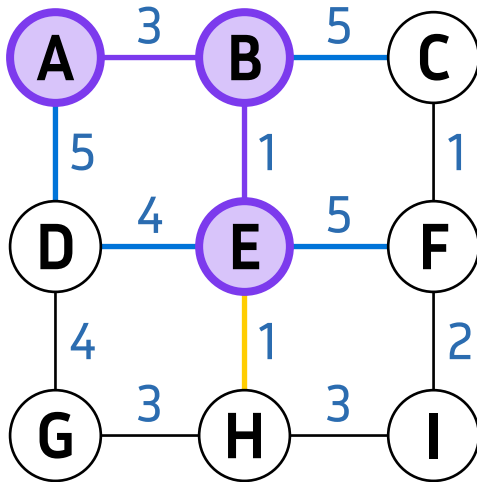
D–E (4)

A–D (5)

B–C (5)

E–F (5)

MST – Prim's Example



Min crossing edge: E–H (1)

min

E–H (1)

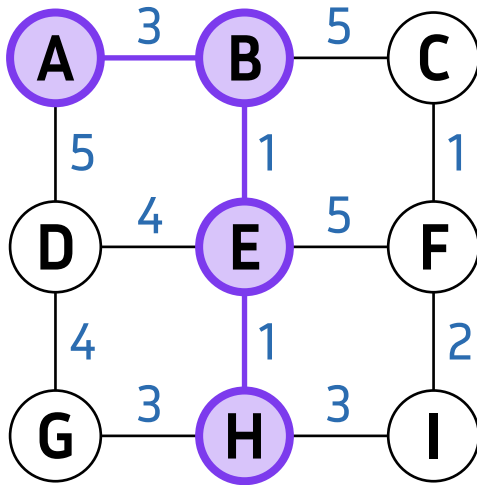
D–E (4)

A–D (5)

B–C (5)

E–F (5)

MST – Prim's Example



Add E–H tree weight 5

min

G–H (3)

H–I (3)

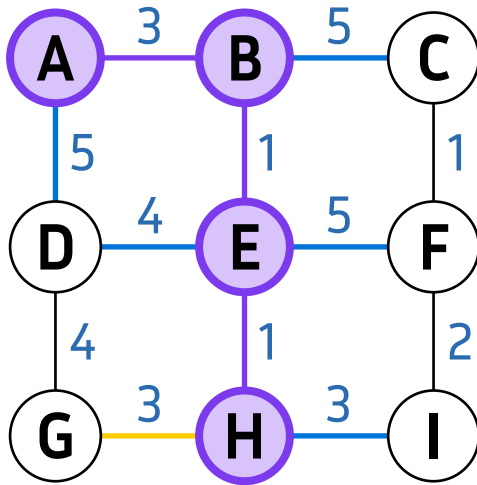
D–E (4)

A–D (5)

B–C (5)

E–F (5)

MST – Prim's Example



Min crossing edge: G–H (3)

min

G–H (3)

H–I (3)

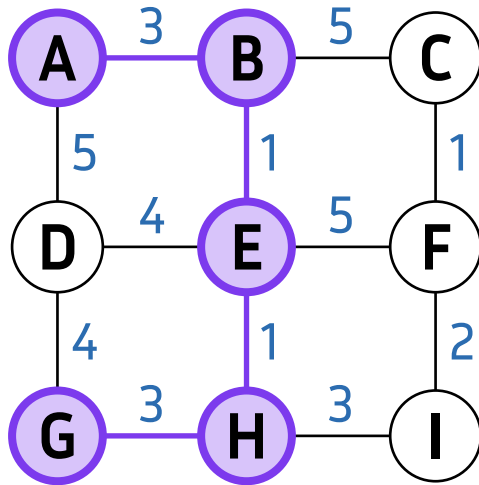
D–E (4)

A–D (5)

B–C (5)

E–F (5)

MST – Prim's Example



Add G–H tree weight 8

min

H–I (3)

D–E (4)

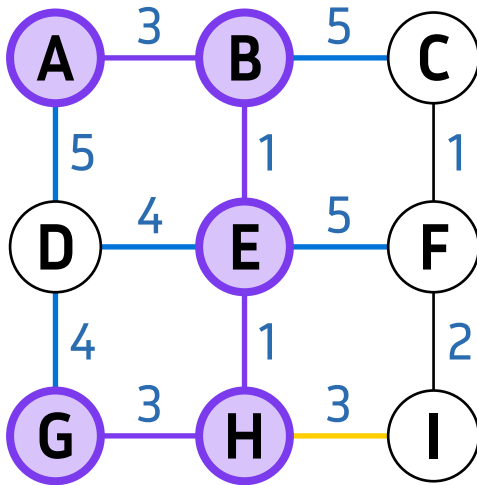
D–G (4)

A–D (5)

B–C (5)

E–F (5)

MST – Prim's Example



Min crossing edge: H-I (3)

min

H-I (3)

D-E (4)

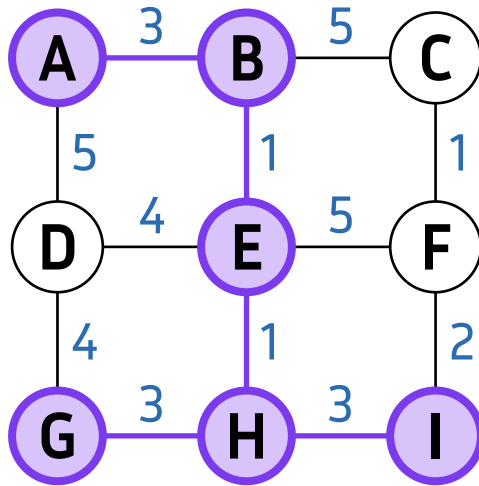
D-G (4)

A-D (5)

B-C (5)

E-F (5)

MST – Prim's Example



Add H–I tree weight 11

min

F–I (2)

D–E (4)

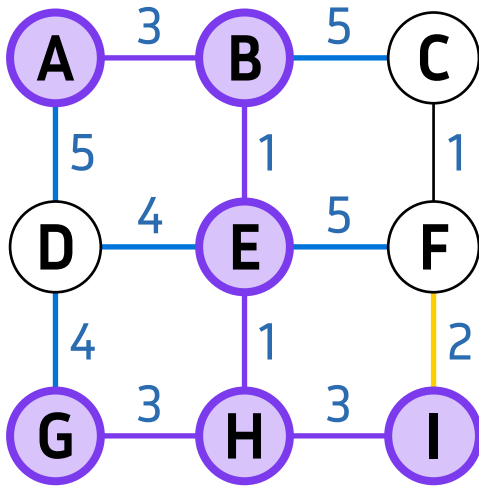
D–G (4)

A–D (5)

B–C (5)

E–F (5)

MST – Prim's Example



Min crossing edge: F-I (2)

min

F-I (2)

D-E (4)

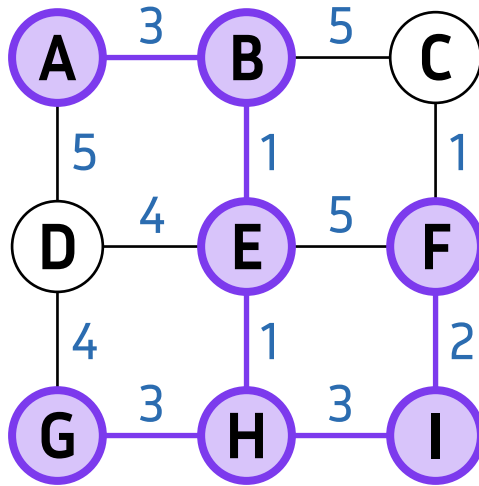
D-G (4)

A-D (5)

B-C (5)

E-F (5)

MST – Prim's Example



Add F–I tree weight 13

min

C–F (1)

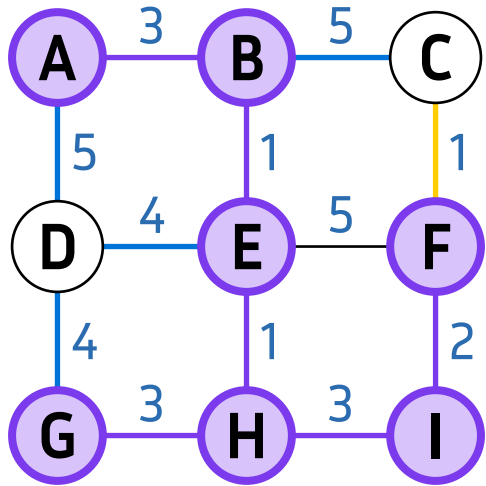
D–E (4)

D–G (4)

A–D (5)

B–C (5)

MST – Prim's Example



Min crossing edge: C–F (1)

min

C–F (1)

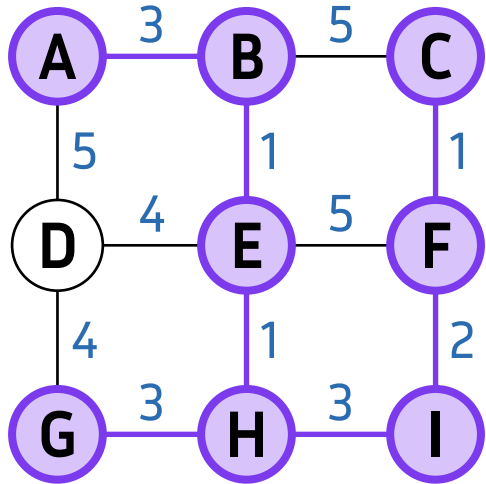
D–E (4)

D–G (4)

A–D (5)

B–C (5)

MST – Prim's Example



min

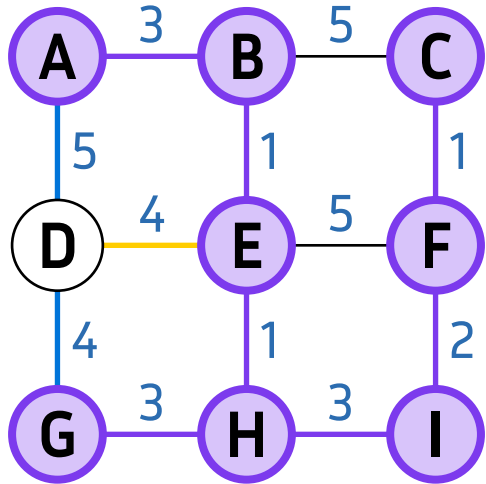
D–E (4)

D–G (4)

A–D (5)

Add C–F tree weight 14

MST – Prim's Example



min

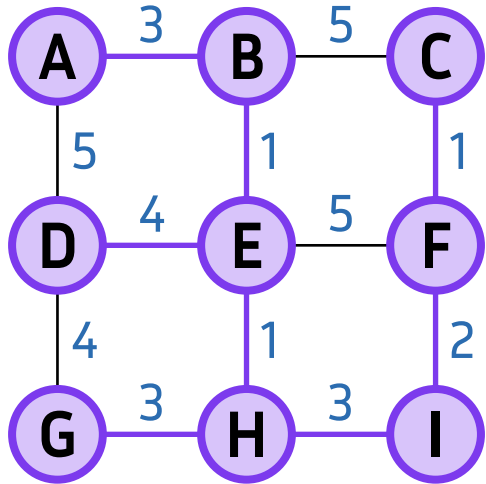
D–E (4)

D–G (4)

A–D (5)

Min crossing edge: D–E (4)

MST – Prim's Example

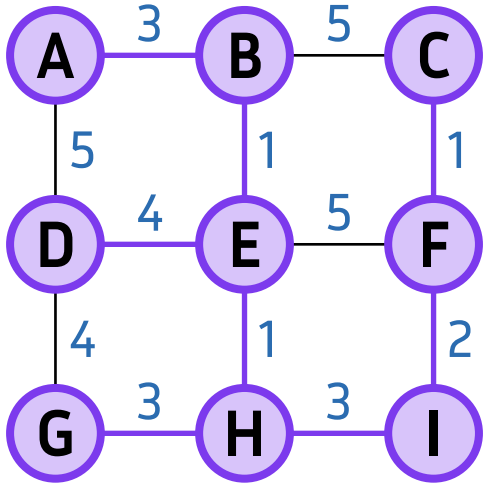


Add D–E tree weight 18

min

(empty)

MST – Prim's Example

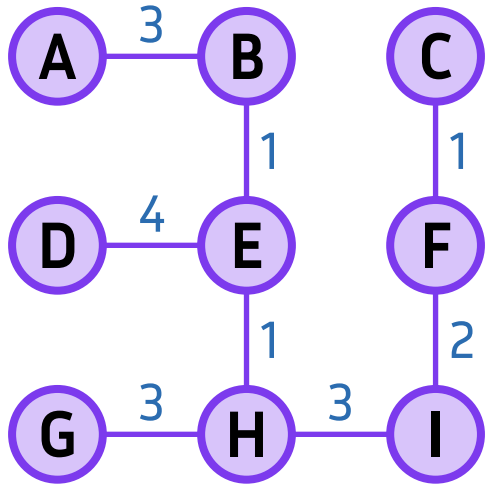


MST weight 18

min

(empty)

MST – Prim's Example



Spanning tree

min

(empty)

MST – Kruskal's Algorithm

kruskal(V, L)

```
1  sort edge list  $L$ 
2   $s \leftarrow [\{v_1\}, \{v_2\}, \dots, \{v_n\}]$ 
3  while  $L$  is not empty:
4      remove  $\{w, u\}$  from  $L$ 
5      if  $u$  and  $w$  are not in the same set in  $s$ :
6          add  $\{w, u\}$  to the spanning tree
7          join the sets for  $u$  and  $w$  in  $s$ 
```

MST – Kruskal's Algorithm

kruskal(V, L)

1 sort **edge list** L

2 $s \leftarrow [\{v_1\}, \{v_2\}, \dots, \{v_n\}]$

3 while L is not empty:

Union-find data structure



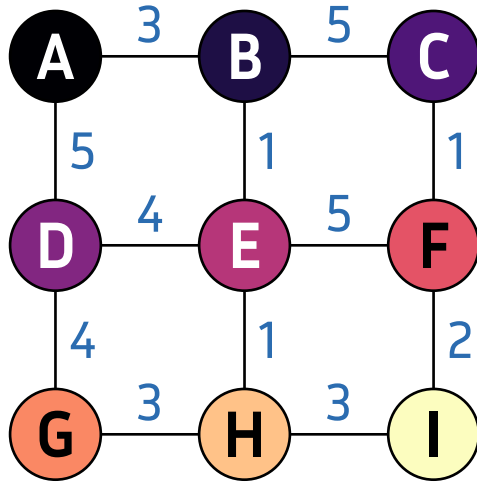
4 **remove** $\{w, u\}$ from L

5 if u and w are **not** in the same set in s :

6 add $\{w, u\}$ to the spanning tree

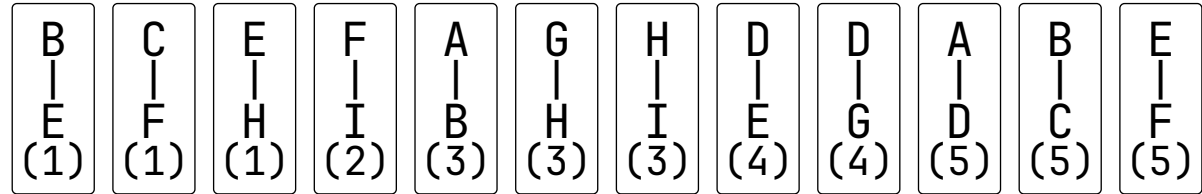
7 **join** the sets for u and w in s

MST – Kruskal's Example

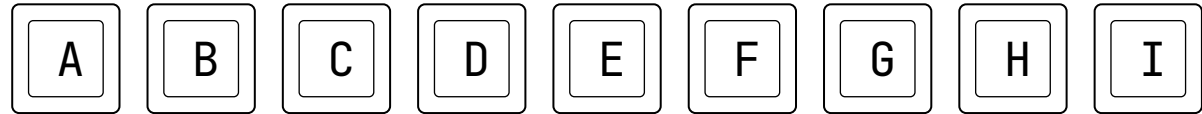


Sort edges by weight

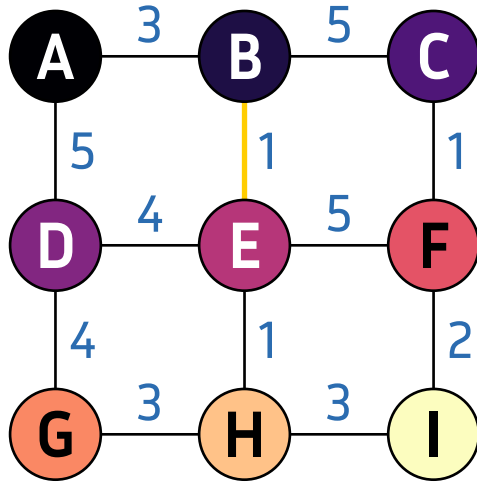
Sorted edges



Components

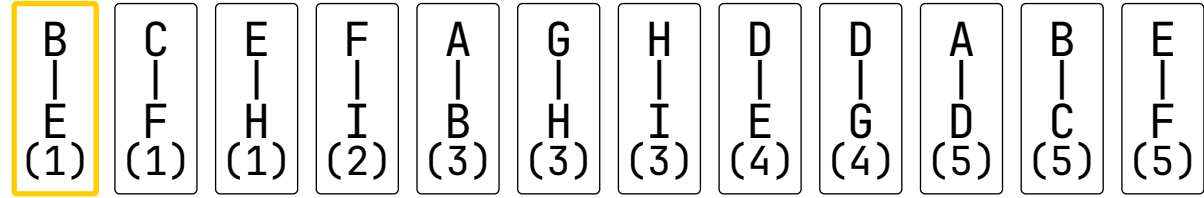


MST – Kruskal's Example

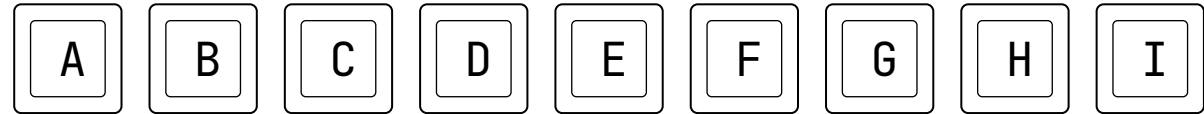


Consider B–E (1)

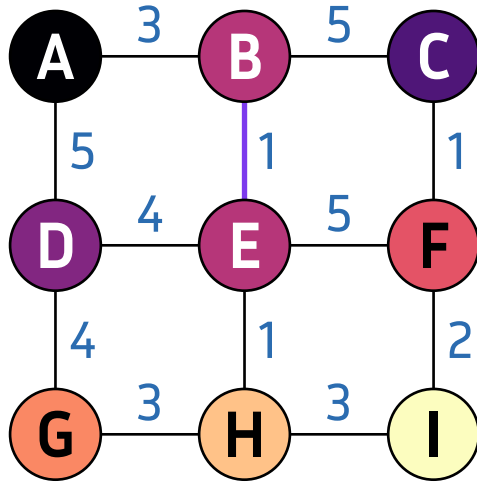
Sorted edges



Components

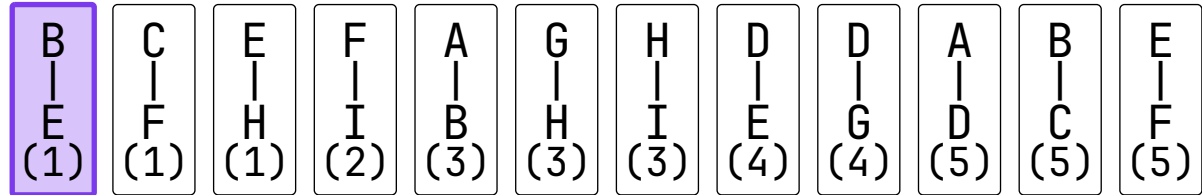


MST – Kruskal's Example

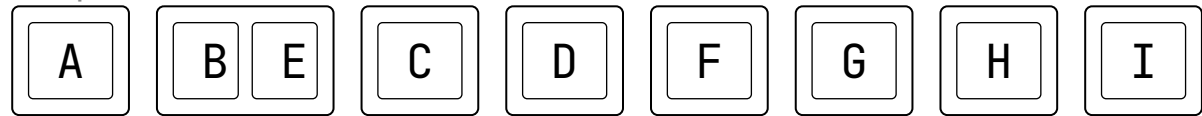


Add B–E weight 1

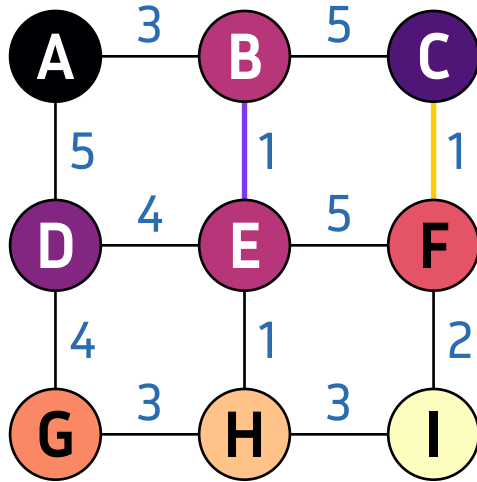
Sorted edges



Components

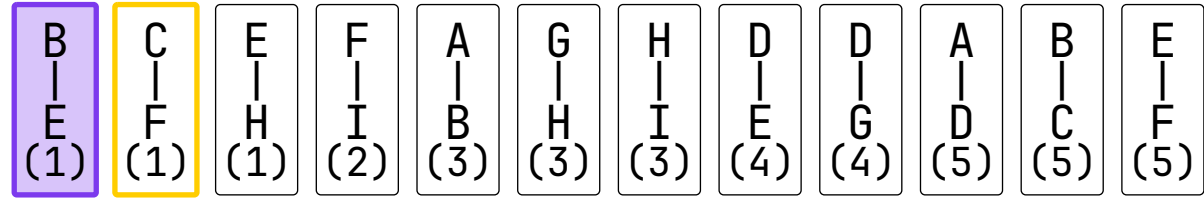


MST – Kruskal's Example

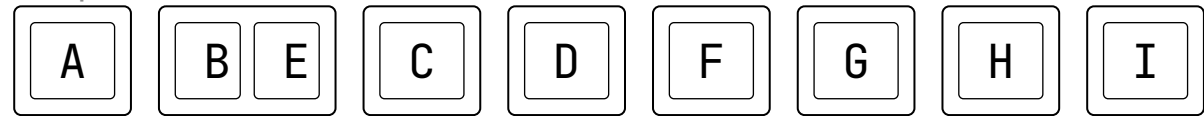


Consider C–F (1)

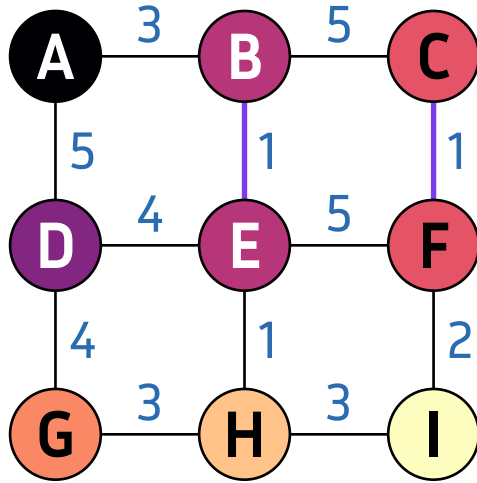
Sorted edges



Components

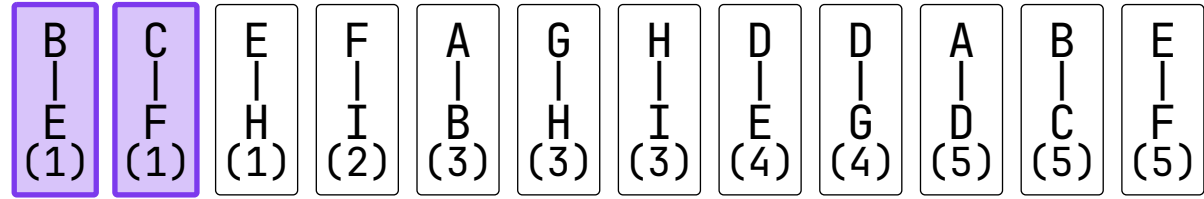


MST – Kruskal's Example

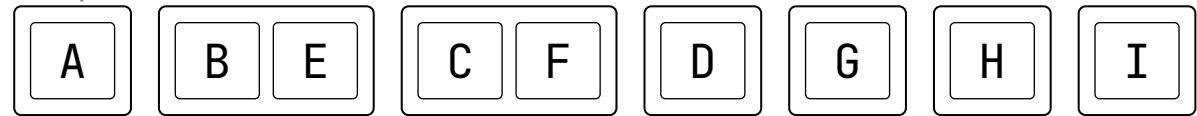


Add C–F weight 2

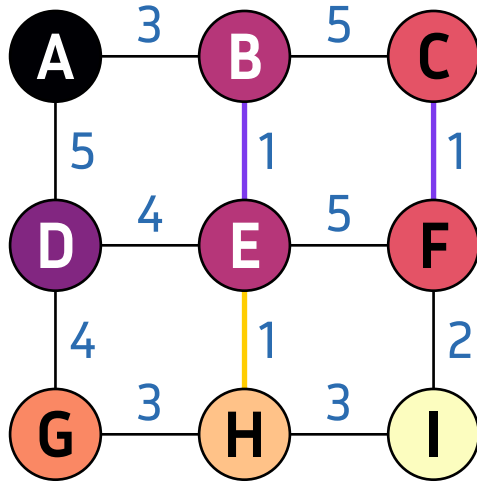
Sorted edges



Components

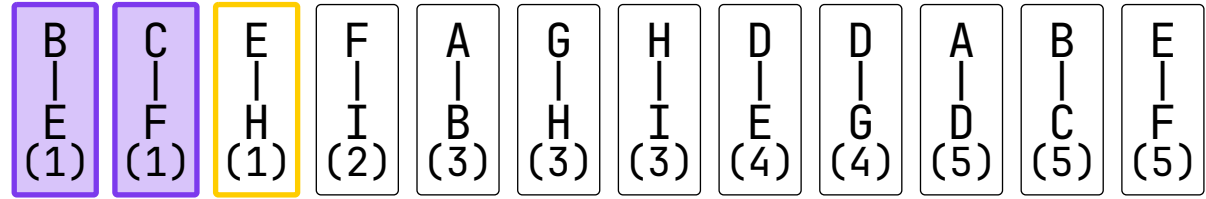


MST – Kruskal's Example

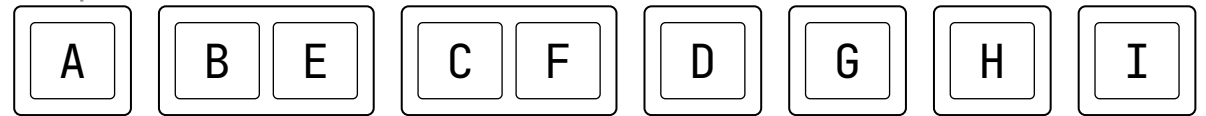


Consider E–H (1)

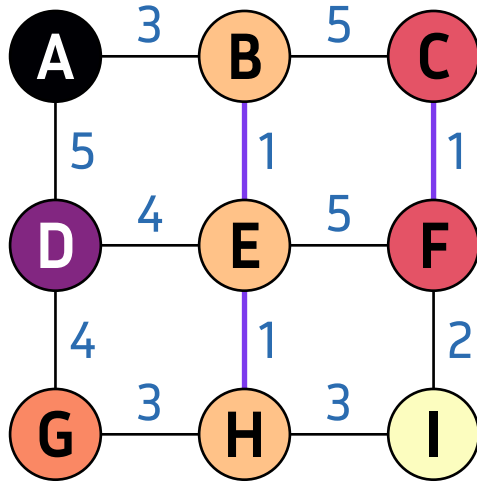
Sorted edges



Components

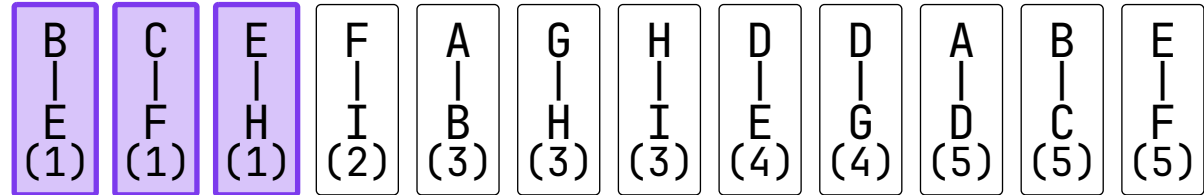


MST – Kruskal's Example

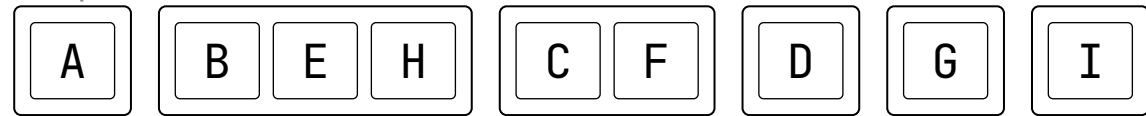


Add E–H weight 3

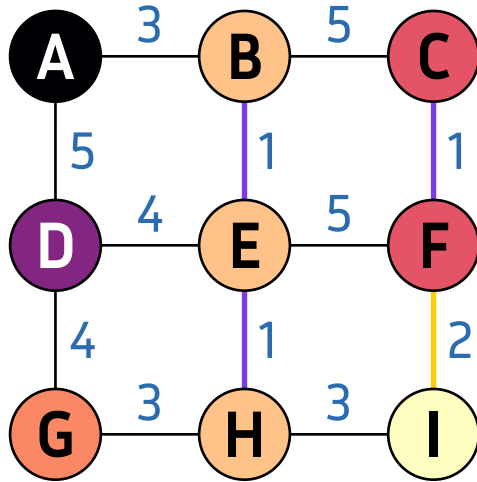
Sorted edges



Components

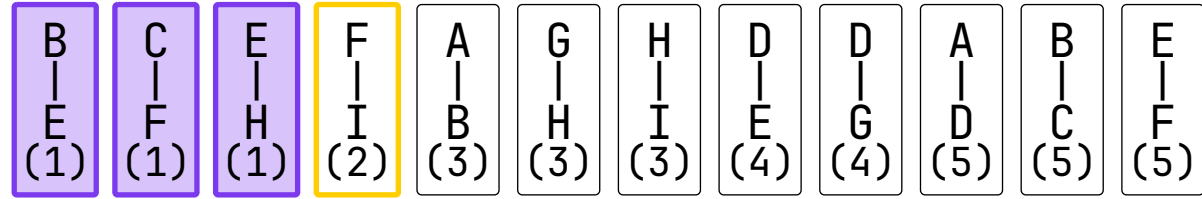


MST – Kruskal's Example

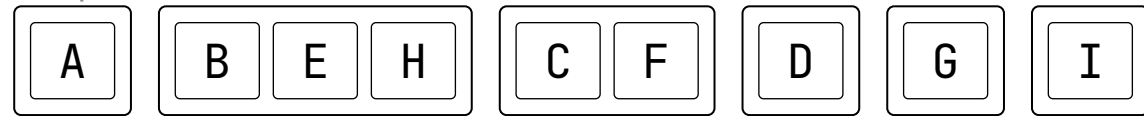


Consider F-I (2)

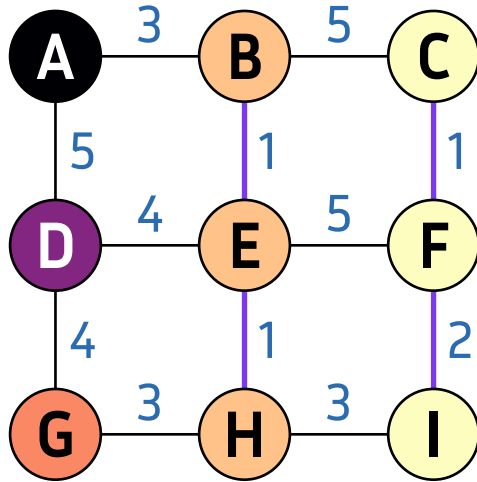
Sorted edges



Components

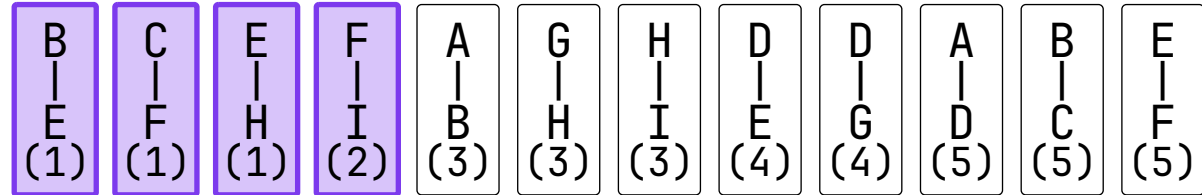


MST – Kruskal's Example

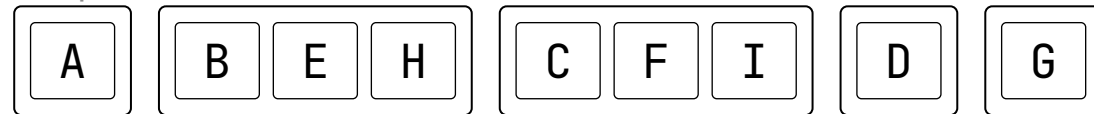


Add F–I weight 5

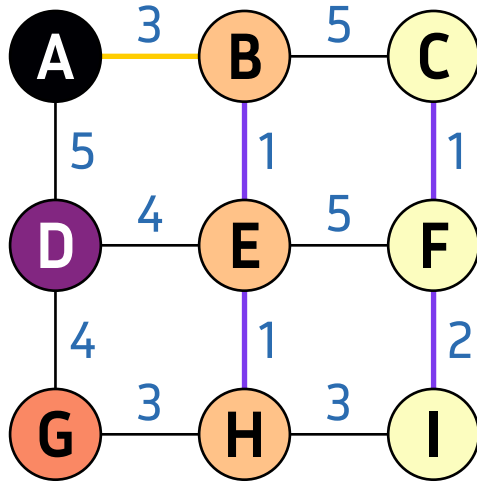
Sorted edges



Components

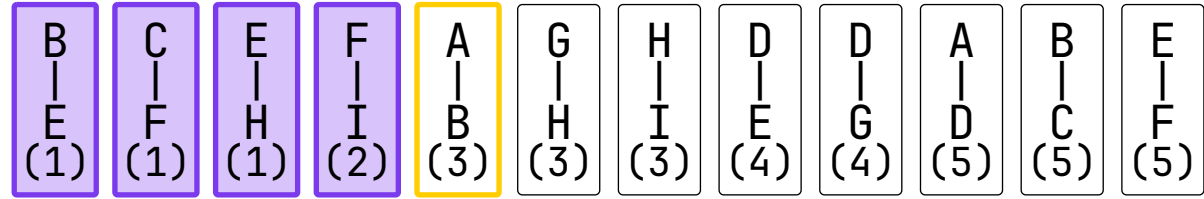


MST – Kruskal's Example

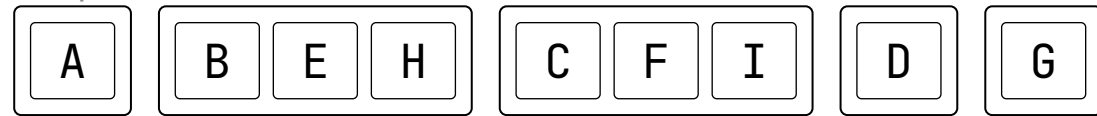


Consider A–B (3)

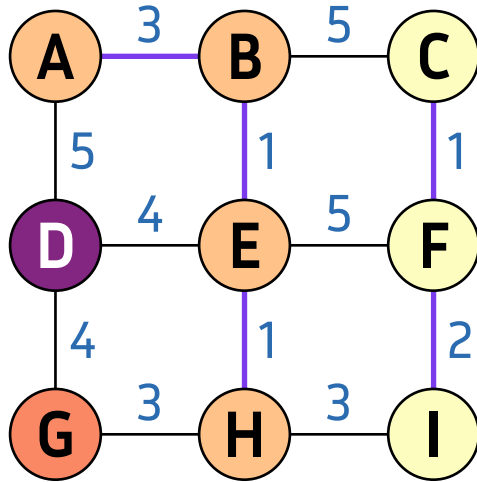
Sorted edges



Components

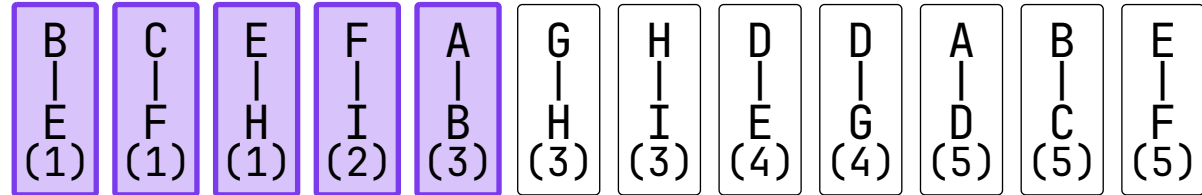


MST – Kruskal's Example

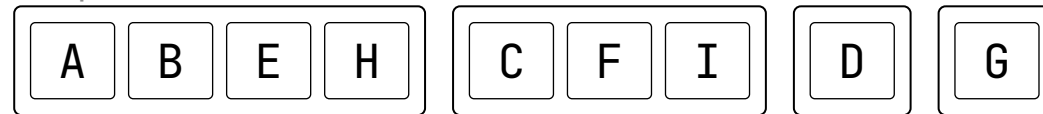


Add A–B weight 8

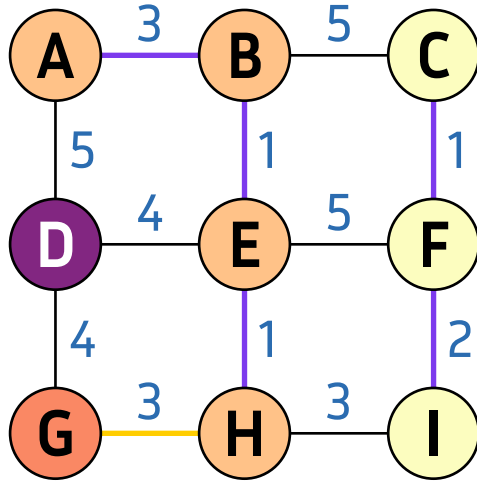
Sorted edges



Components

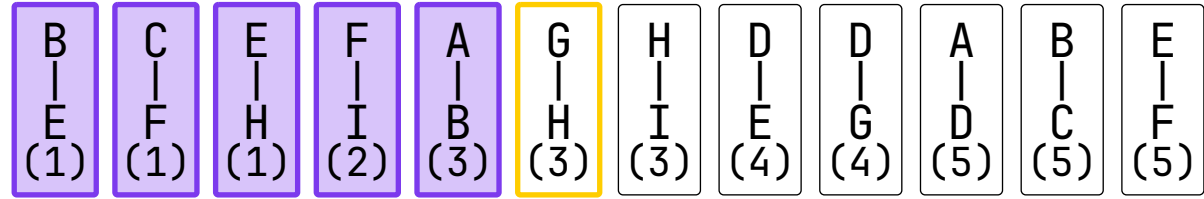


MST – Kruskal's Example

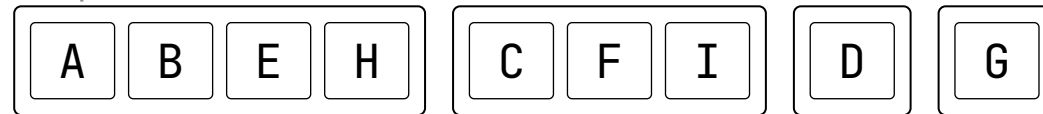


Consider G–H (3)

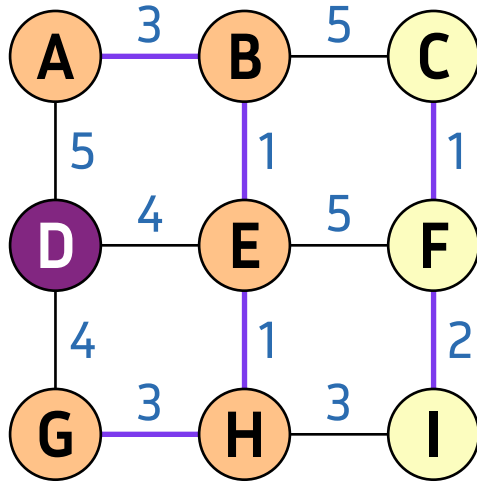
Sorted edges



Components

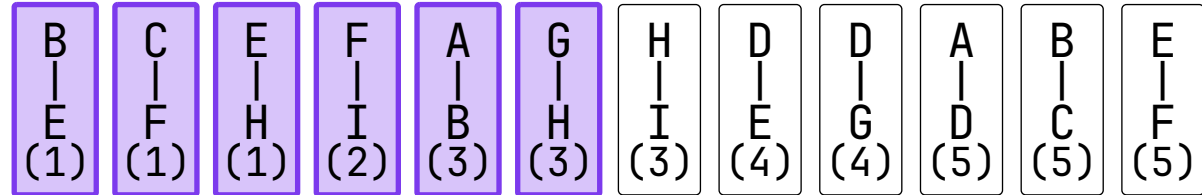


MST – Kruskal's Example

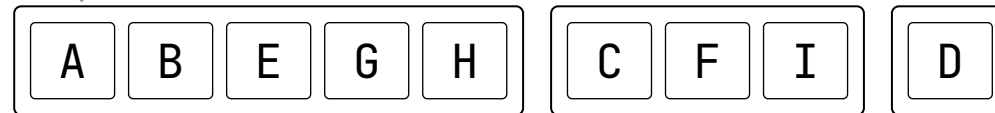


Add G–H weight 11

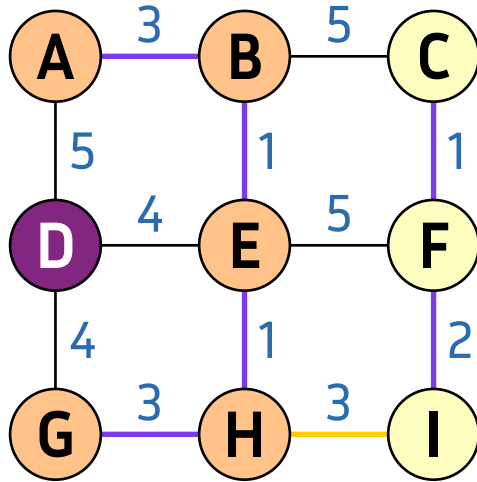
Sorted edges



Components

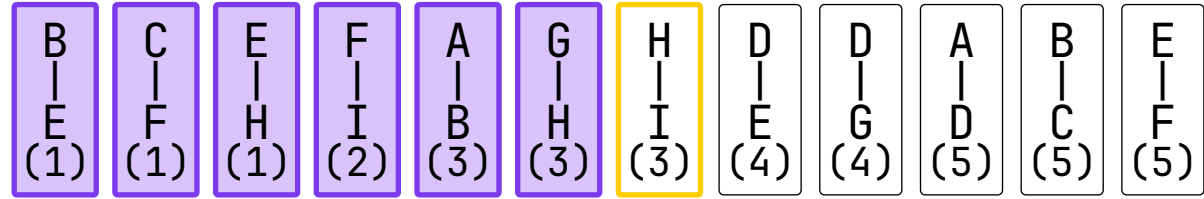


MST – Kruskal's Example

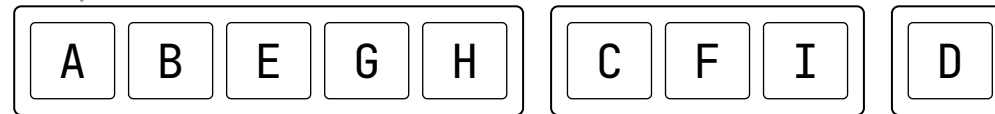


Consider H–I (3)

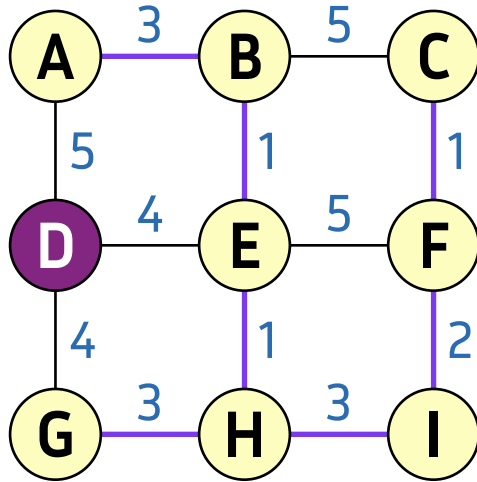
Sorted edges



Components

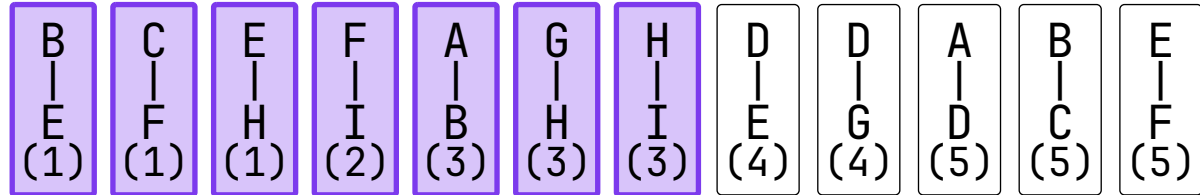


MST – Kruskal's Example

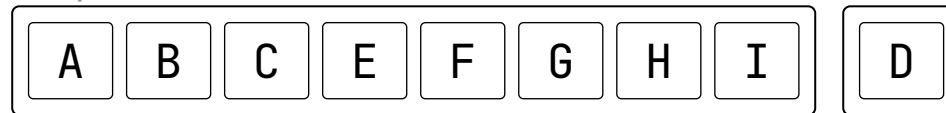


Add H-I weight 14

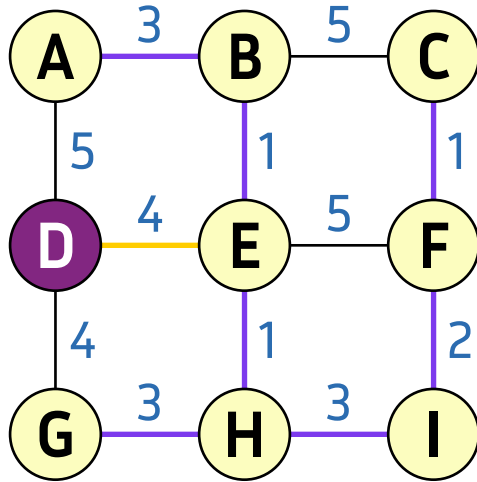
Sorted edges



Components

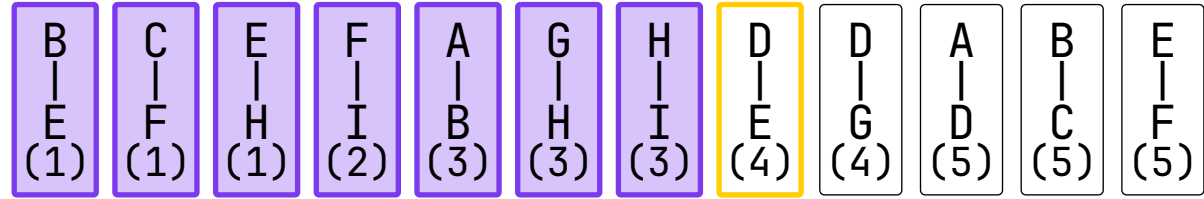


MST – Kruskal's Example

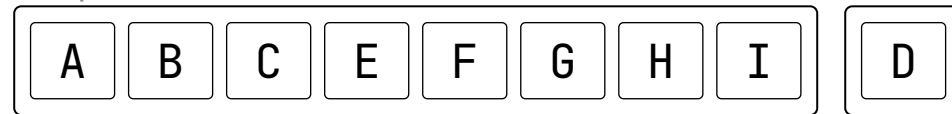


Consider D–E (4)

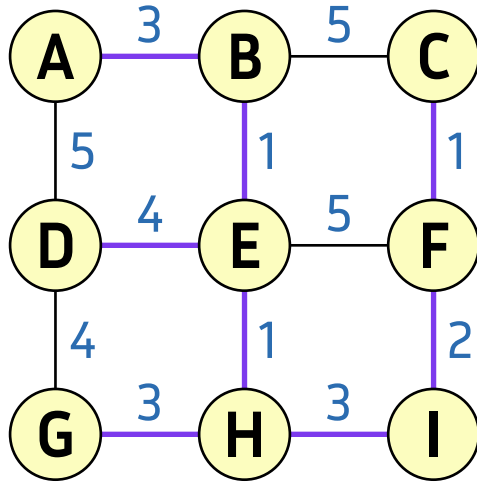
Sorted edges



Components

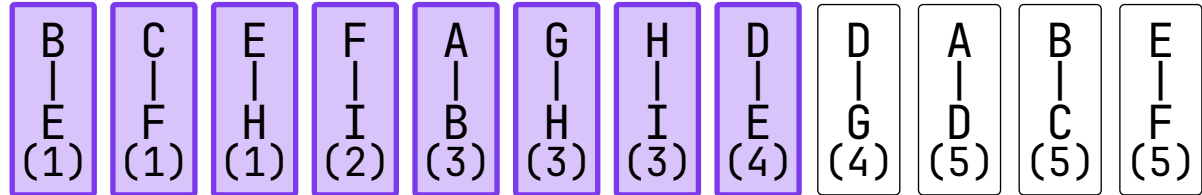


MST – Kruskal's Example

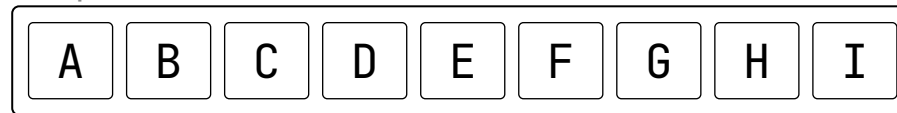


Add D–E weight 18

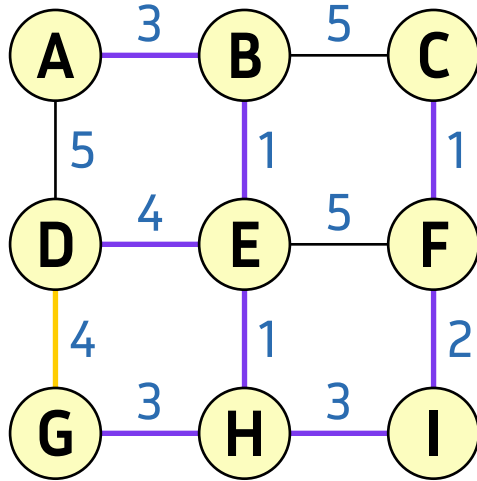
Sorted edges



Components

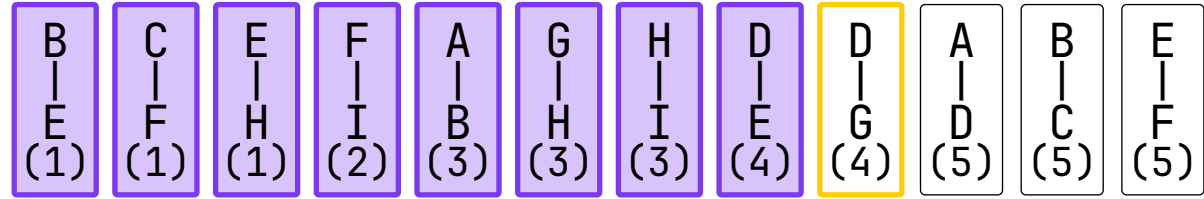


MST – Kruskal's Example

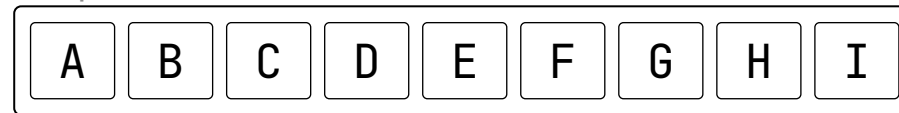


Consider D–G (4)

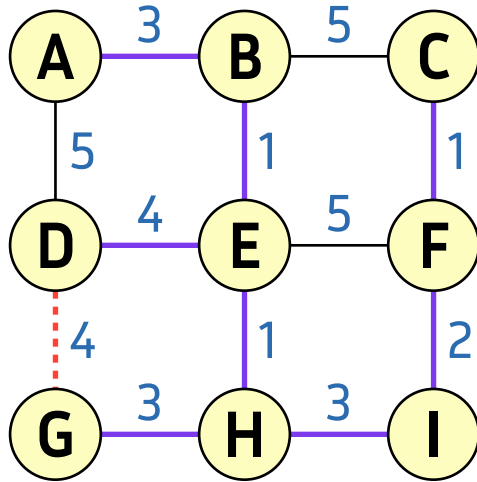
Sorted edges



Components

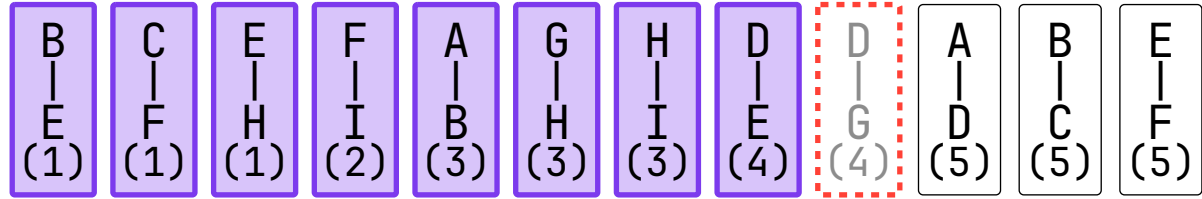


MST – Kruskal's Example

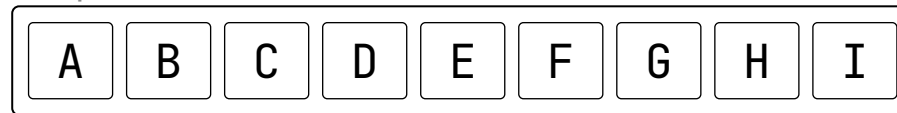


Reject D–G (cycle)

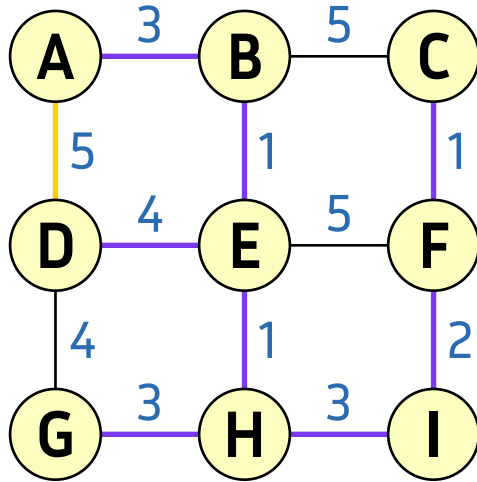
Sorted edges



Components

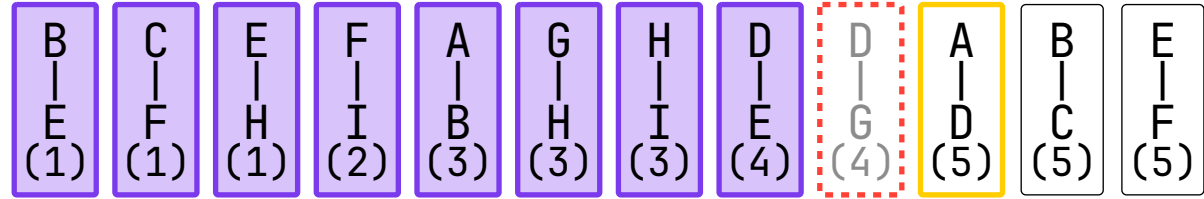


MST – Kruskal's Example

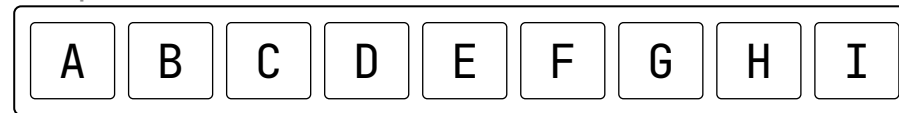


Consider A–D (5)

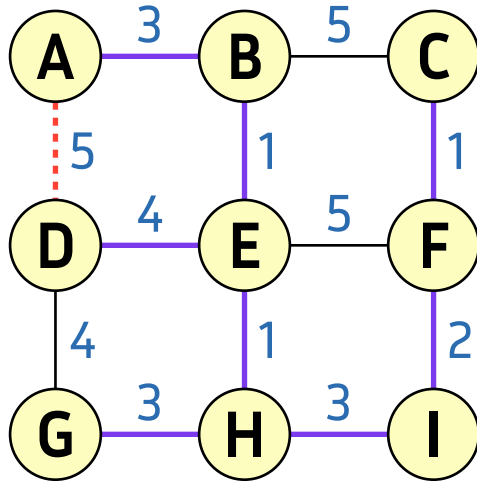
Sorted edges



Components

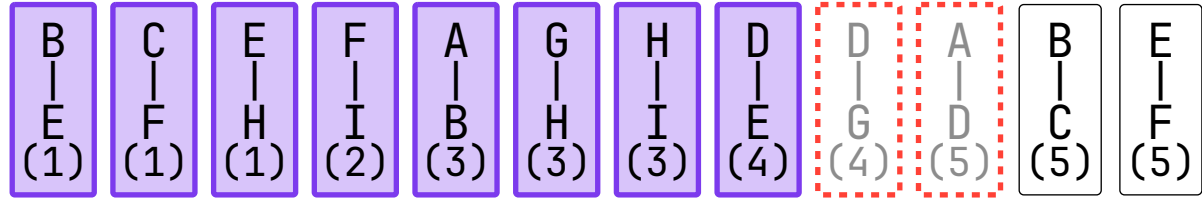


MST – Kruskal's Example

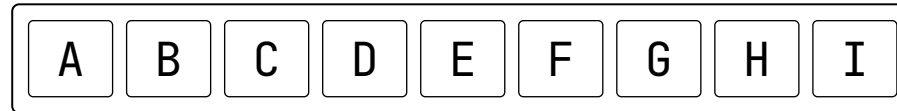


Reject A–D (cycle)

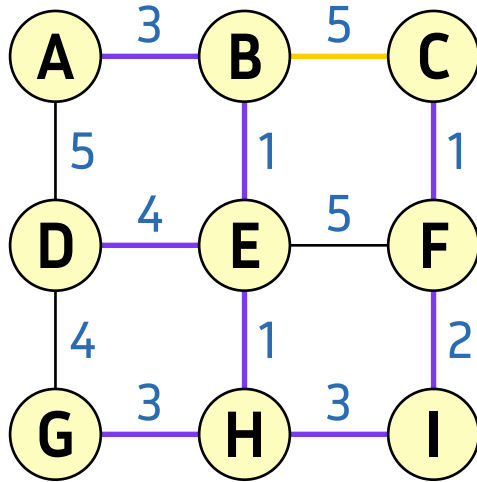
Sorted edges



Components

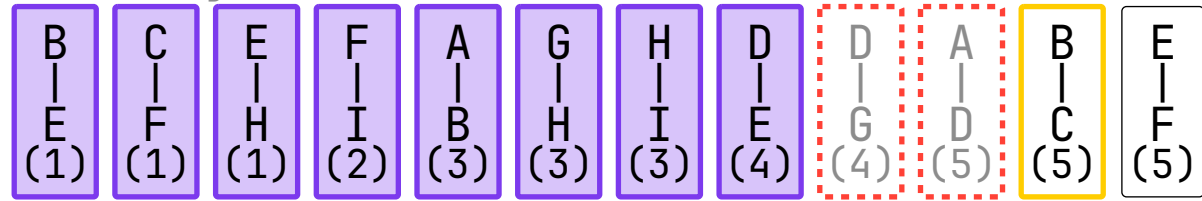


MST – Kruskal's Example

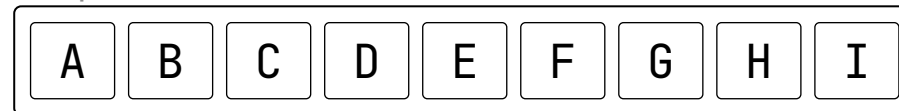


Consider B–C (5)

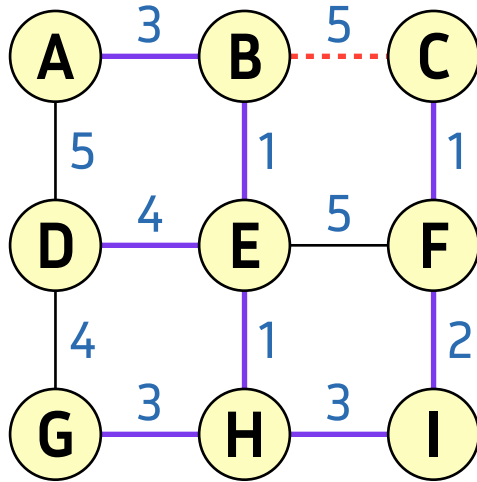
Sorted edges



Components

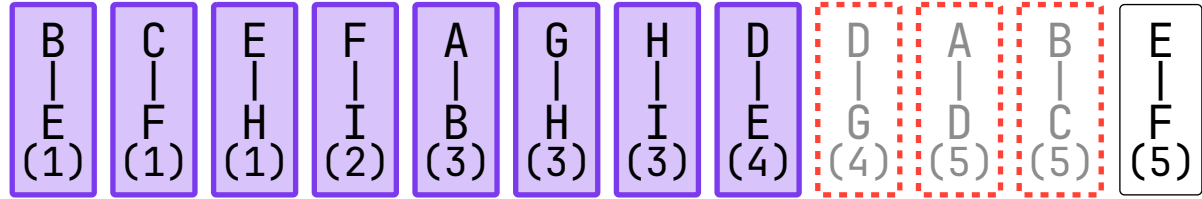


MST – Kruskal's Example

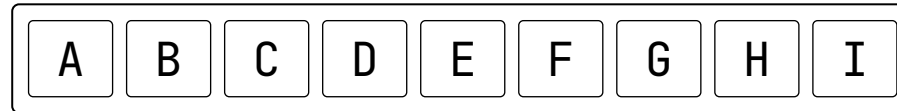


Reject B–C (cycle)

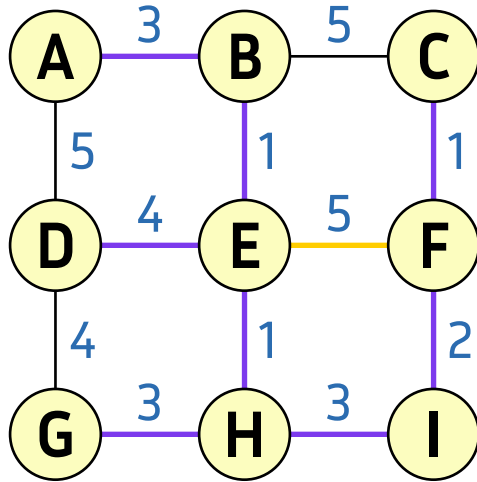
Sorted edges



Components

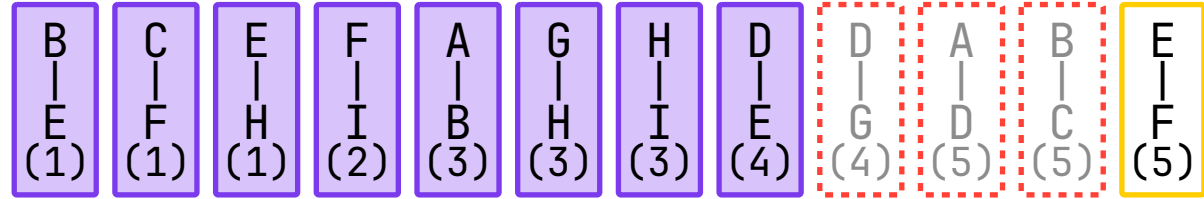


MST – Kruskal's Example

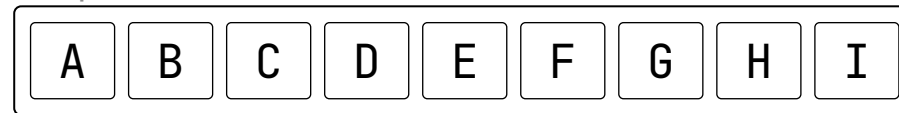


Consider E–F (5)

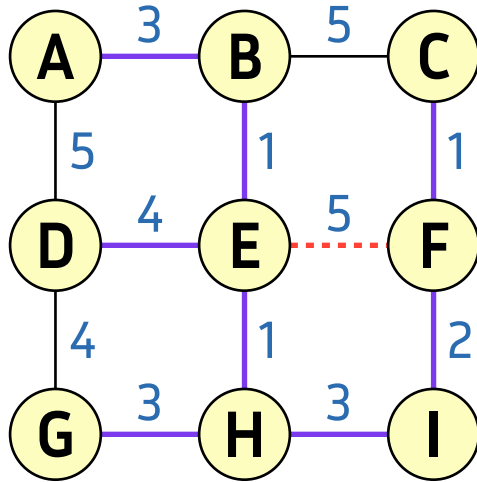
Sorted edges



Components

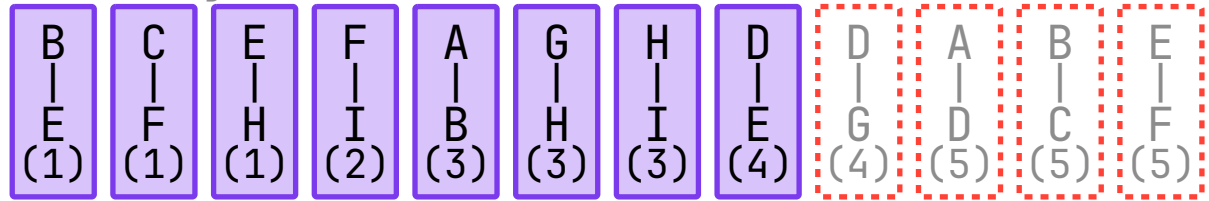


MST – Kruskal's Example

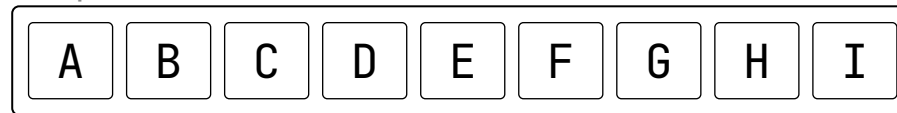


Reject E–F (cycle)

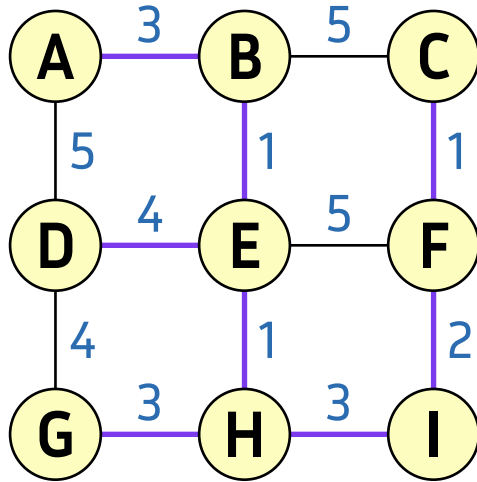
Sorted edges



Components

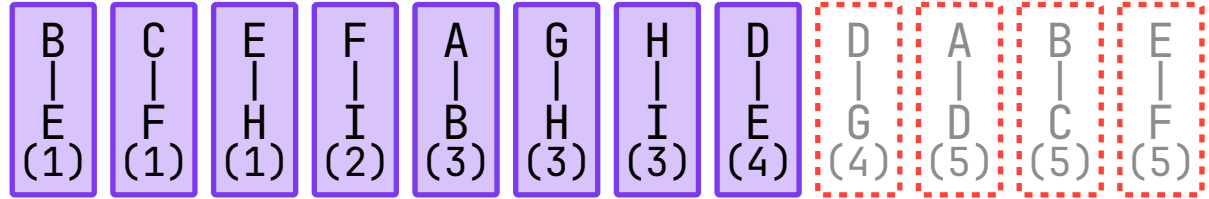


MST – Kruskal's Example

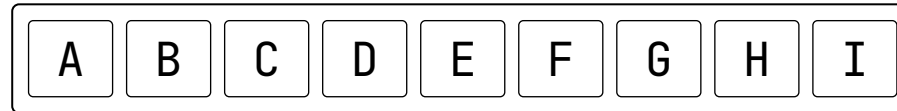


MST weight 18

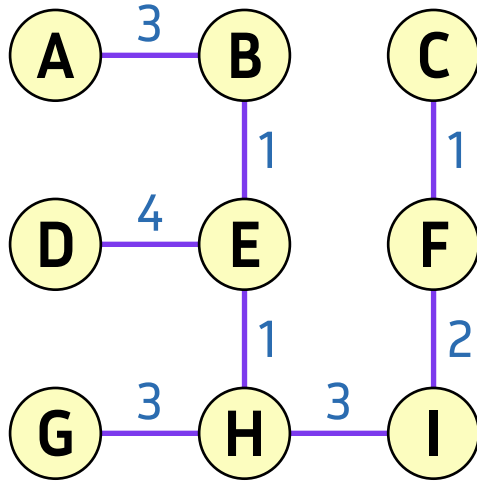
Sorted edges



Components

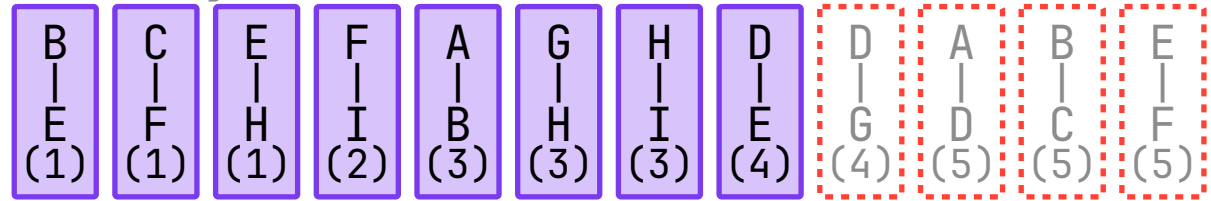


MST – Kruskal's Example

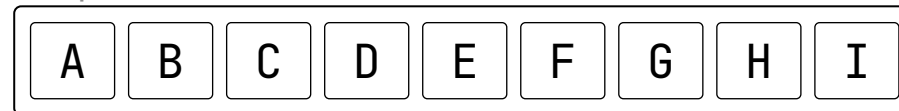


Spanning tree

Sorted edges



Components



Shortest Path

Dijkstra(*start*)

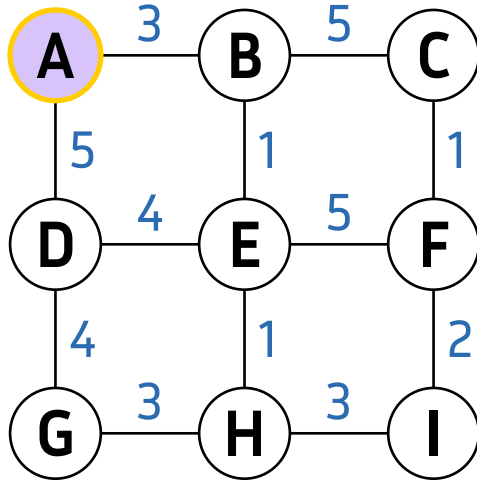
```
1  visited ← new Map; dist ← new Map
2  queue ← new Priority Queue; prev ← new Map
3  queue.add(start, 0); dist[start] ← 0; prev[start] ← ∅
4  while queue is not empty:
5      u ← queue.poll()
6      if u is not visited:
7          mark u as visited
8          for each neighbor v of u:
9              if v is not visited:
10                 new-dist ← dist[u] + cost(u, v)
11                 if new-dist < dist[v]:
12                     queue.add(v, new-dist); dist[v] ← new-dist; prev[v] ← u
```

Shortest Path

ConstructShortestPath(*prev*, *dest*)

```
1  | path ← new List
2  | current ← dest
3  | while current is not null:
4  |   | insert current at the beginning of path
5  |   | current ← prev[current]
6  | return path
```


Shortest Path – Example

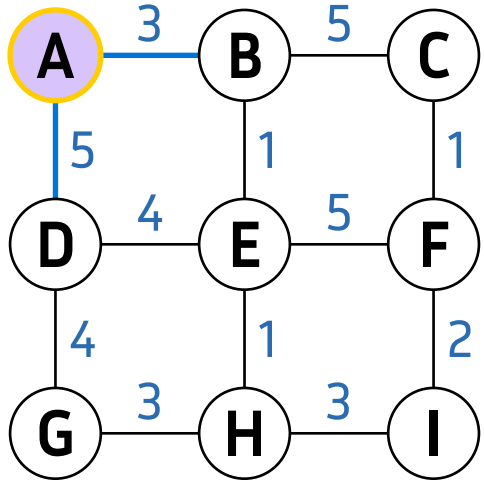


Visit A (0)

Priority queue
min **A: 0**

[illegible]

Shortest Path – Example



Update neighbors of A

Priority queue

min

B: 3

D: 5

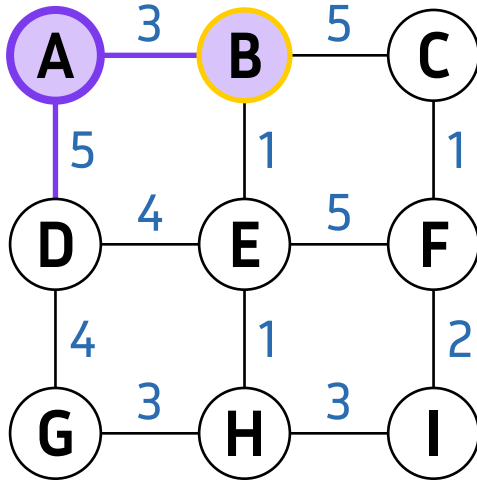
Distances

A	B	C	D	E	F	G	H	I
0	3	∞	5	∞	∞	∞	∞	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	$\emptyset$	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Shortest Path – Example



Visit B (3)

Priority queue

min

B: 3

D: 5

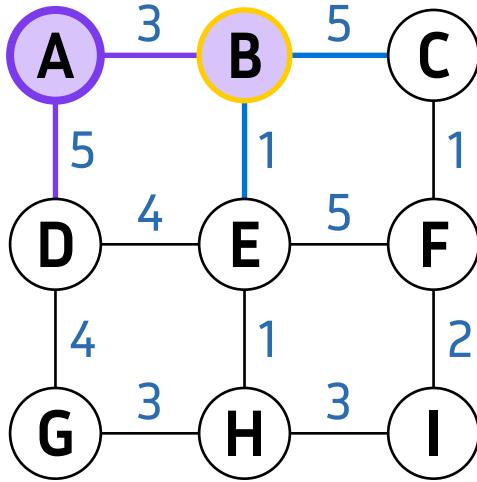
Distances

A	B	C	D	E	F	G	H	I
0	3	∞	5	∞	∞	∞	∞	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	$\emptyset$	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Shortest Path – Example



Update neighbors of B

Priority queue

min

E: 4

D: 5

C: 8

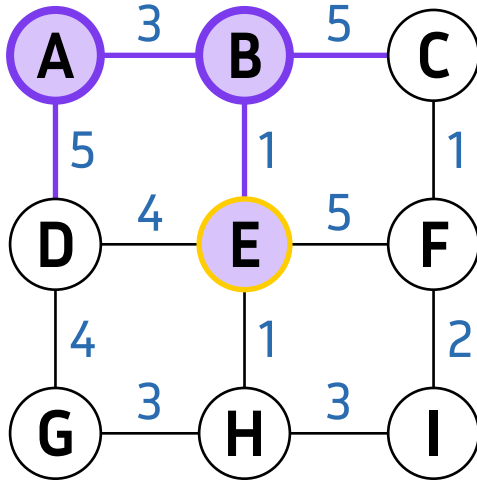
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	∞	∞	∞	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	B	A	B	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Shortest Path – Example



Visit E (4)

Priority queue

min

E: 4

D: 5

C: 8

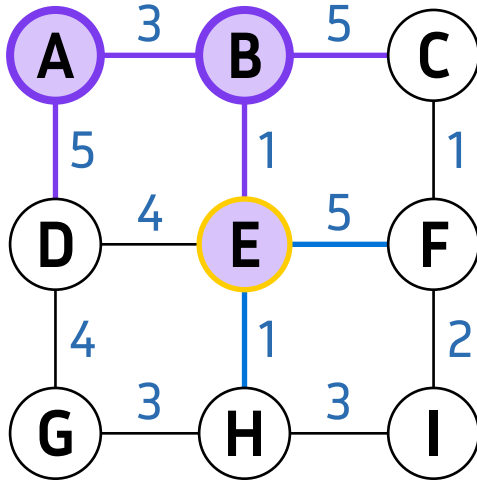
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	∞	∞	∞	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	B	A	B	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Shortest Path – Example



Update neighbors of E

Priority queue

min

D: 5
H: 5
C: 8
F: 9

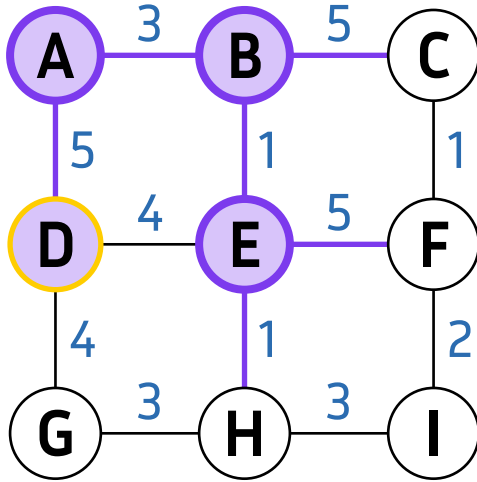
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	∞	5	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	B	A	B	E	$\emptyset$	E	$\emptyset$

Shortest Path – Example



Visit D (5)

Priority queue

min

D: 5

H: 5

C: 8

F: 9

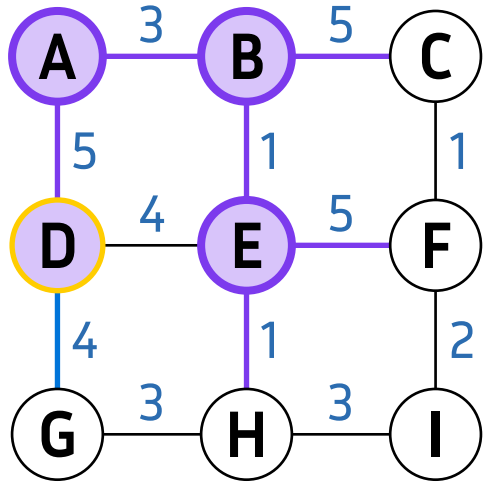
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	∞	5	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	B	A	B	E	$\emptyset$	E	$\emptyset$

Shortest Path – Example



Update neighbors of D

Priority queue

min

H: 5
C: 8
F: 9
G: 9

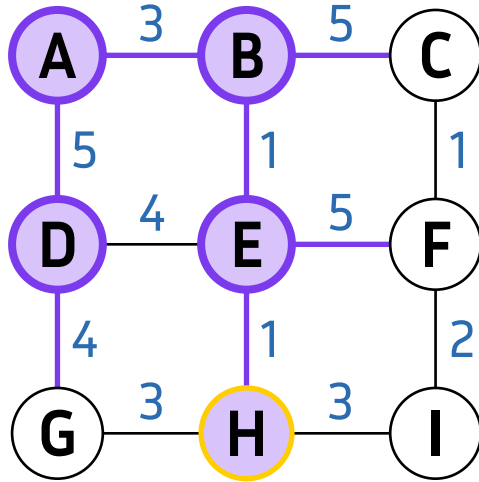
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	9	5	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	B	A	B	E	D	E	$\emptyset$

Shortest Path – Example



Visit H (5)

Priority queue

min

H: 5

C: 8

F: 9

G: 9

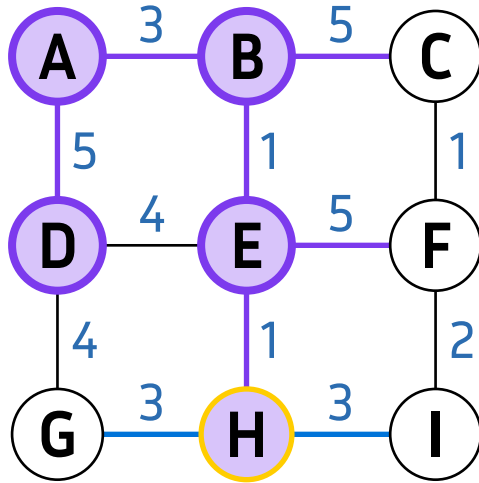
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	9	5	∞

Predecessors

A	B	C	D	E	F	G	H	I
$\emptyset$	A	B	A	B	E	D	E	$\emptyset$

Shortest Path – Example



Update neighbors of H

Priority queue

min	C: 8
	G: 8
	I: 8
	F: 9
	G: 9

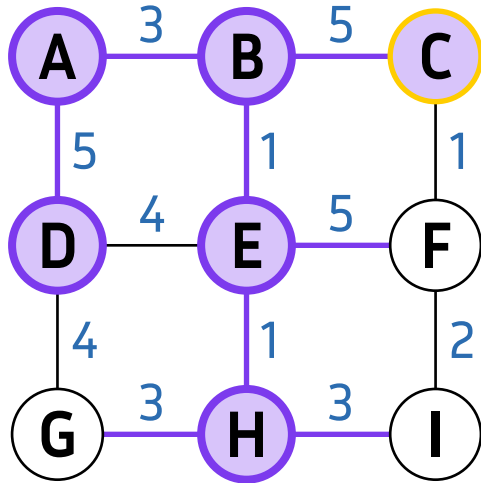
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Visit C (8)

Priority queue

min

C: 8
G: 8
I: 8
F: 9
G: 9

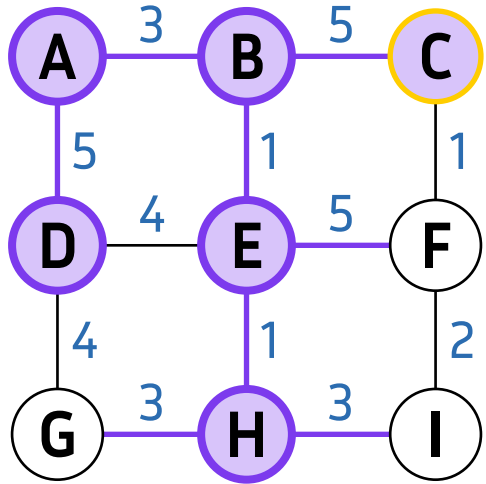
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Update neighbors of C

Priority queue

min

G: 8
I: 8
F: 9
G: 9

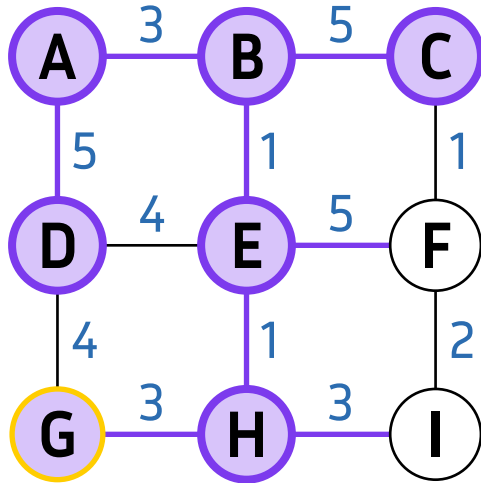
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Visit G (8)

Priority queue

min

G: 8

I: 8

F: 9

G: 9

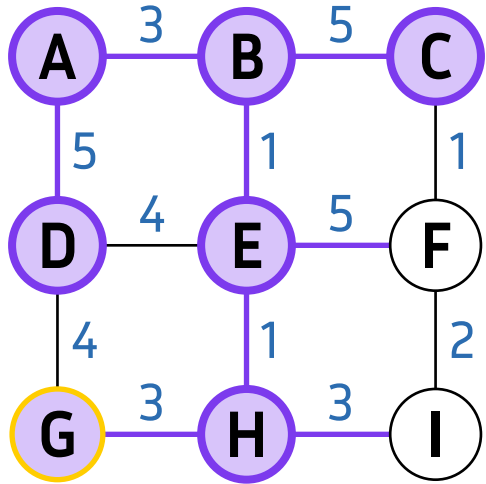
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Update neighbors of G

Priority queue

min

I: 8

F: 9

G: 9

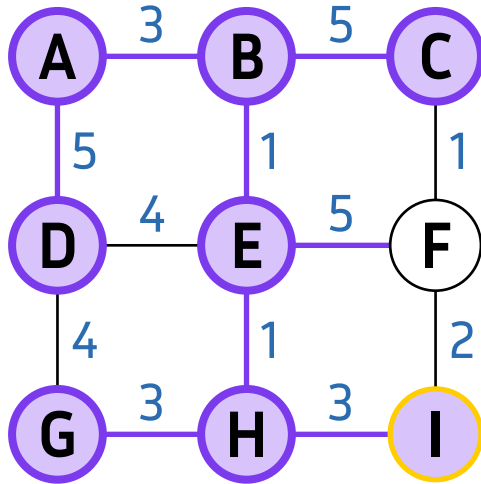
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Visit I (8)

Priority queue

min I: 8

F: 9

G: 9

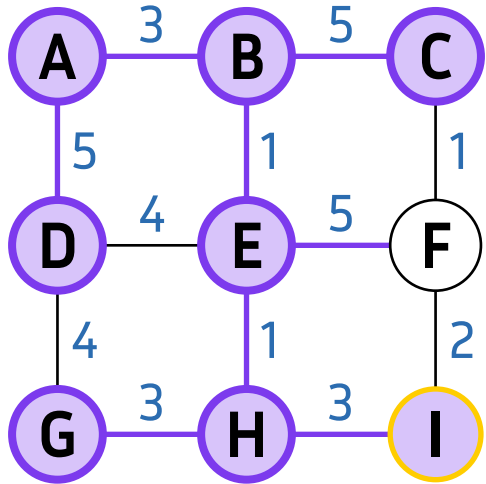
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Update neighbors of I

Priority queue

min

F: 9

G: 9

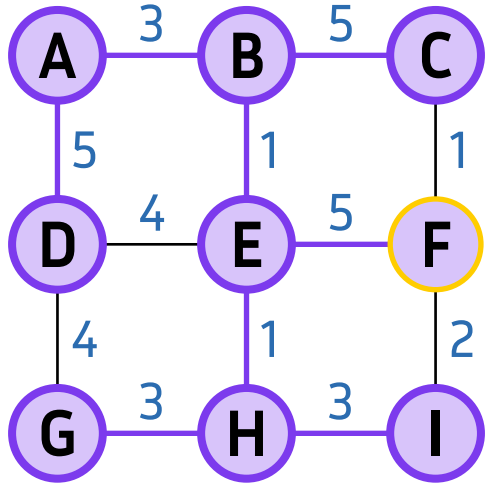
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Visit F (9)

Priority queue

min

F: 9

G: 9

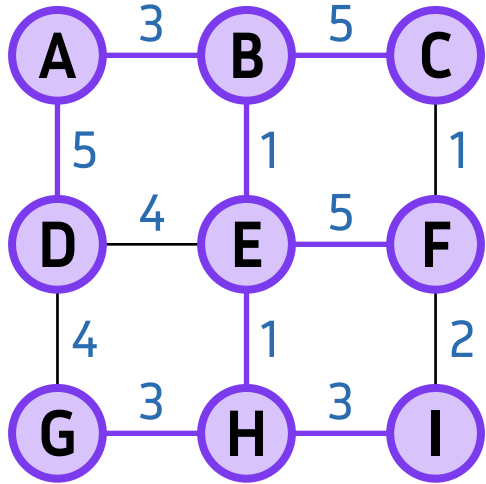
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Reached F (9)

Priority queue
min G: 9

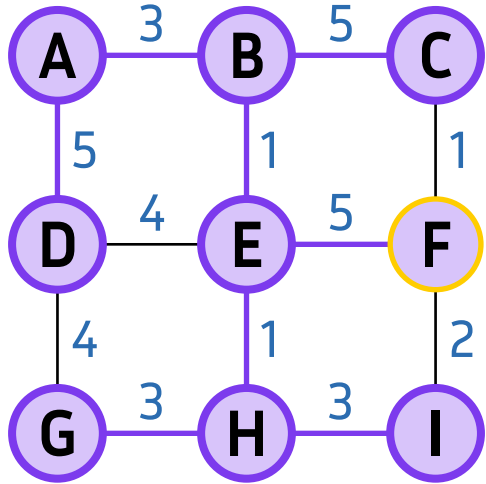
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Path: F

Priority queue
min G: 9

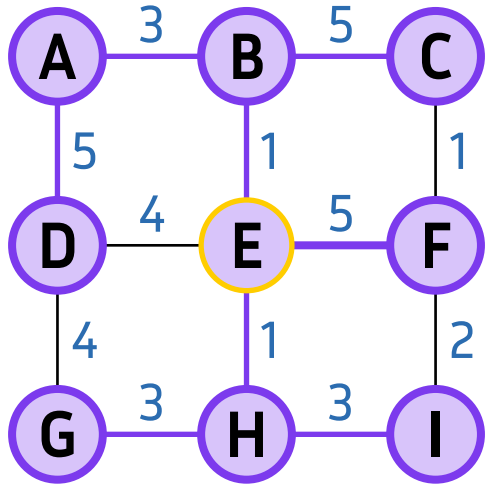
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Path: E → F

Priority queue
min G: 9

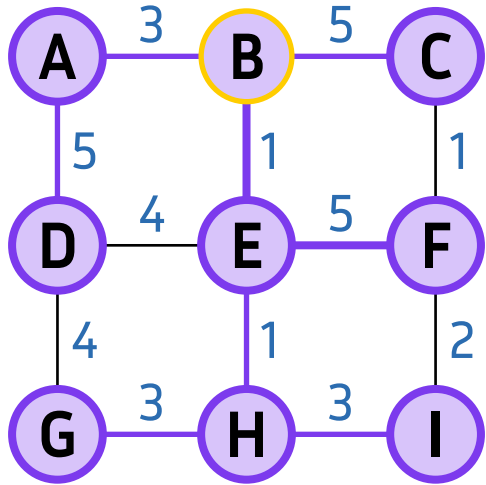
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Path: B → E → F

Priority queue
min G: 9

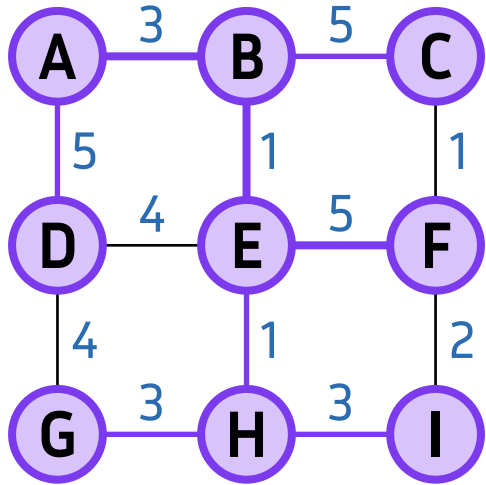
Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Shortest Path – Example



Shortest path: A → B → E → F (9)

Priority queue
min G: 9

Distances

A	B	C	D	E	F	G	H	I
0	3	8	5	4	9	8	5	8

Predecessors

A	B	C	D	E	F	G	H	I
∅	A	B	A	B	E	H	E	H

Tries

Definition (Trie) Also called a “prefix tree”, a data structure which stores a set of strings which can be queried by their prefixes.

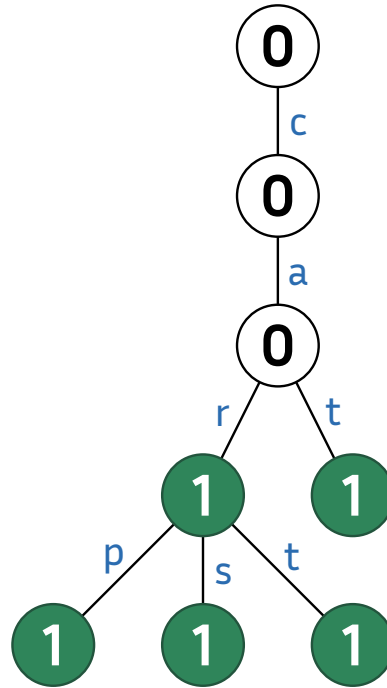
- ✦ Tries allow a user to query strings based on prefixes and they support **quickly resolving an update** to the query.
- ✦ Tries are a graph, but rather than storing data on the nodes, we **store the data along the edges**.
- ✦ Each edge corresponds to a **single character** in the string which is represented by a **path in the tree**.

Tries – Lookup

- 1 Start at the **root**
- 2 For each letter in the word we are looking up:
 - 3 | If there is an edge corresponding to that letter:
 - 4 | | Take **that edge** to the next node
 - 5 | | Else
 - 6 | | Return **false**
- 7 If the current node is **final**, return **true**, else return **false**

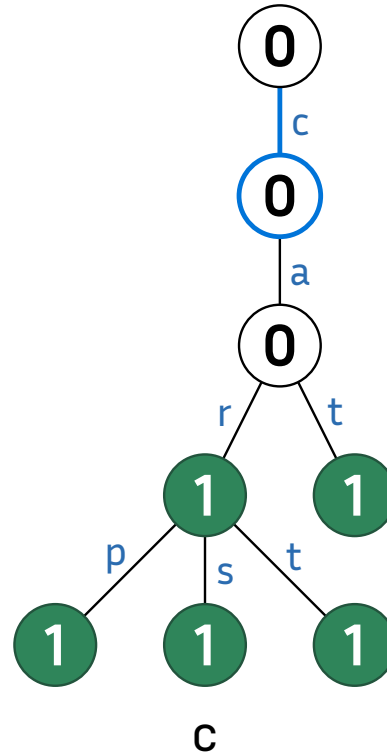
Tries – Lookup Example

◆ Lookup “cats”



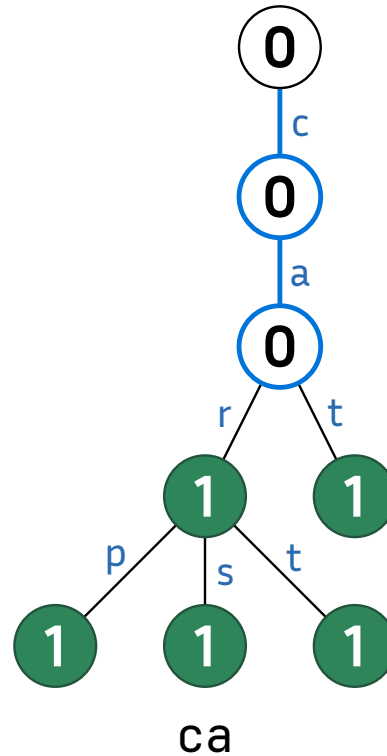
Tries – Lookup Example

◆ Lookup “cats”



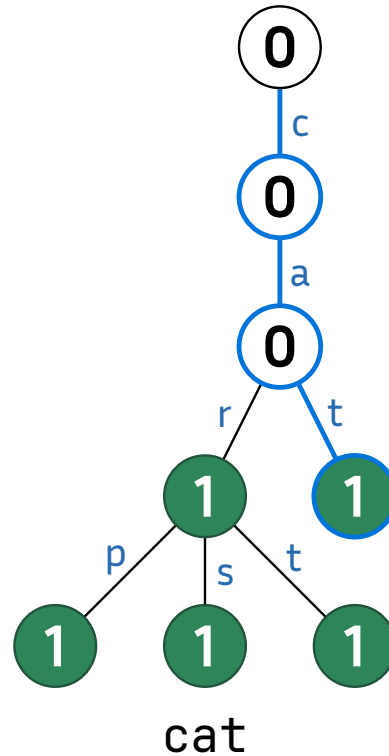
Tries – Lookup Example

◆ Lookup “cats”



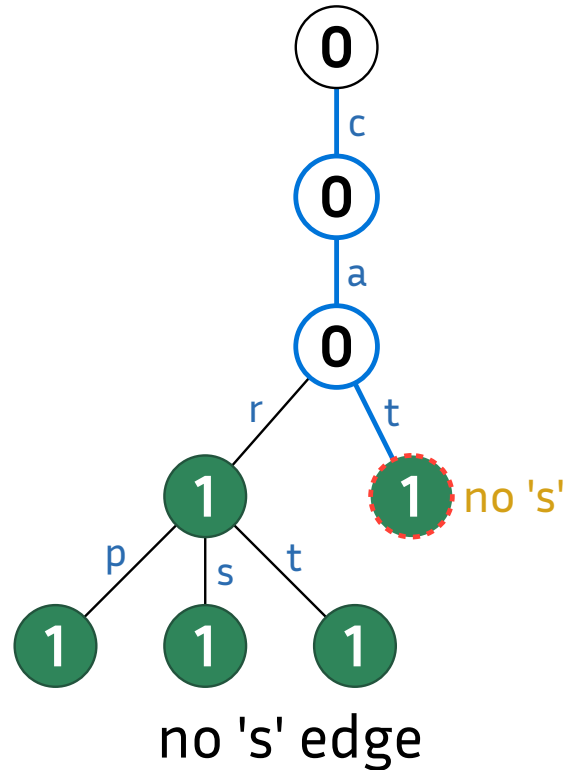
Tries – Lookup Example

◆ Lookup “cats”



Tries – Lookup Example

✦ Lookup “cats”

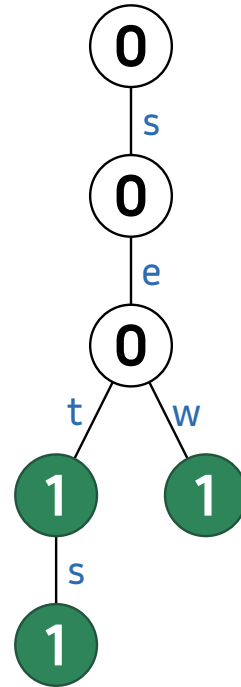


Tries – Insert

- 1 Start at the **root**
- 2 For each letter in the word we are looking up:
 - 3 | If there is an edge corresponding to that letter:
 - 4 | | Take **that edge** to the next node
 - 5 | | Else
 - 6 | | Return **false**
 - 7 If the current node is **final**, return **true**, else return **false**

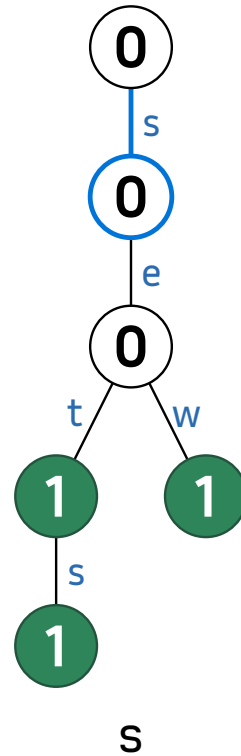
Tries – Insert Example

✦ Insert “saw”



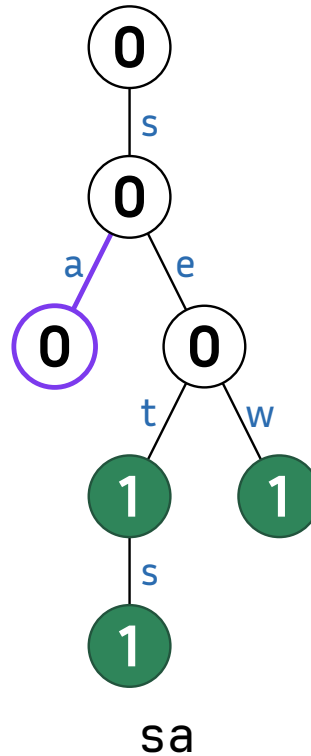
Tries – Insert Example

✦ Insert “saw”



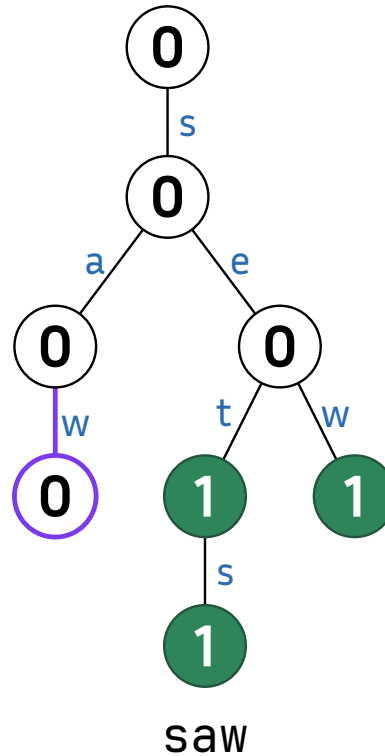
Tries – Insert Example

✦ Insert “saw”



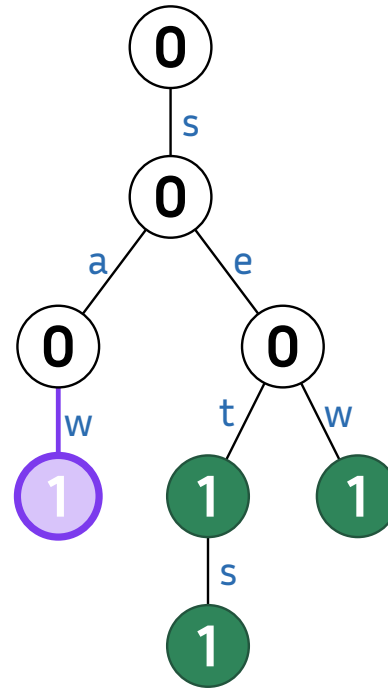
Tries – Insert Example

✦ Insert “saw”



Tries – Insert Example

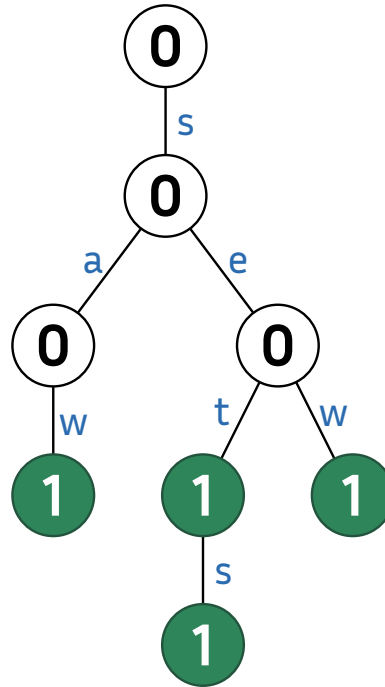
✦ Insert "saw"



inserted "saw"

Tries – Insert Example

✦ Insert “saw”

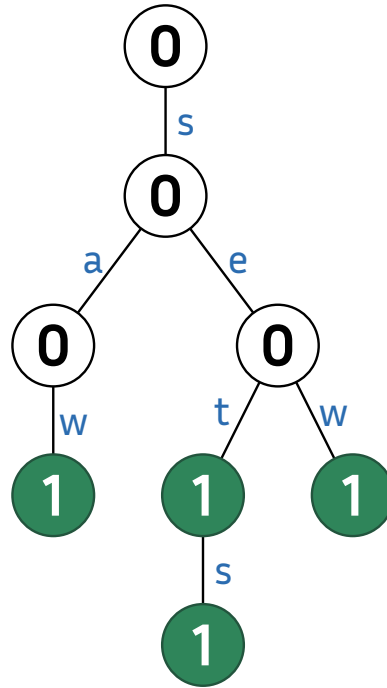


Tries – Deletion

- 1 Perform a **lookup** of the word
- 2 Set the current node to **non-final** to remove it from trie
- 3 While the current node is **not the root node**:
 - 4 | If the current node is **non-final** and has **no children**:
 - 5 | | **Delete** the node from the trie
 - 6 | Set current node to the **parent** node

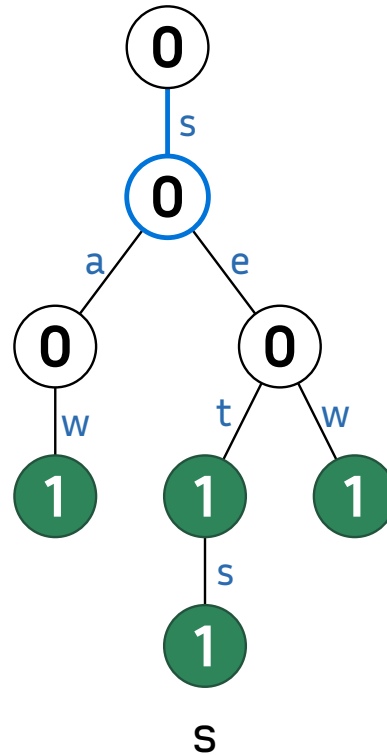
Tries – Deletion Example

✦ Delete “saw”



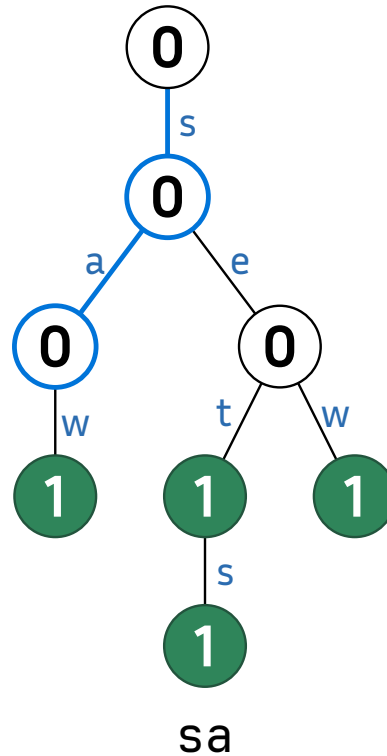
Tries – Deletion Example

✦ Delete “saw”



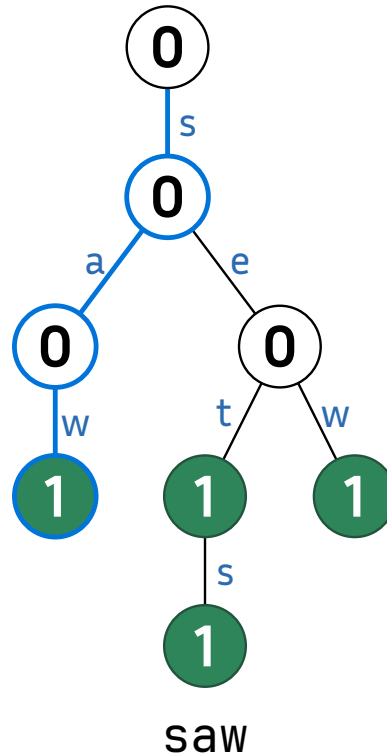
Tries – Deletion Example

✦ Delete “saw”



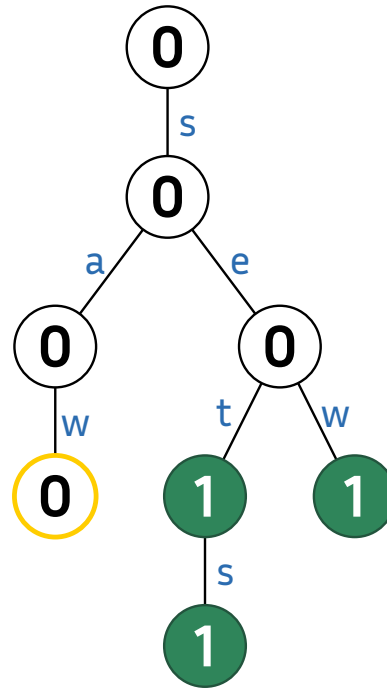
Tries – Deletion Example

✦ Delete “saw”



Tries – Deletion Example

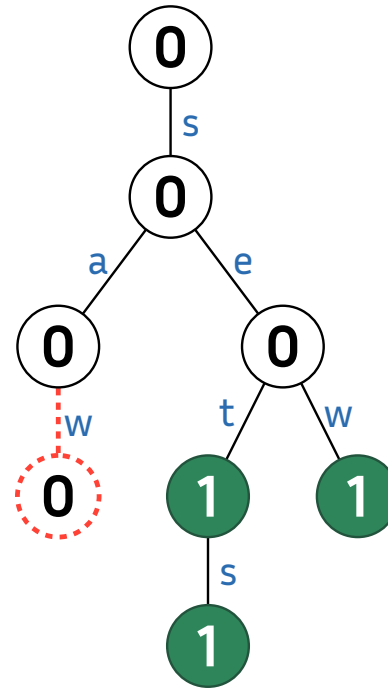
✦ Delete "saw"



unmark "saw"

Tries – Deletion Example

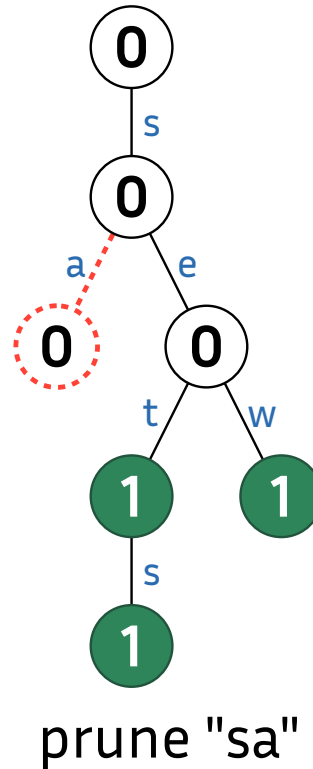
✦ Delete "saw"



prune "saw"

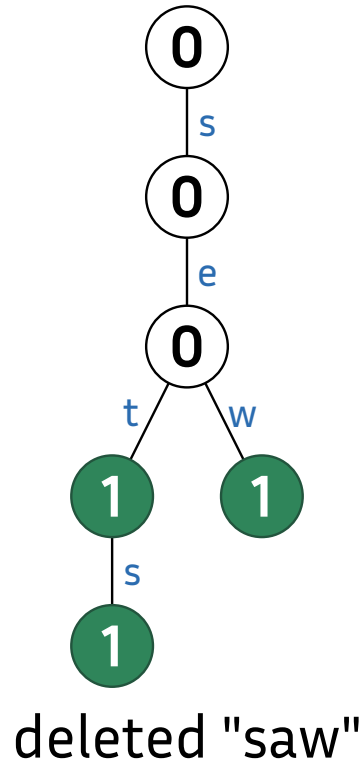
Tries – Deletion Example

✦ Delete “saw”



Tries – Deletion Example

✦ Delete "saw"



Linear Sorts – Counting Sort

```
1 count ← array of  $k$  zeros
2 output ← array same length as input
3 for  $i \leftarrow 0$  to length(input) - 1 do
4   |  $j \leftarrow \text{key}(\text{input}[i])$ 
5   | count[ $j$ ] ← count[ $j$ ] + 1
6 for  $i \leftarrow 1$  to  $k$  do
7   | count[ $i$ ] ← count[ $i$ ] + count[ $i - 1$ ]
8 for  $i \leftarrow \text{length}(\text{input}) - 1$  to 0 do
9   |  $j \leftarrow \text{key}(\text{input}[i])$ 
10  | count[ $j$ ] ← count[ $j$ ] - 1
11  | output[count[ $j$ ]] ← input[ $i$ ]
```

Count Values

Convert to Cumulative

Copy to Output

Linear Sorts – Radix Sort

Definition (Radix Sort) A linear sort using **counting sort**'s stability to sort each digit one at a time.

- ✦ Can compare digits right-to-left / least significant digit (**LSD**) or left-to-right / most significant digit (**MSD**).
- ✦ We will **only use LSD** in this course.

812, 995

- ✦ LSD → Digit 2, Digit 1, Digit 0
- ✦ MSD → Digit 0, Digit 1, Digit 2

Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

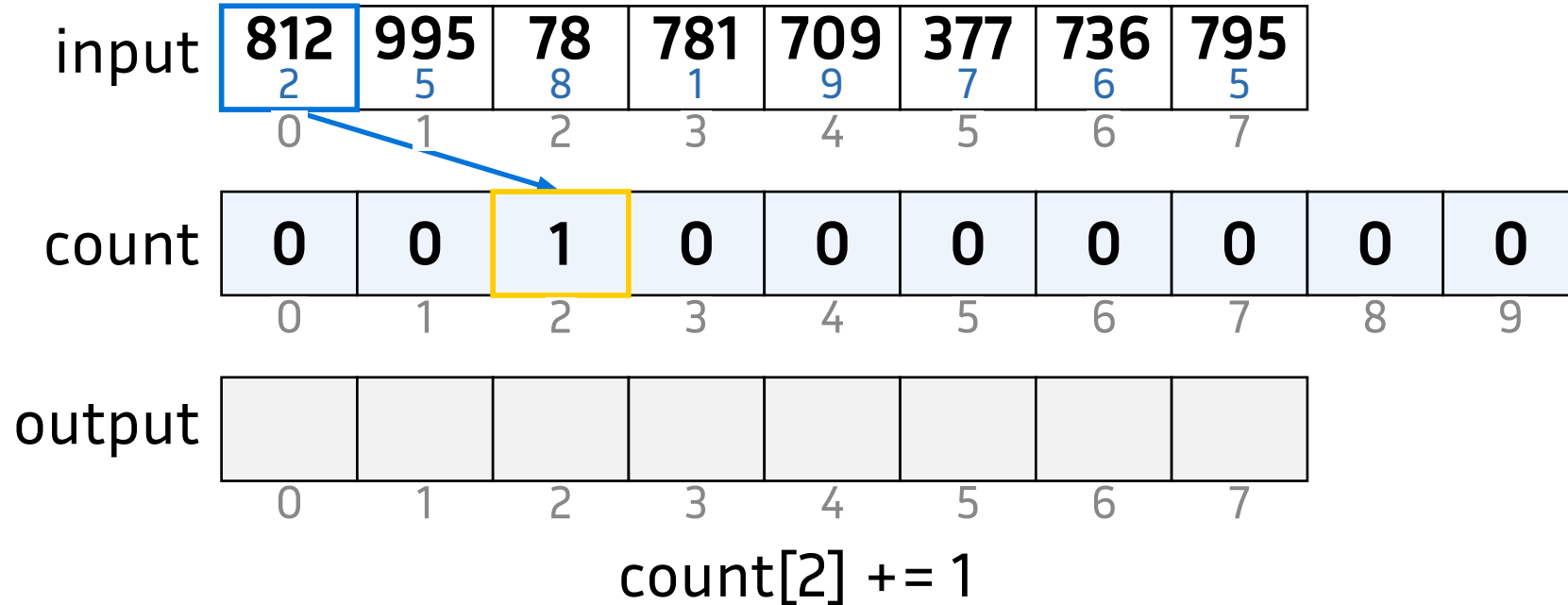
0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

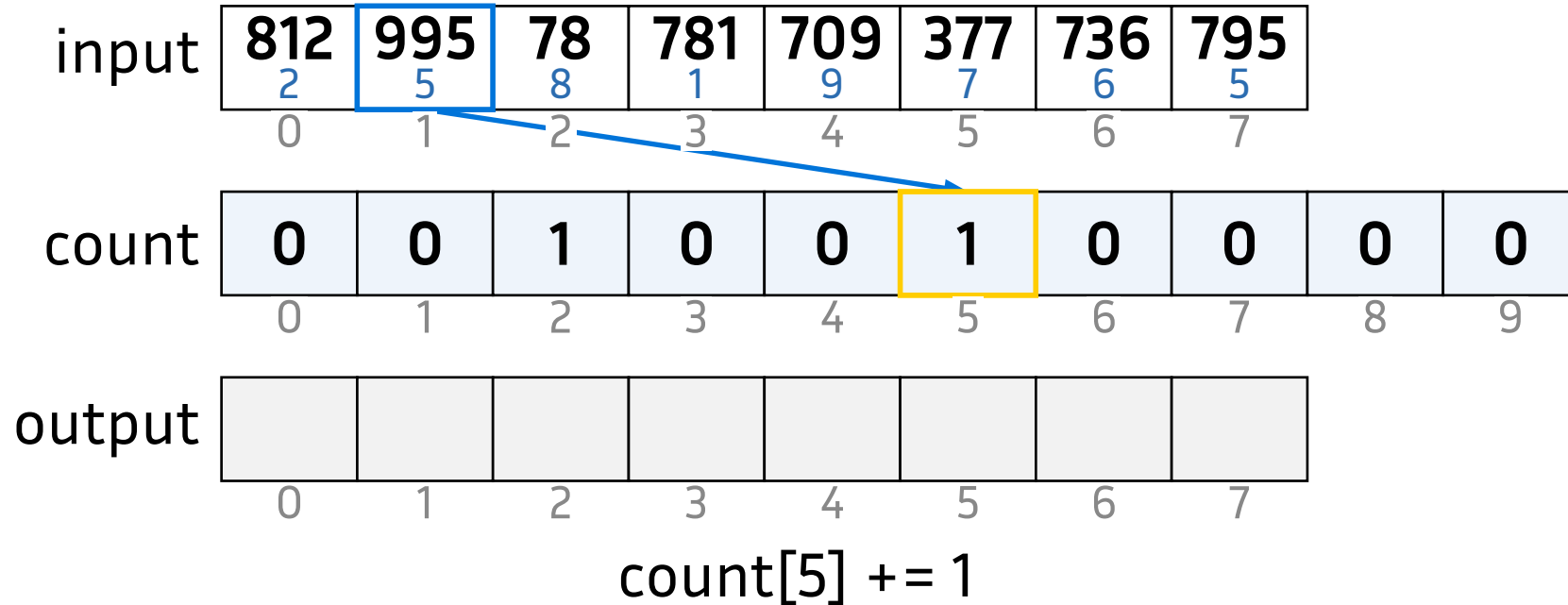
0	1	2	3	4	5	6	7

pass 1: ones digit

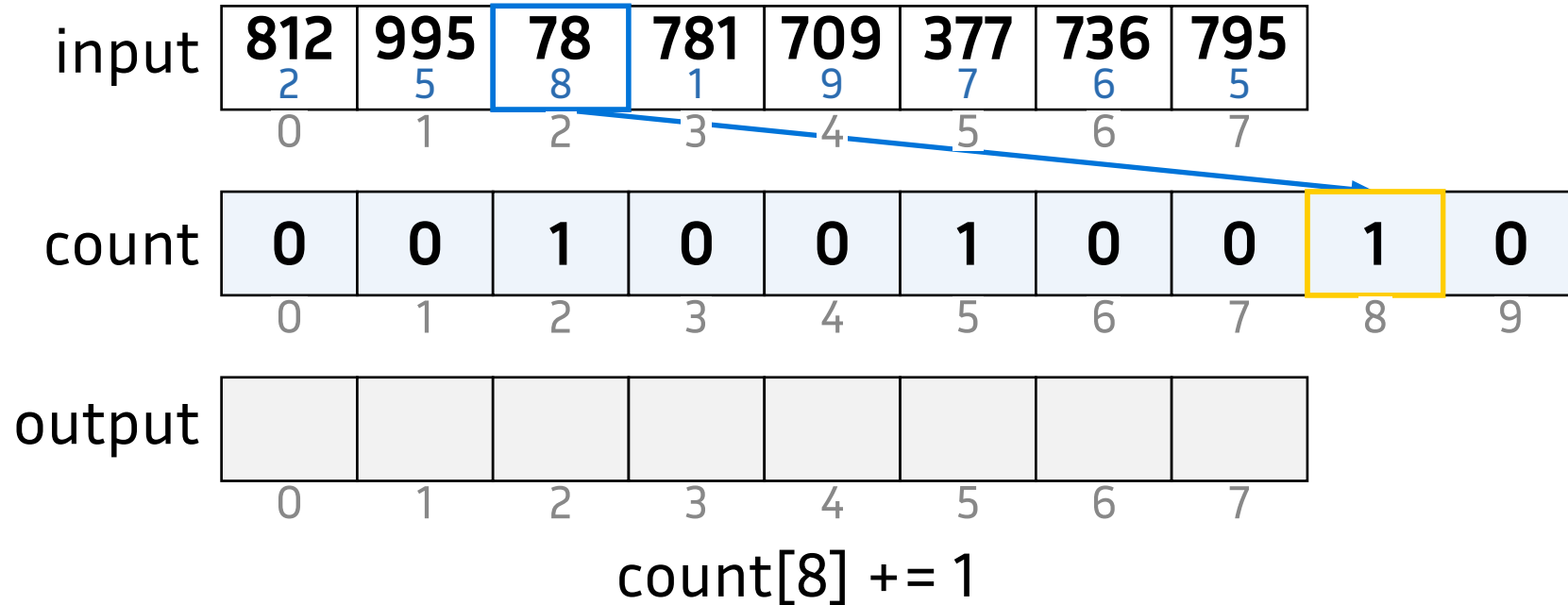
Linear Sorts – Radix Example



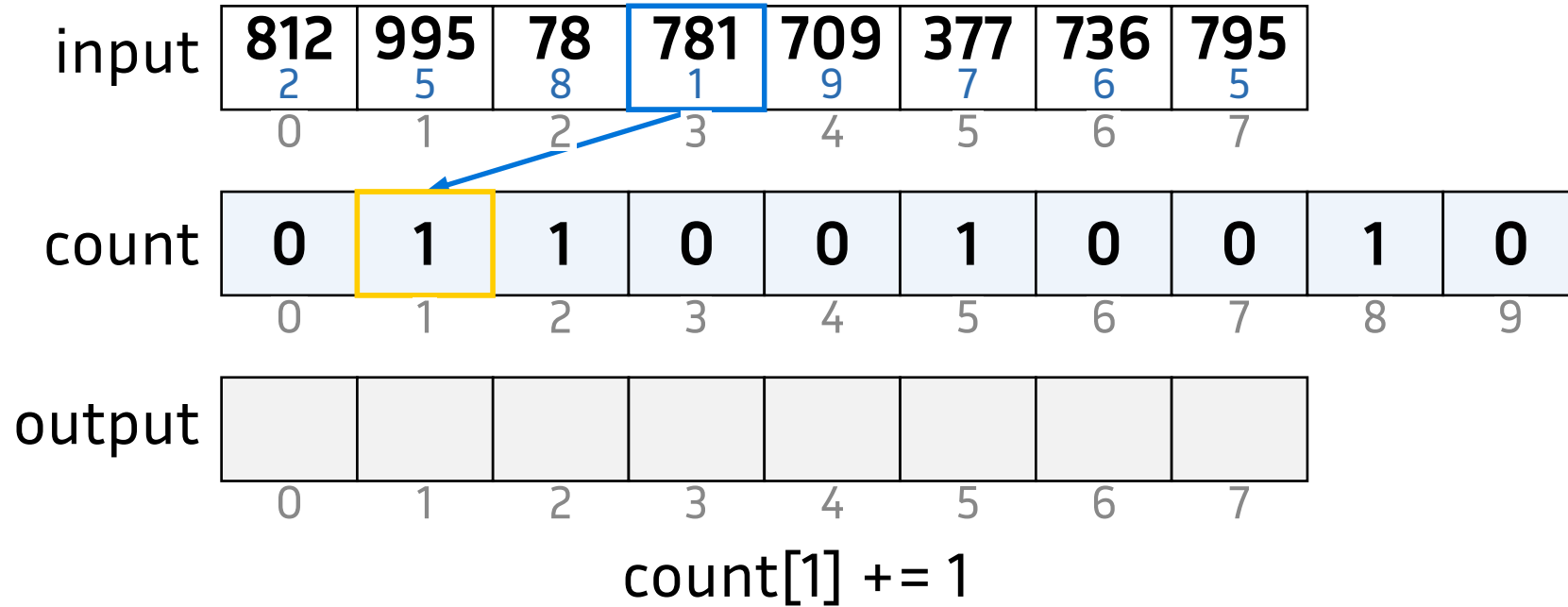
Linear Sorts – Radix Example



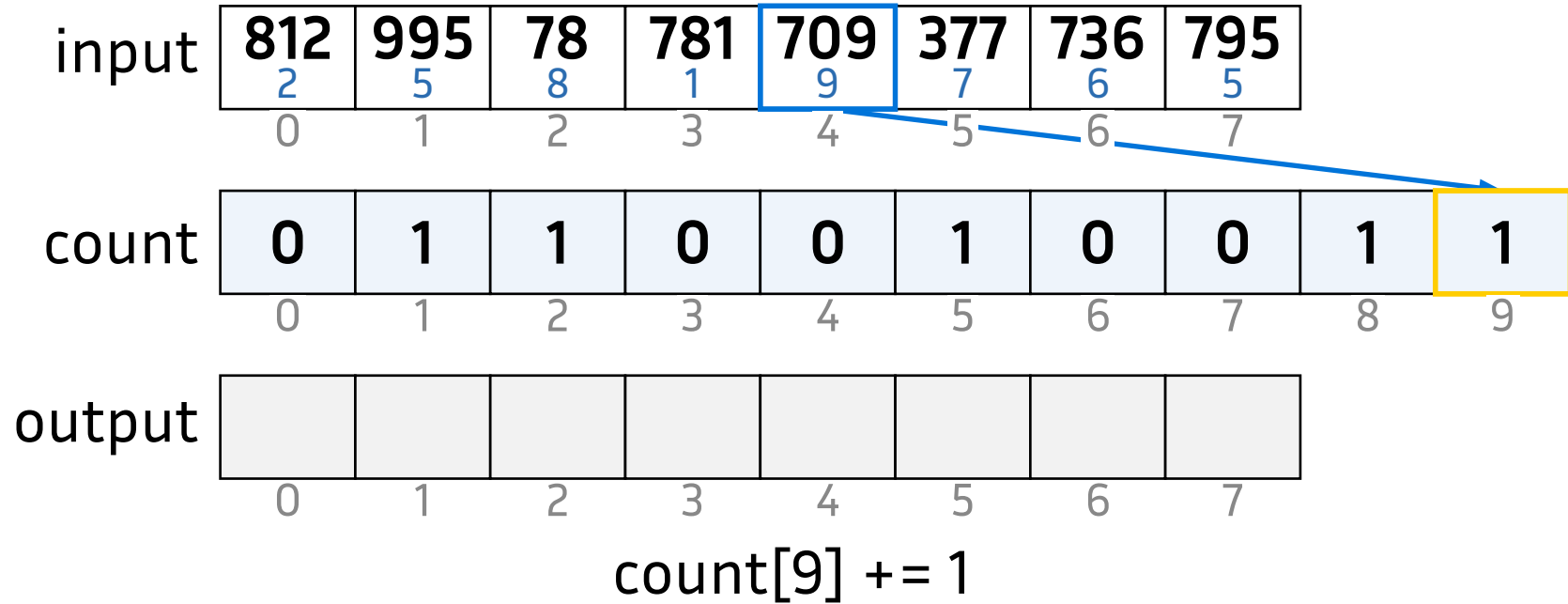
Linear Sorts – Radix Example



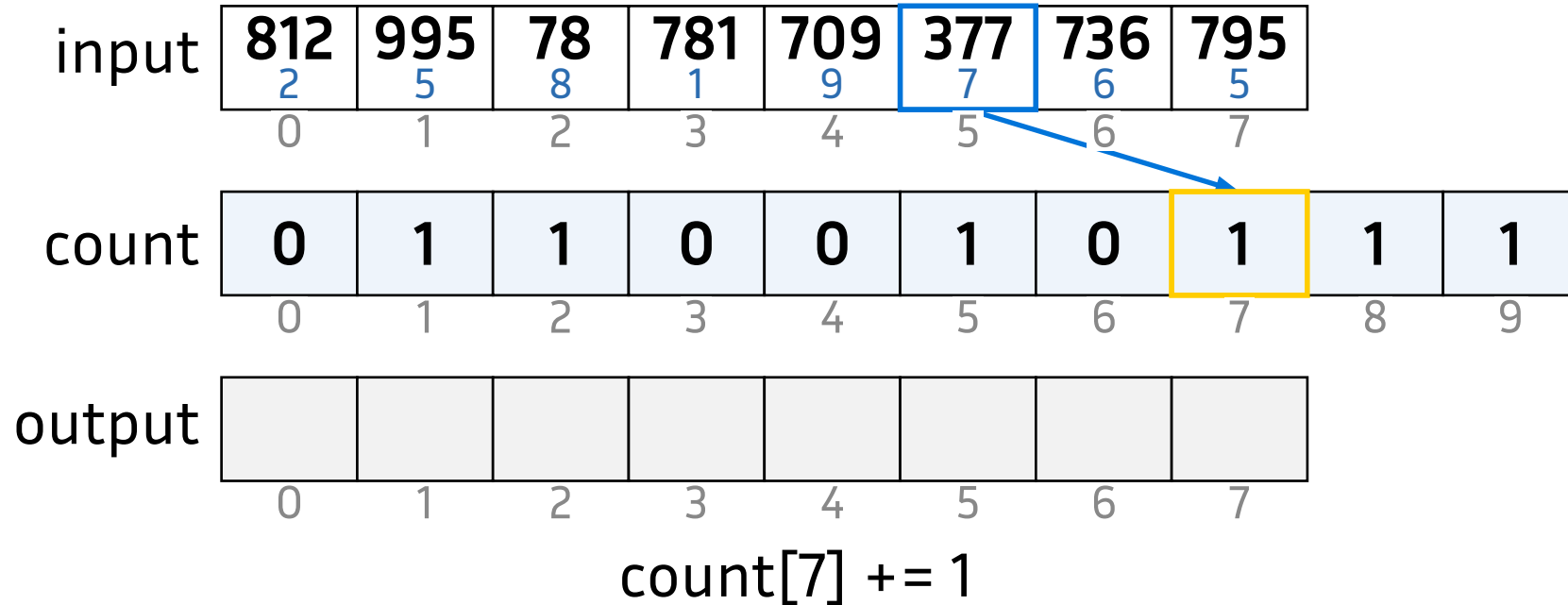
Linear Sorts – Radix Example



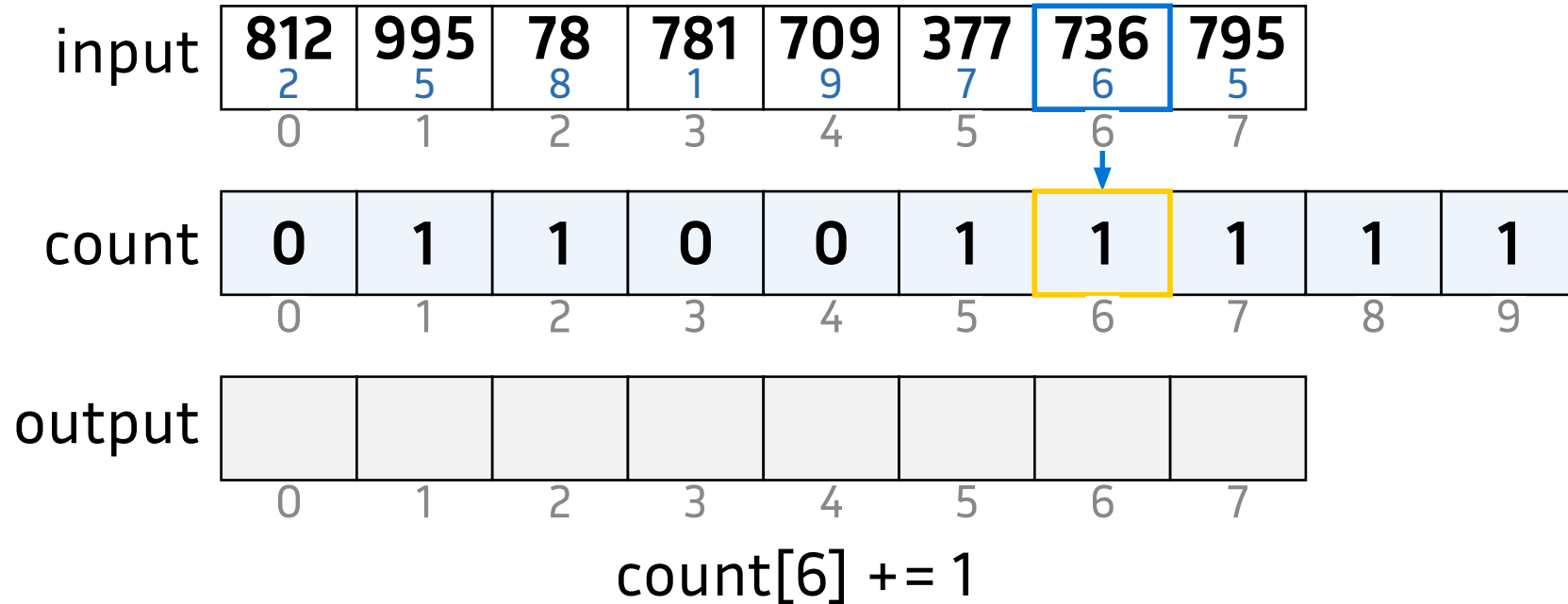
Linear Sorts – Radix Example



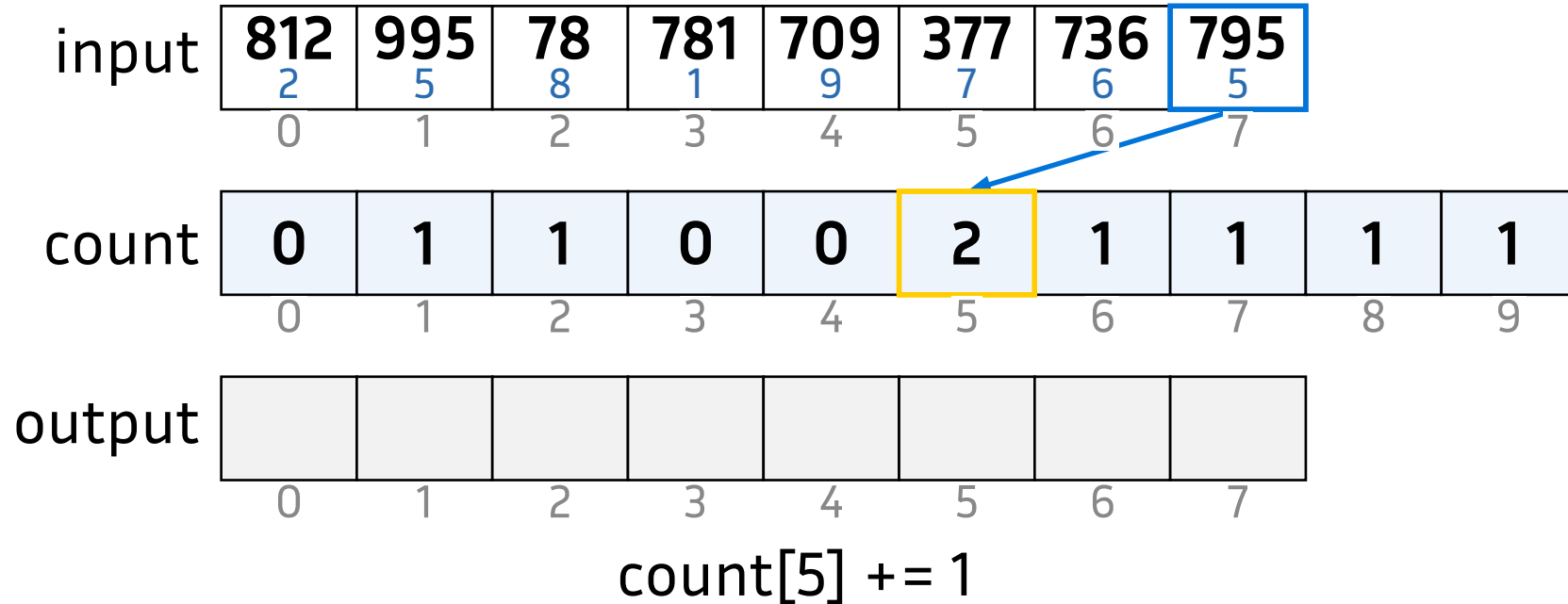
Linear Sorts – Radix Example



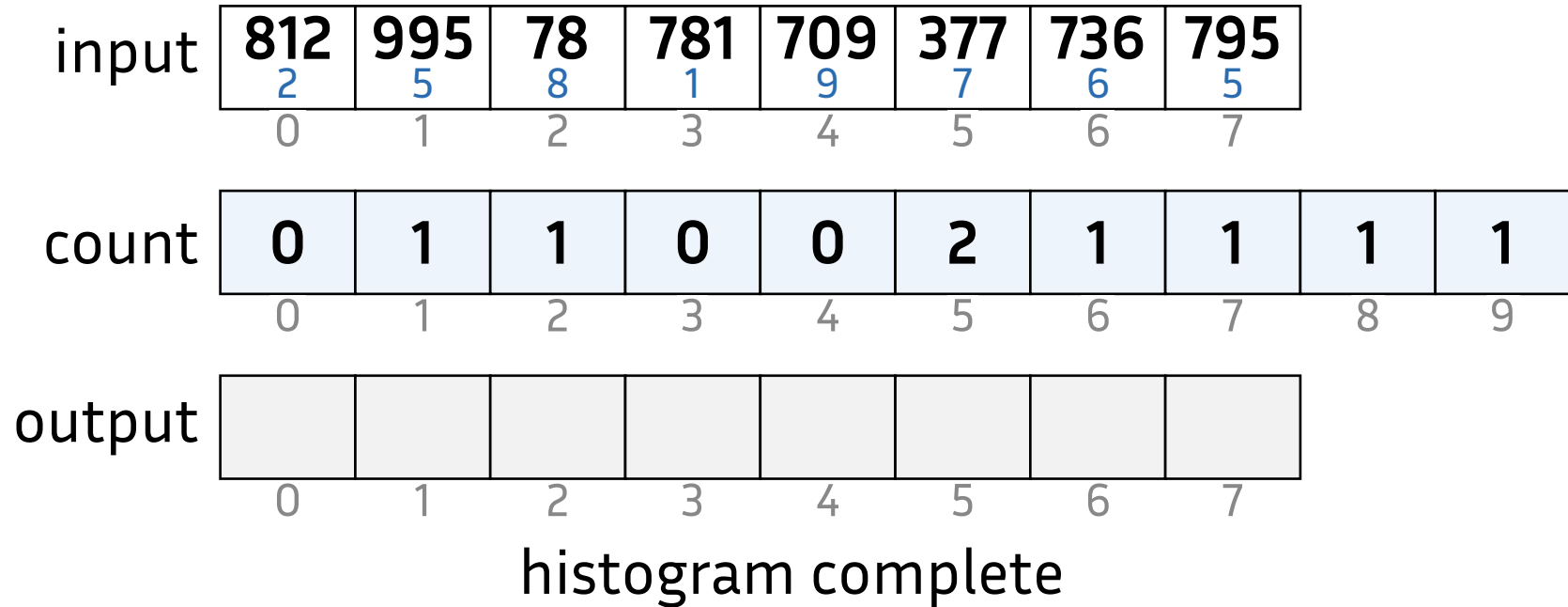
Linear Sorts – Radix Example



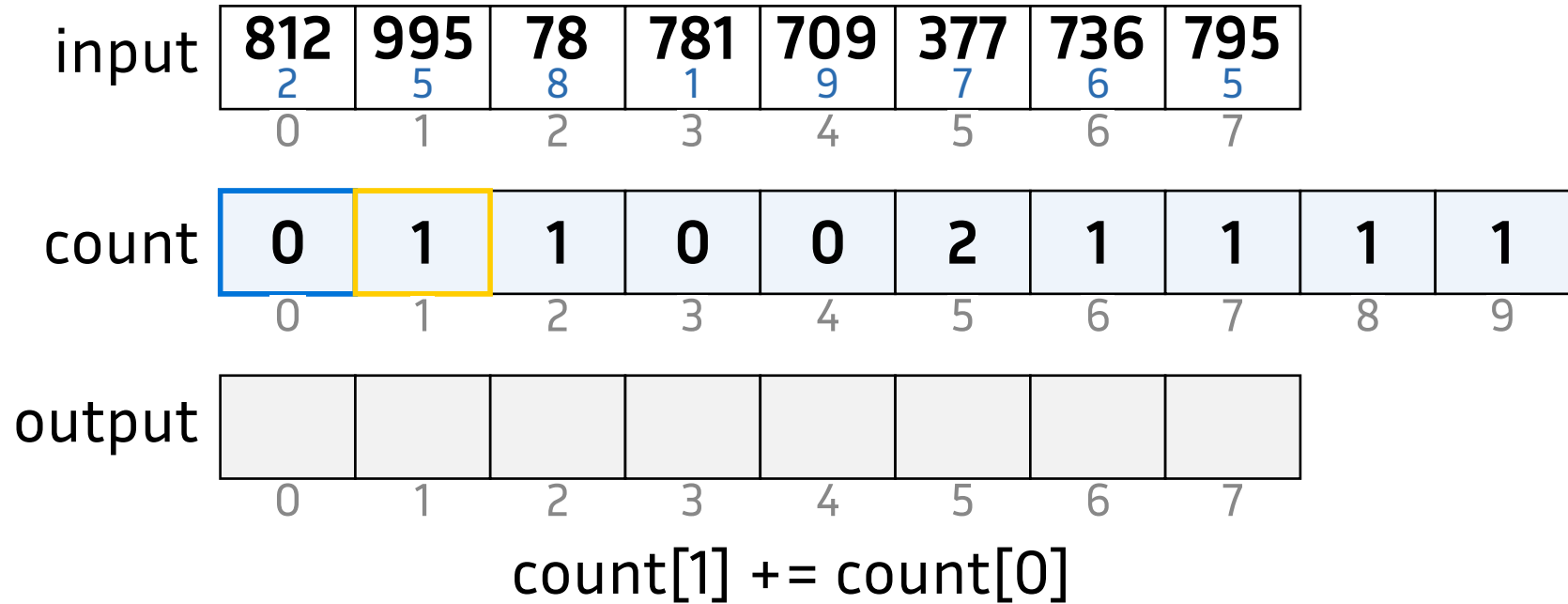
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

0	1	2	0	0	2	1	1	1	1
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[2] += count[1]

Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

0	1	2	2	0	2	1	1	1	1
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[3] += count[2]

Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

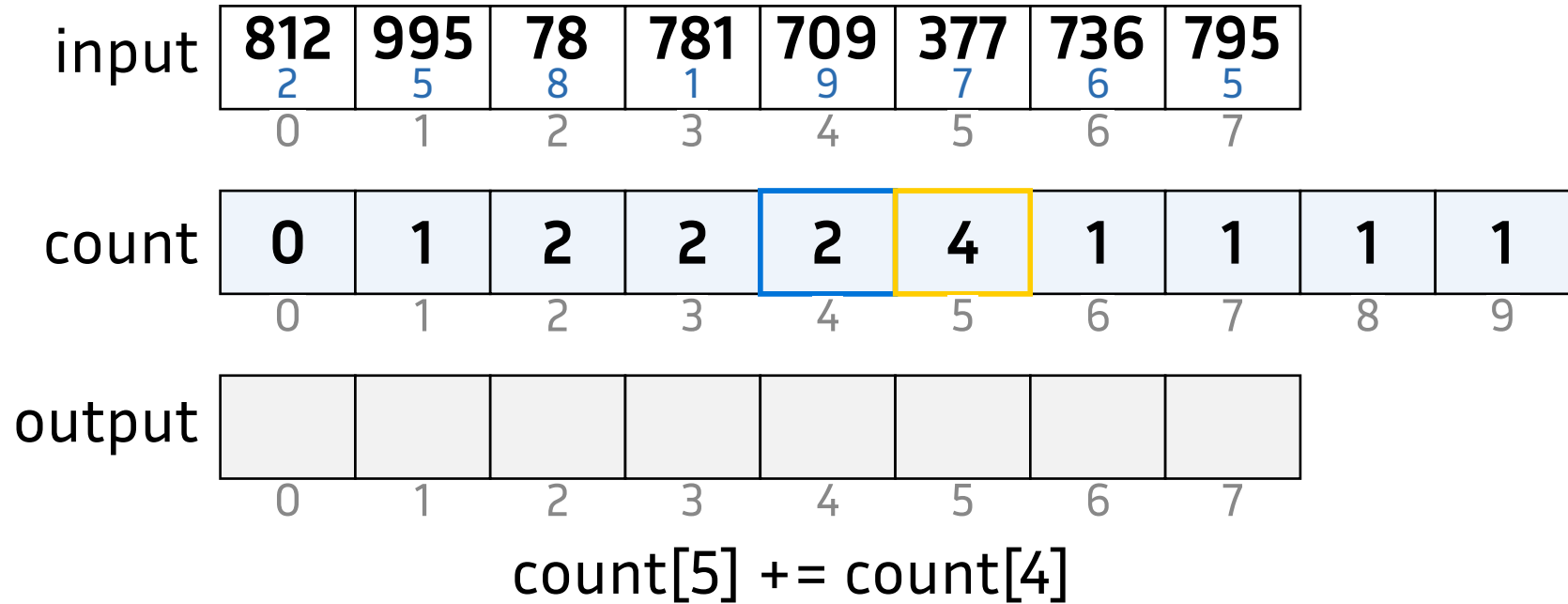
0	1	2	2	2	2	1	1	1	1
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[4] += count[3]

Linear Sorts – Radix Example



Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

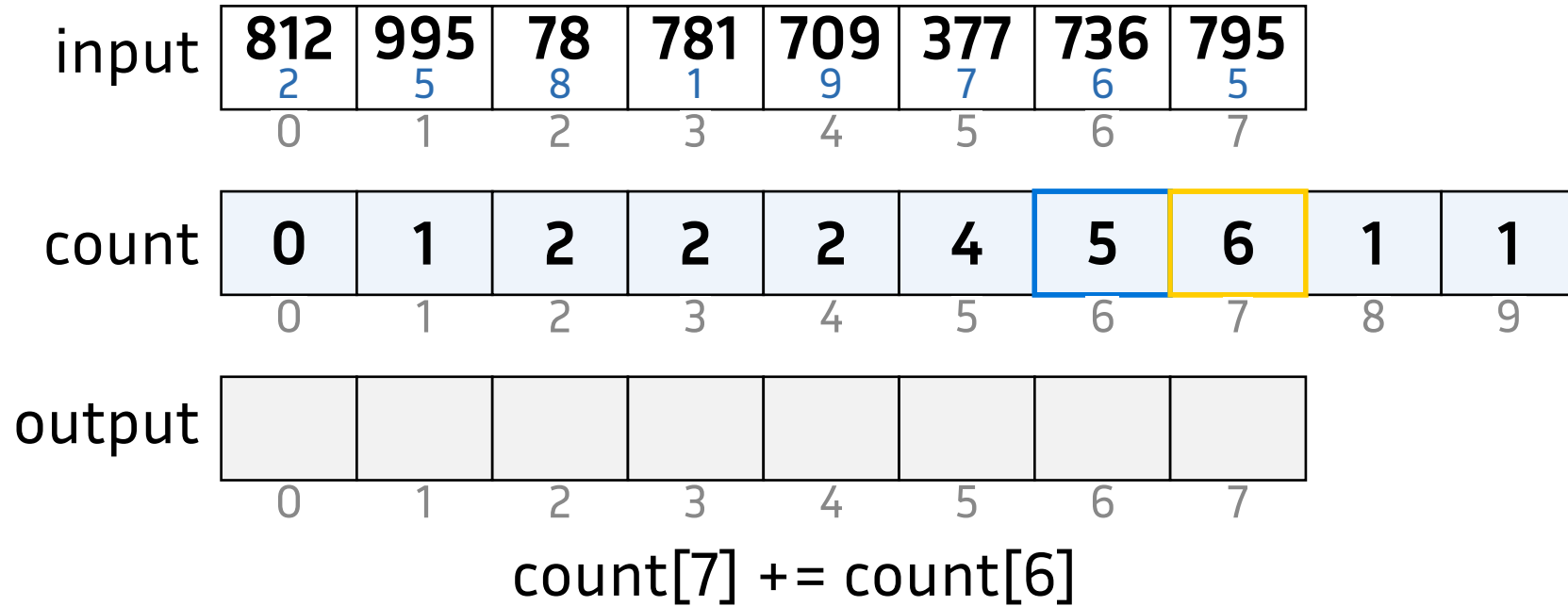
0	1	2	2	2	4	5	1	1	1
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[6] += count[5]

Linear Sorts – Radix Example



Linear Sorts – Radix Example

input	812	995	78	781	709	377	736	795		
	2	5	8	1	9	7	6	5		
	0	1	2	3	4	5	6	7		
count	0	1	2	2	2	4	5	6	7	1
	0	1	2	3	4	5	6	7	8	9
output										
	0	1	2	3	4	5	6	7		

$\text{count}[8] += \text{count}[7]$

Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

0	1	2	2	2	4	5	6	7	8
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[9] += count[8]

Linear Sorts – Radix Example

input

812	995	78	781	709	377	736	795
2	5	8	1	9	7	6	5
0	1	2	3	4	5	6	7

count

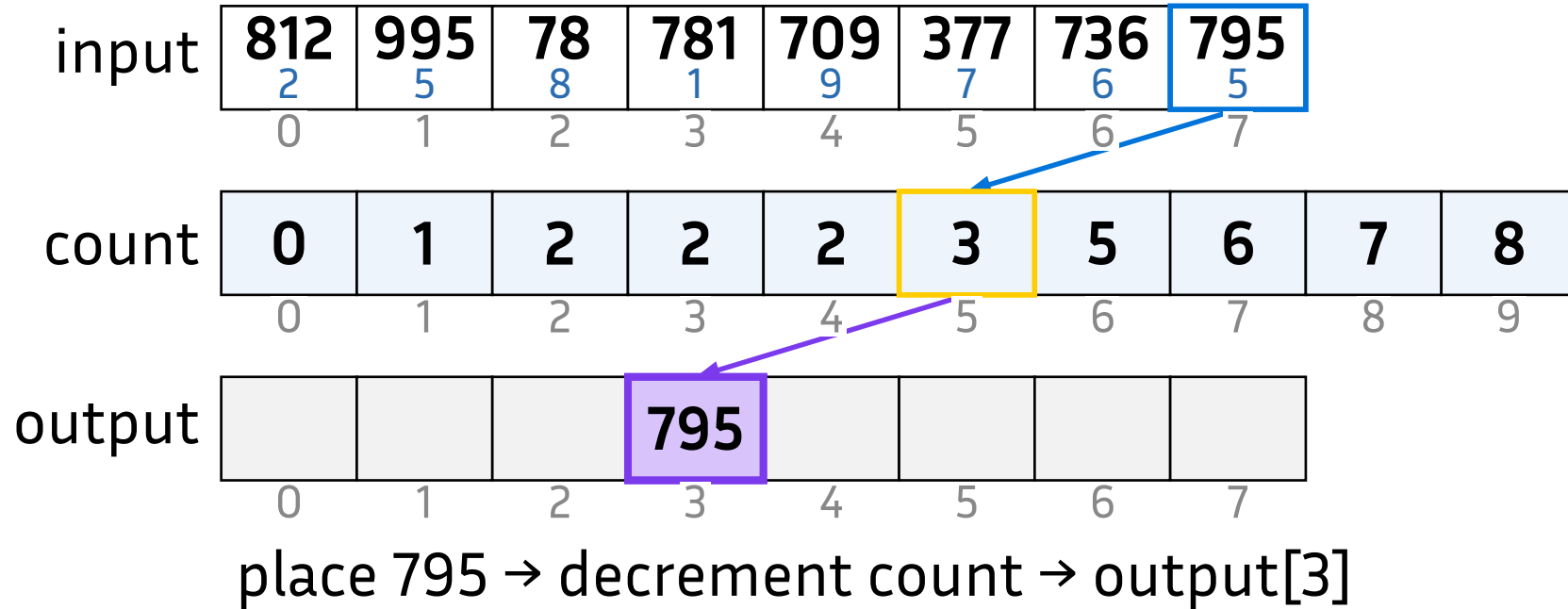
0	1	2	2	2	4	5	6	7	8
0	1	2	3	4	5	6	7	8	9

output

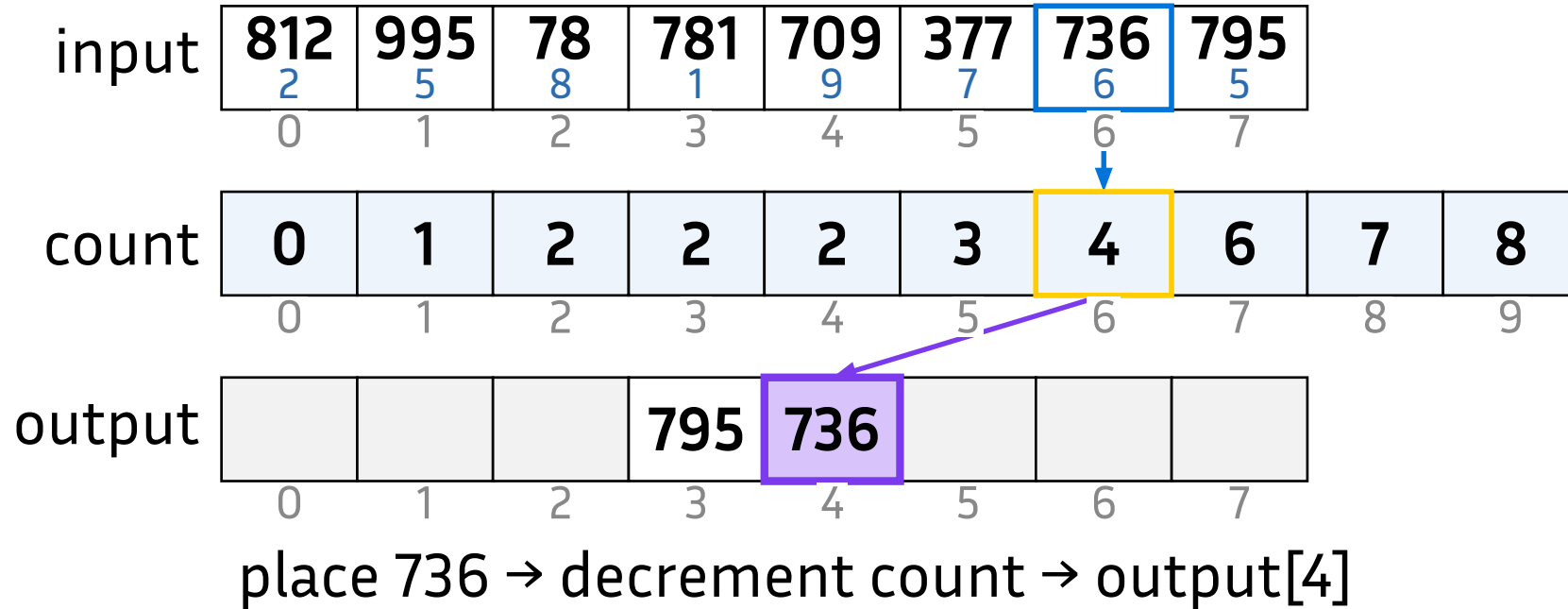
0	1	2	3	4	5	6	7

counts → end positions

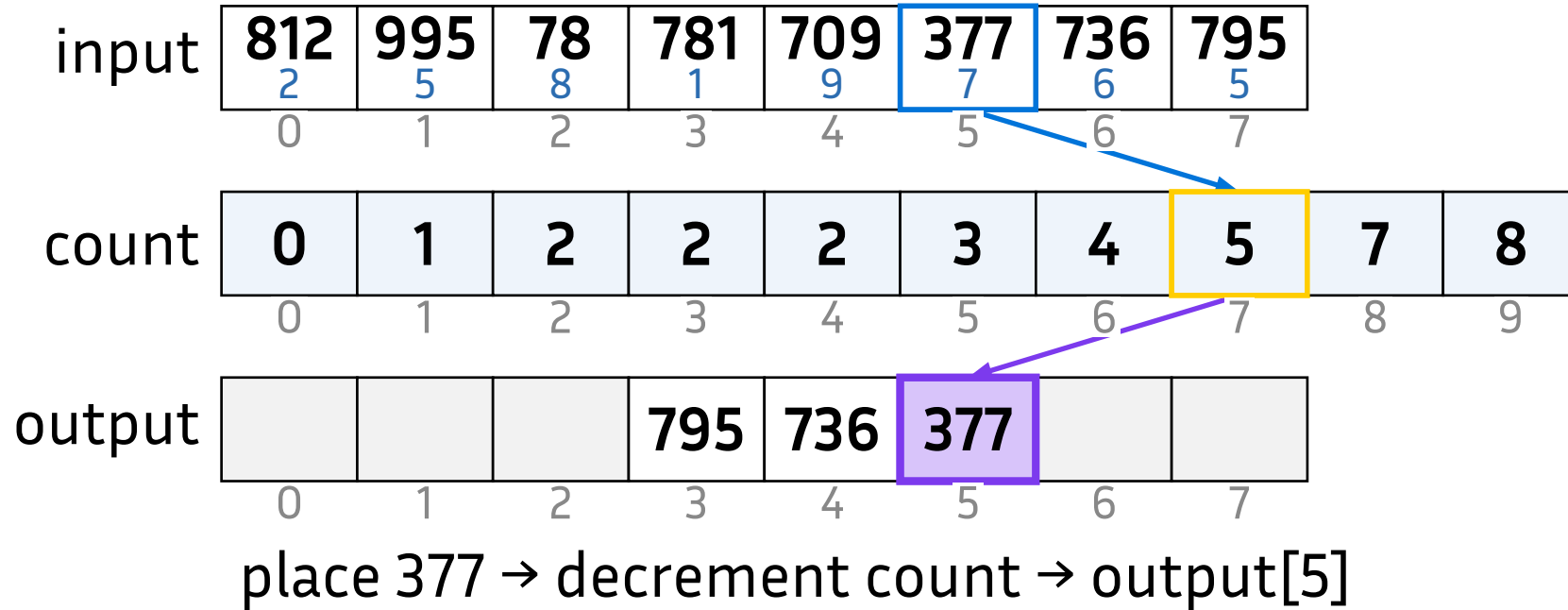
Linear Sorts – Radix Example



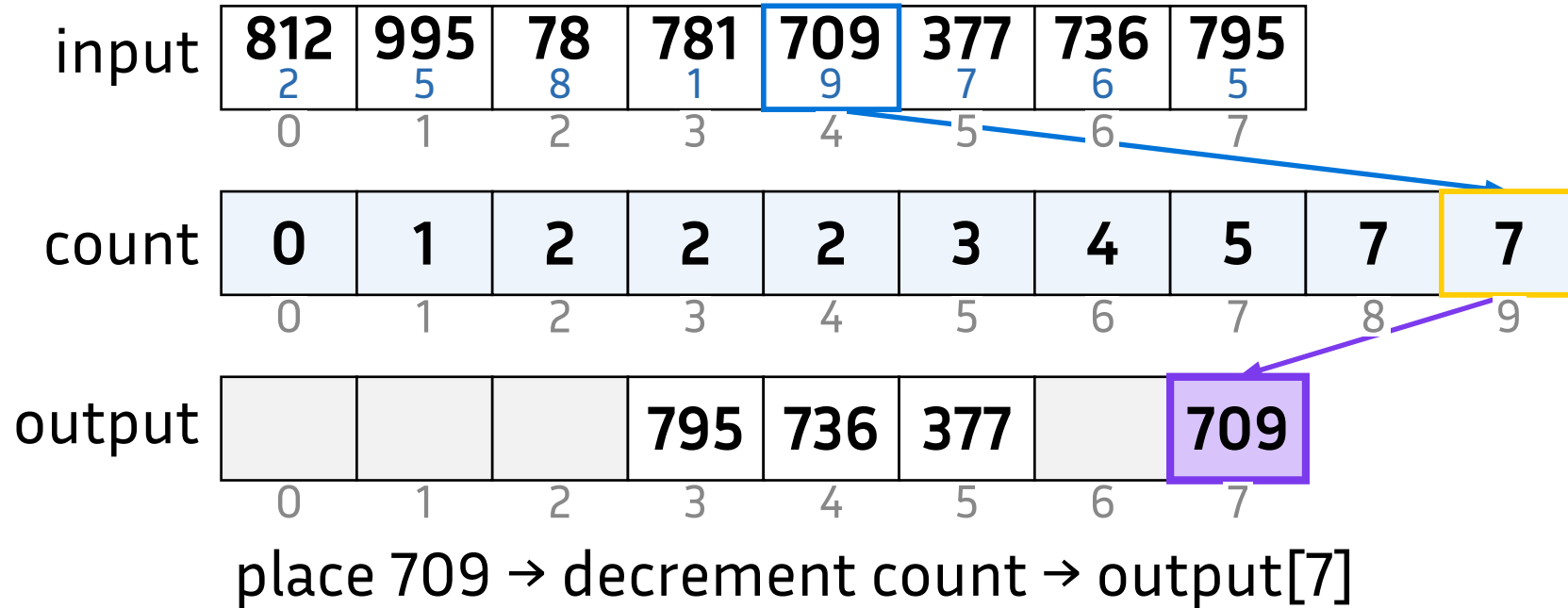
Linear Sorts – Radix Example



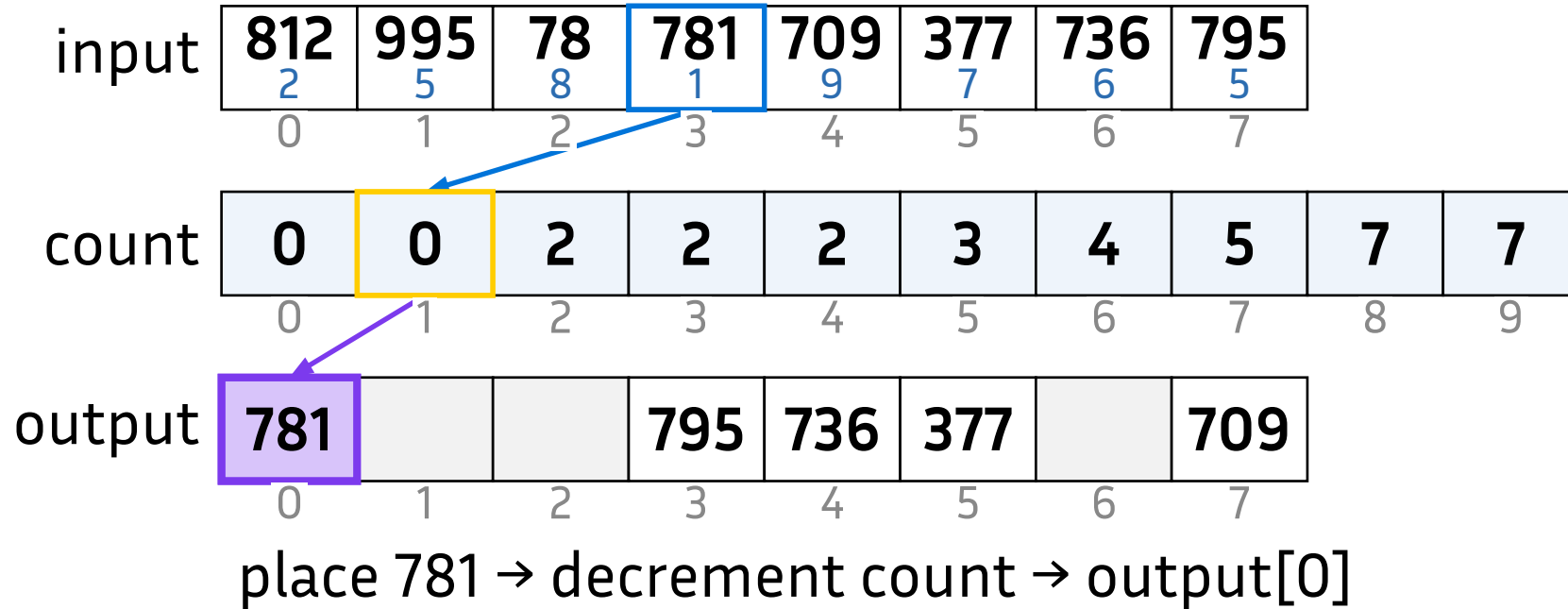
Linear Sorts – Radix Example



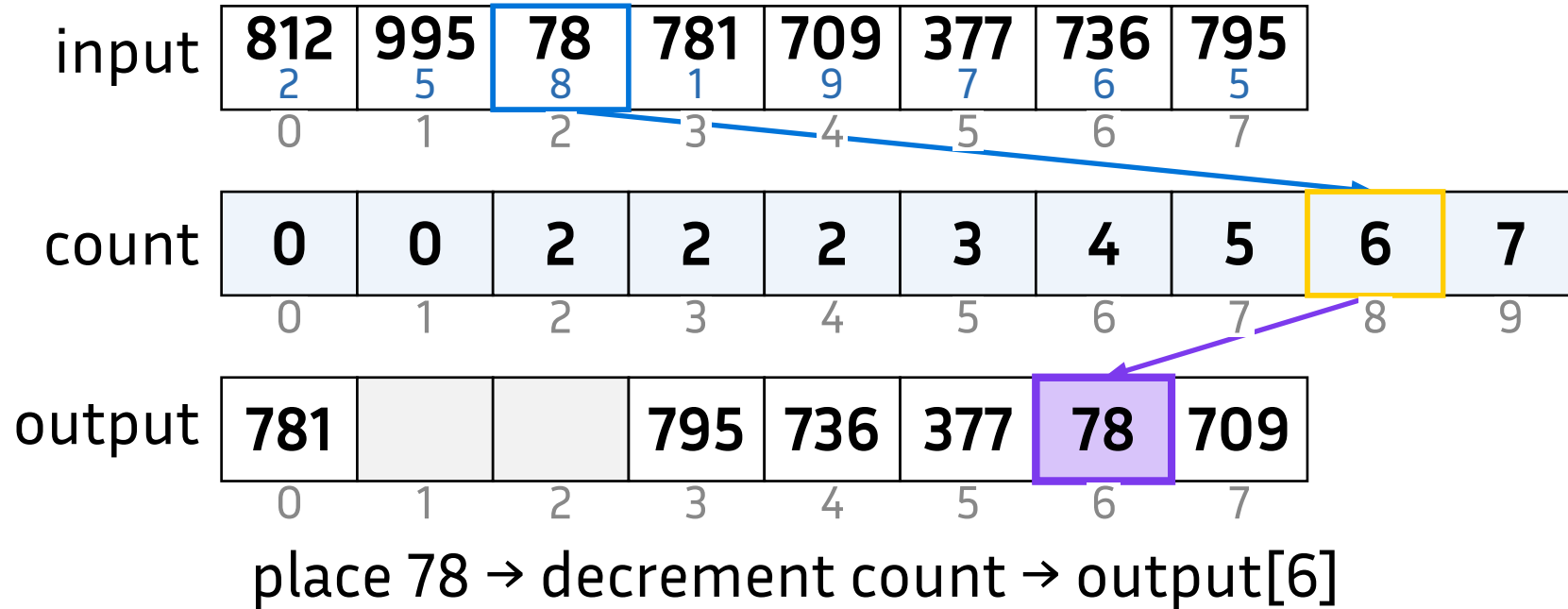
Linear Sorts – Radix Example



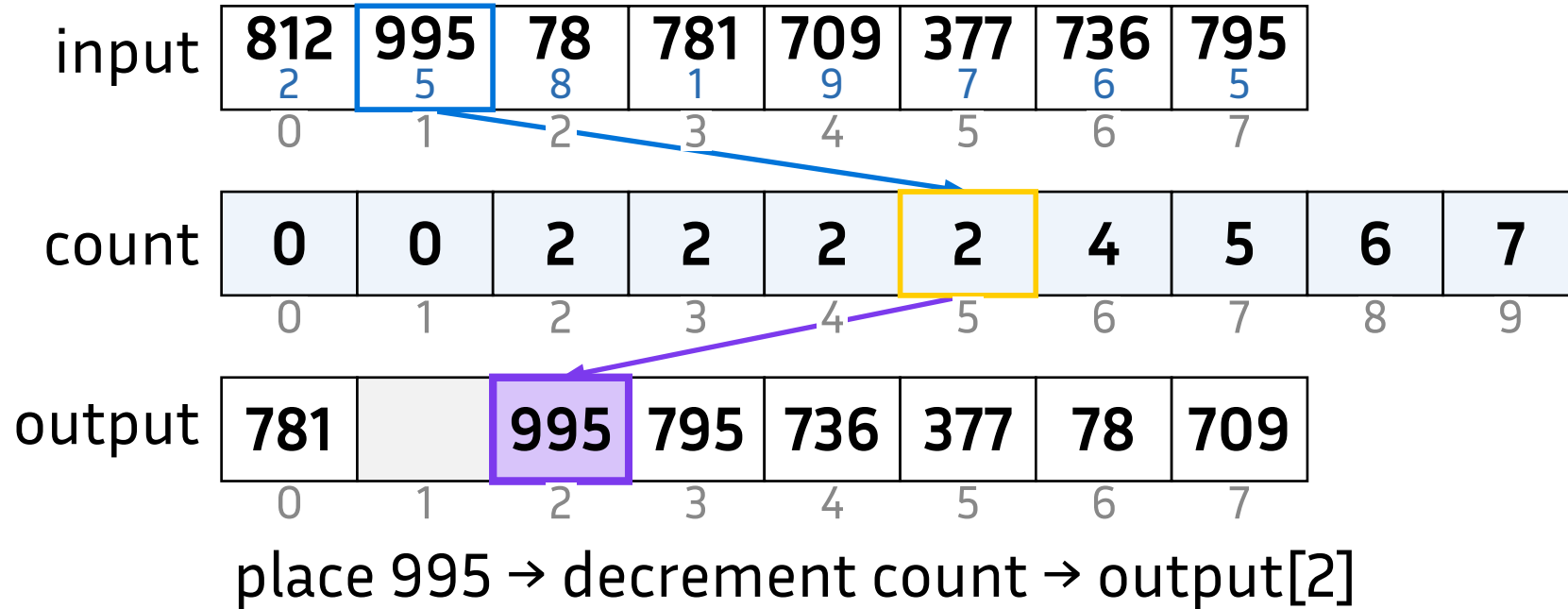
Linear Sorts – Radix Example



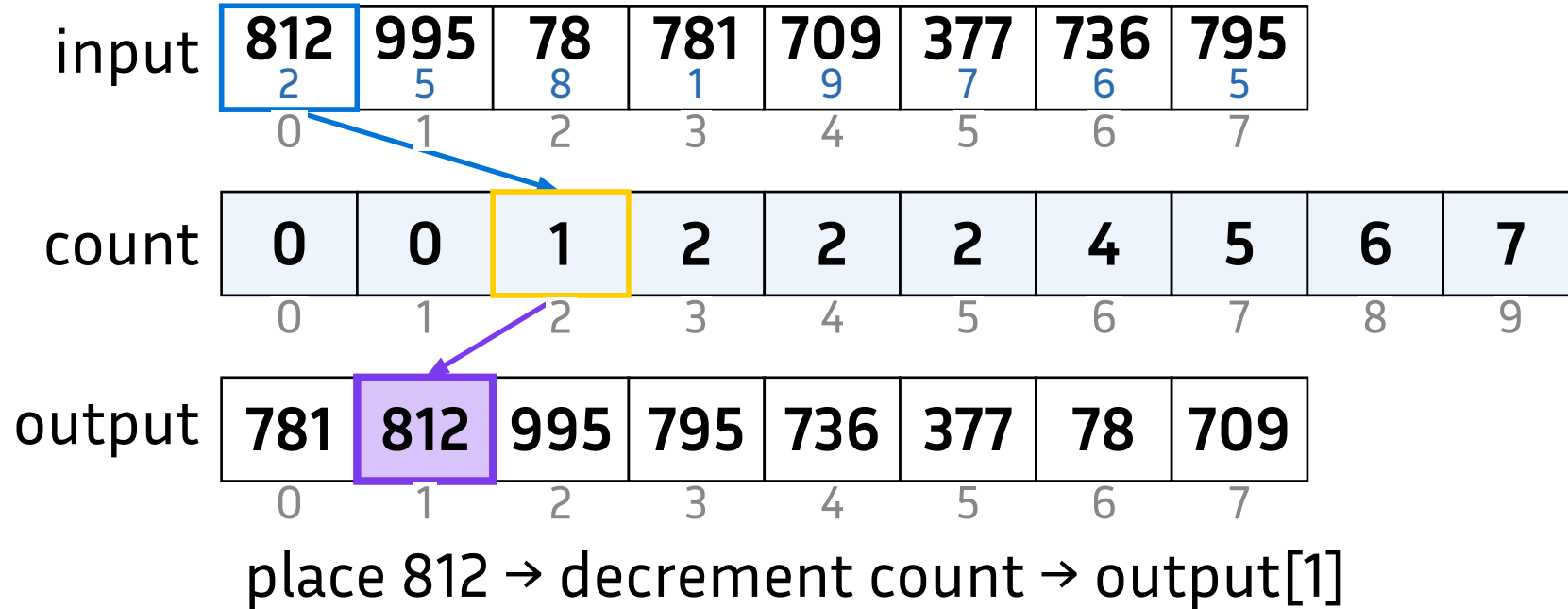
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

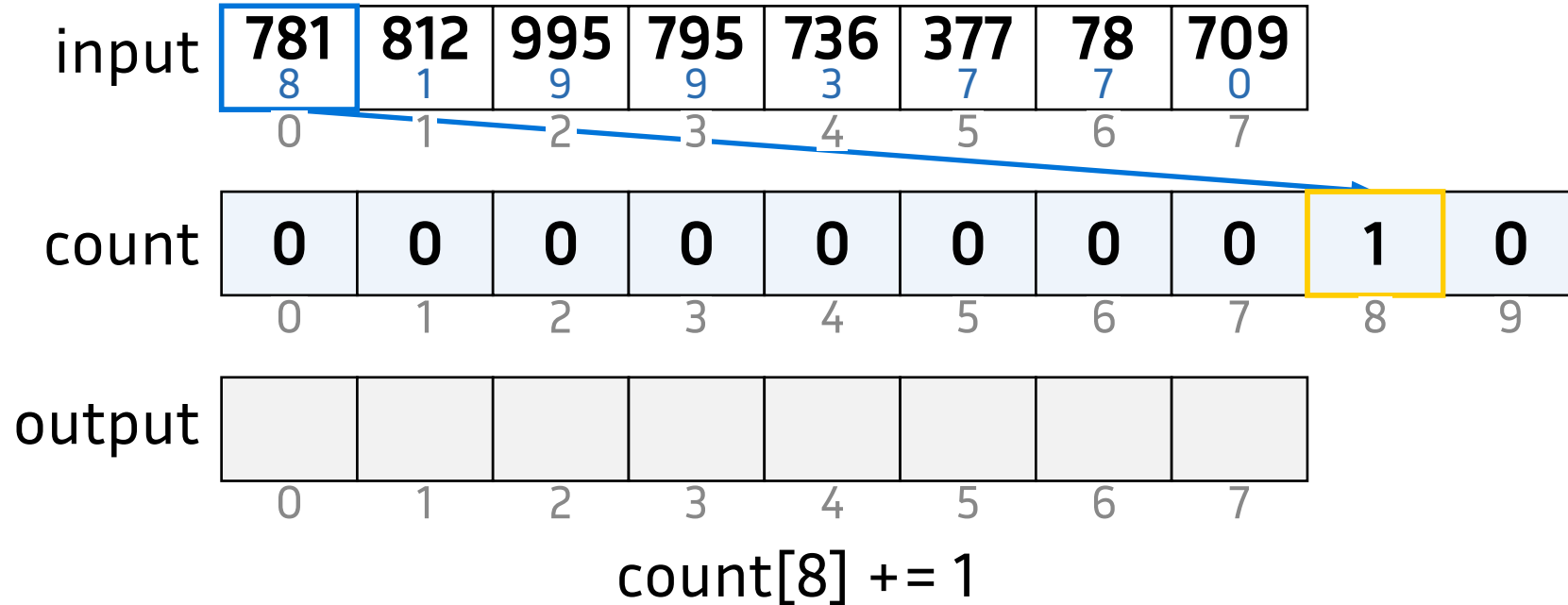


Linear Sorts – Radix Example

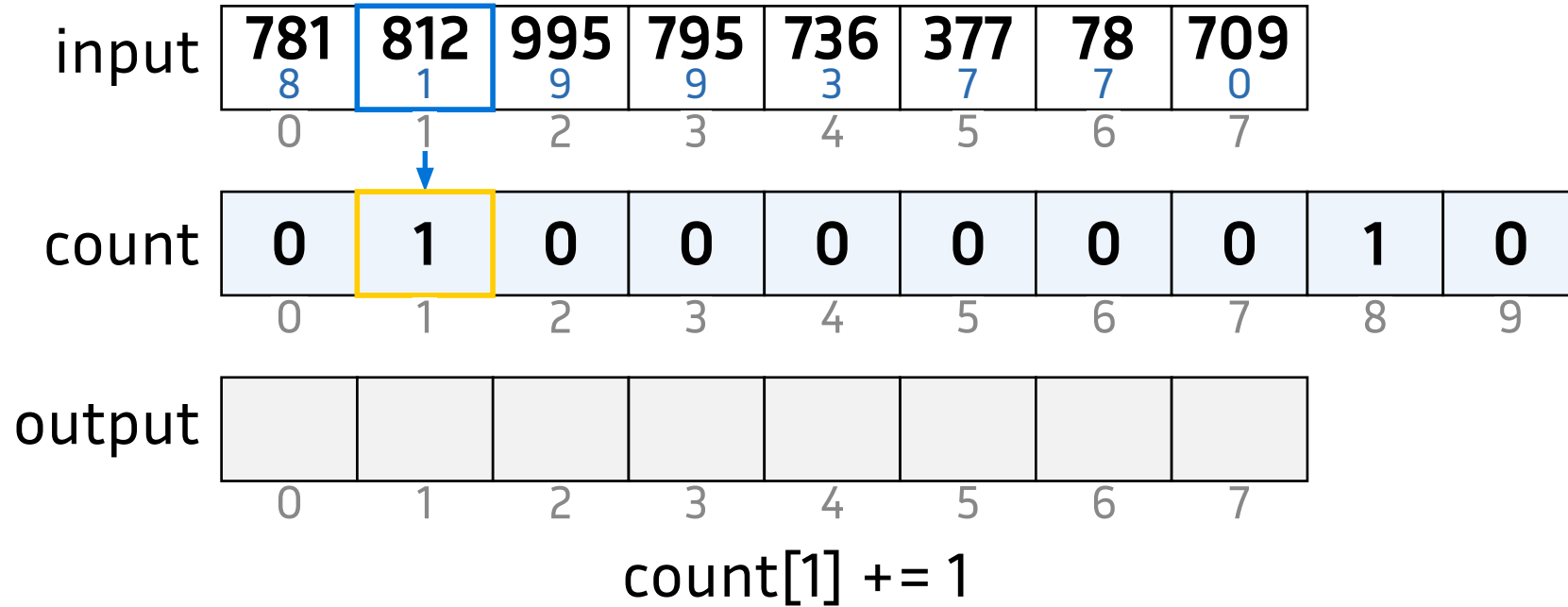
input	781	812	995	795	736	377	78	709		
	8	1	9	9	3	7	7	0		
	0	1	2	3	4	5	6	7		
count	0	0	0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9
output										
	0	1	2	3	4	5	6	7		

pass 2: tens digit

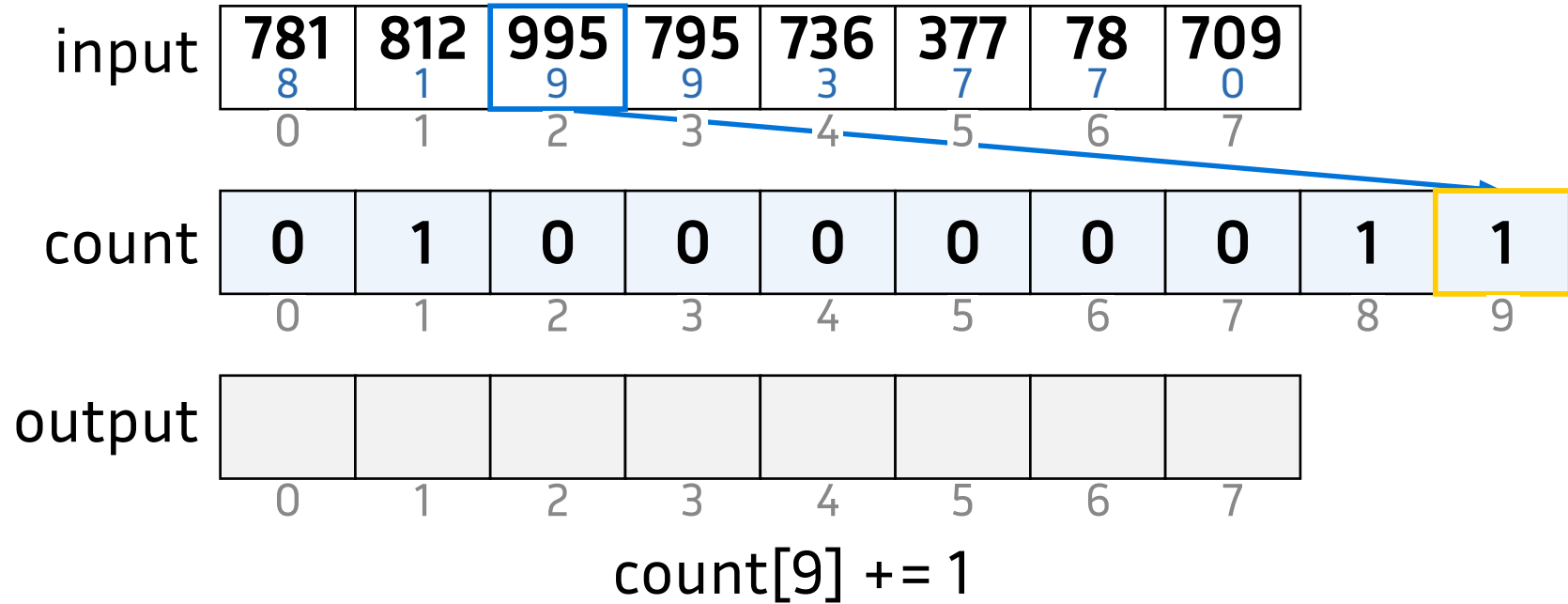
Linear Sorts – Radix Example



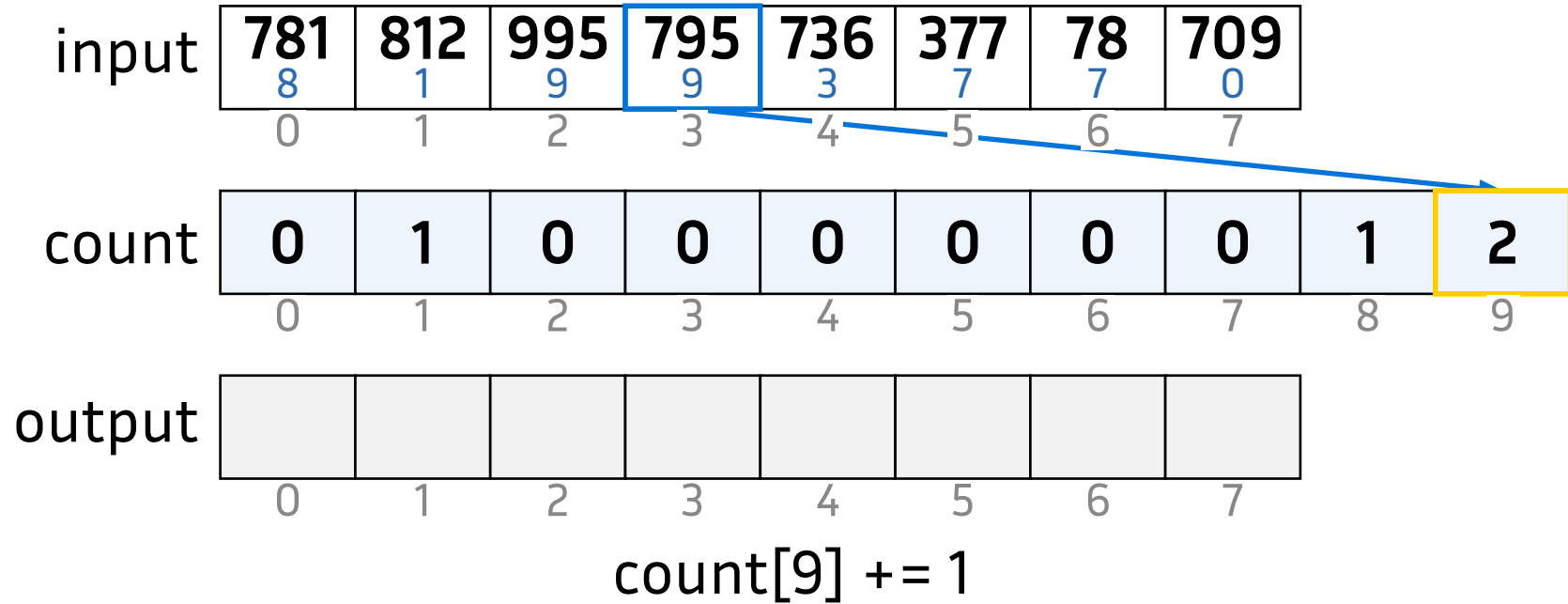
Linear Sorts – Radix Example



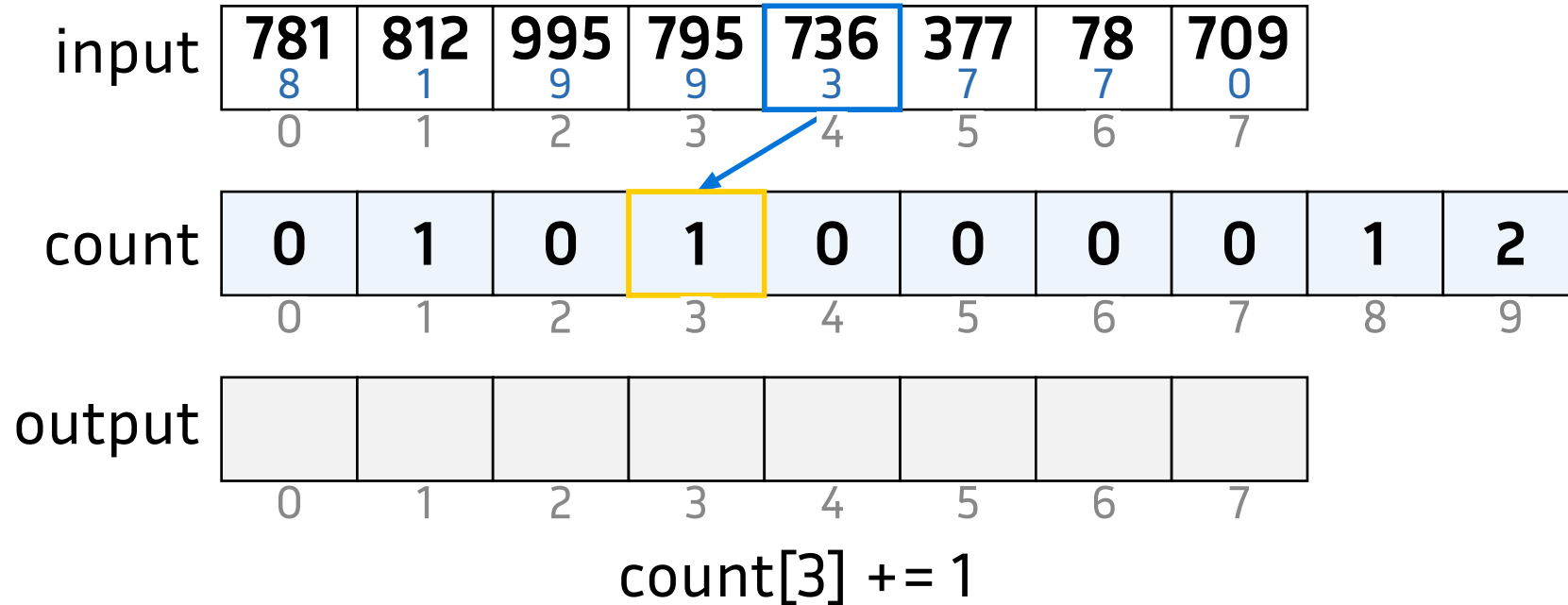
Linear Sorts – Radix Example



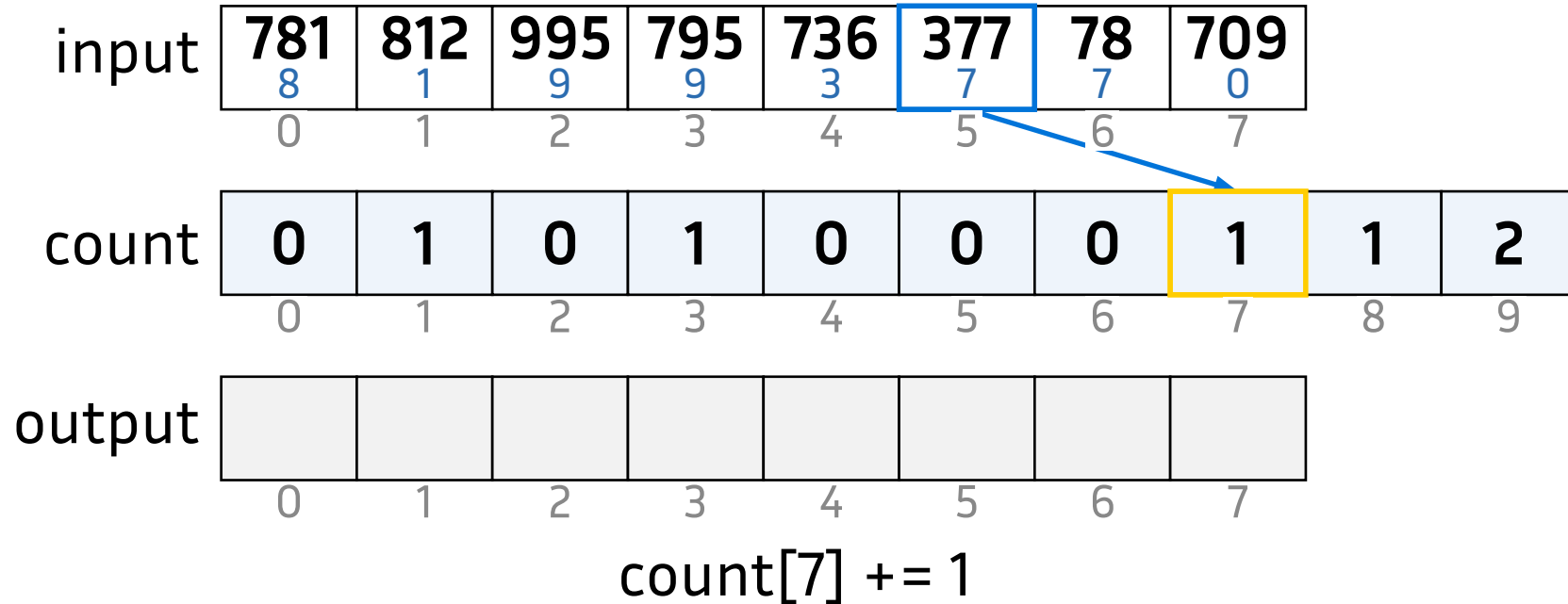
Linear Sorts – Radix Example



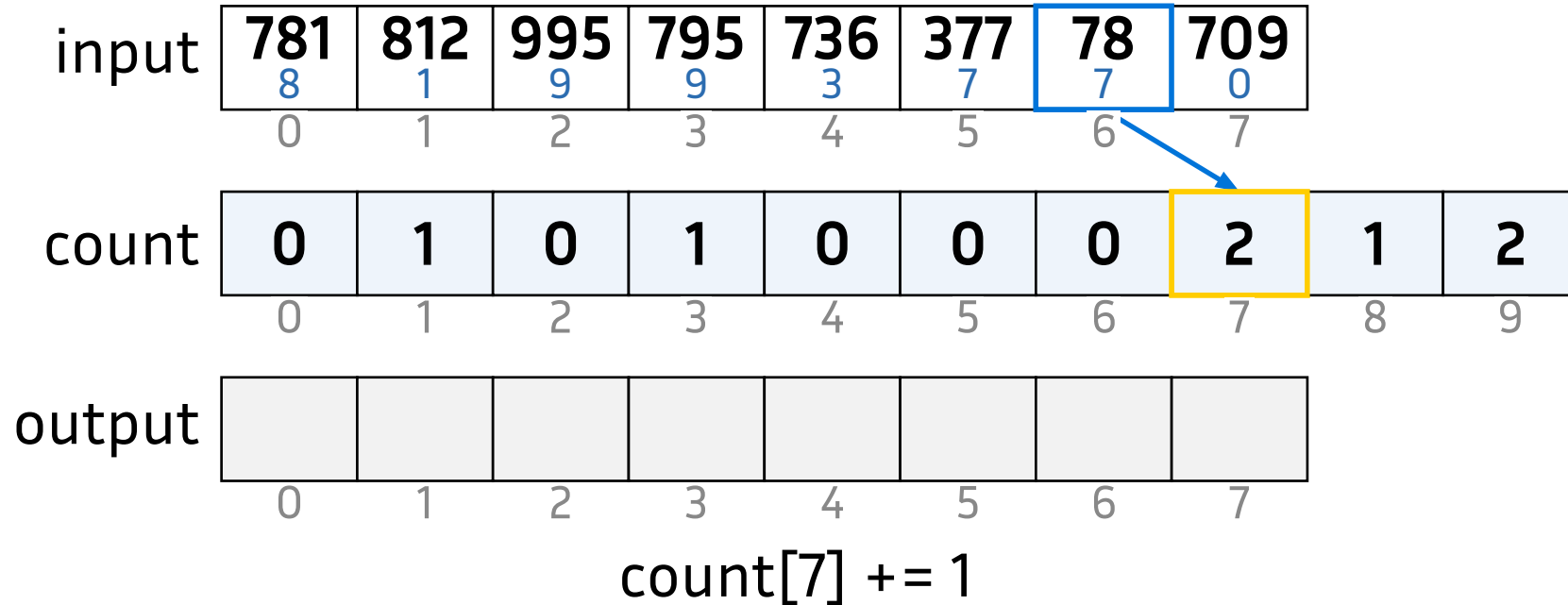
Linear Sorts – Radix Example



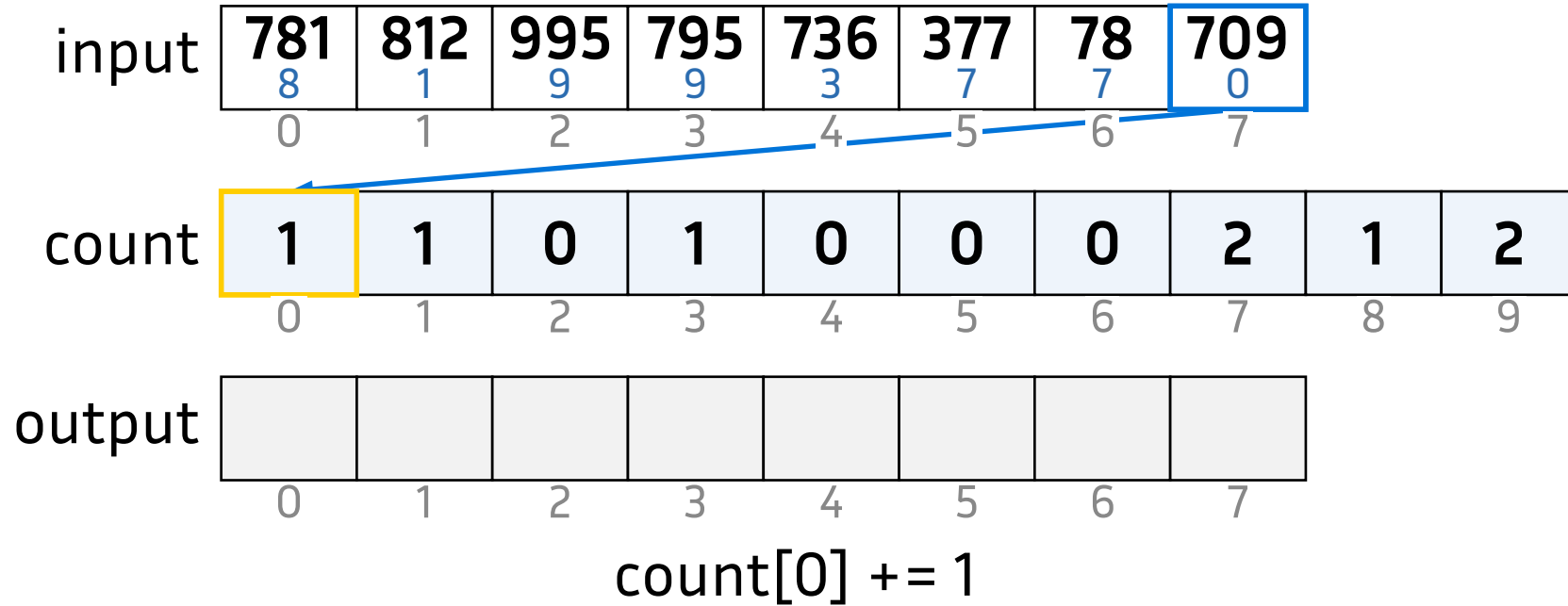
Linear Sorts – Radix Example



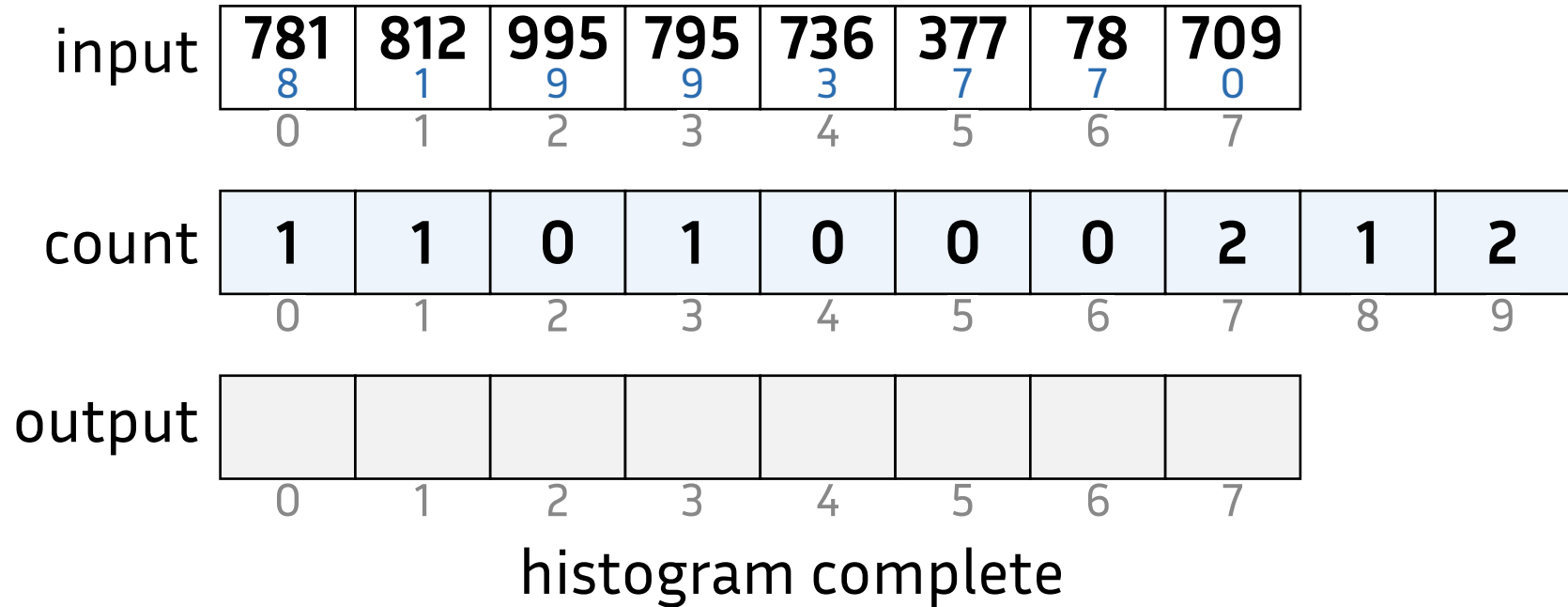
Linear Sorts – Radix Example



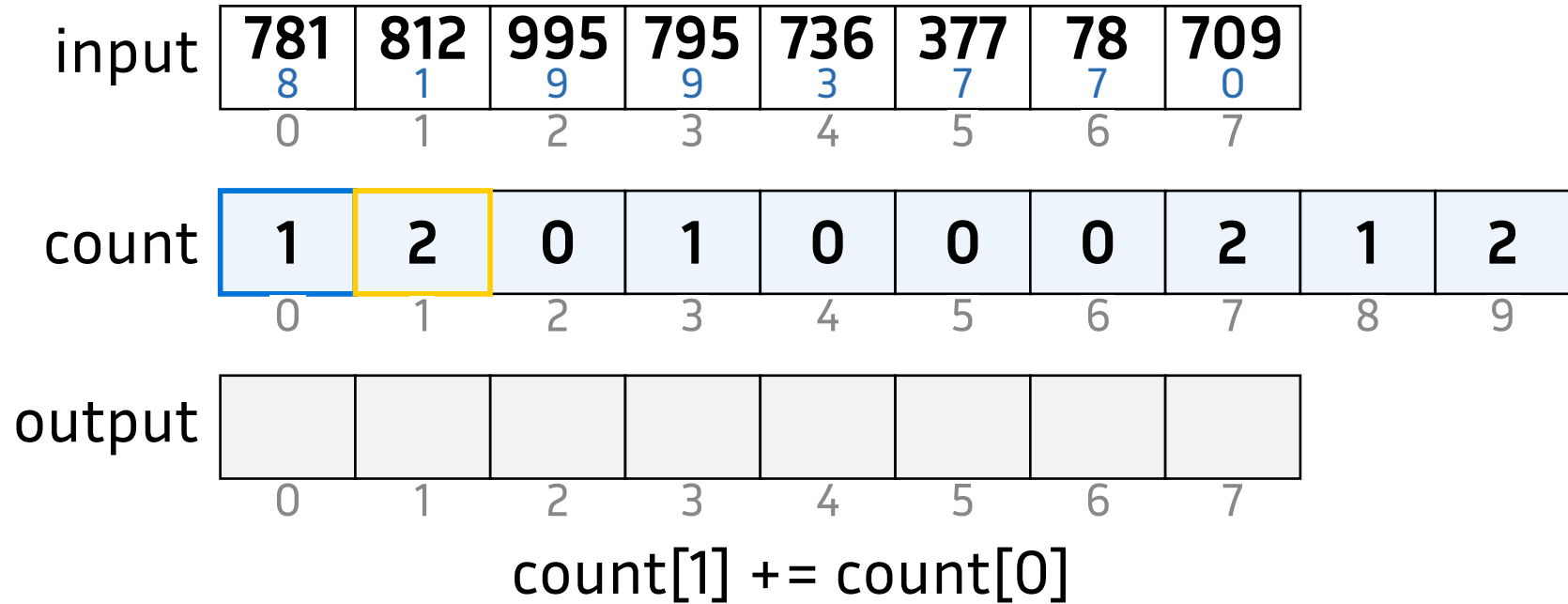
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

input	781	812	995	795	736	377	78	709		
	8	1	9	9	3	7	7	0		
	0	1	2	3	4	5	6	7		
count	1	2	2	1	0	0	0	2	1	2
	0	1	2	3	4	5	6	7	8	9
output										
	0	1	2	3	4	5	6	7		

$\text{count}[2] += \text{count}[1]$

Linear Sorts – Radix Example

input

781	812	995	795	736	377	78	709
8	1	9	9	3	7	7	0
0	1	2	3	4	5	6	7

count

1	2	2	3	0	0	0	2	1	2
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[3] += count[2]

Linear Sorts – Radix Example

input

781	812	995	795	736	377	78	709
8	1	9	9	3	7	7	0
0	1	2	3	4	5	6	7

count

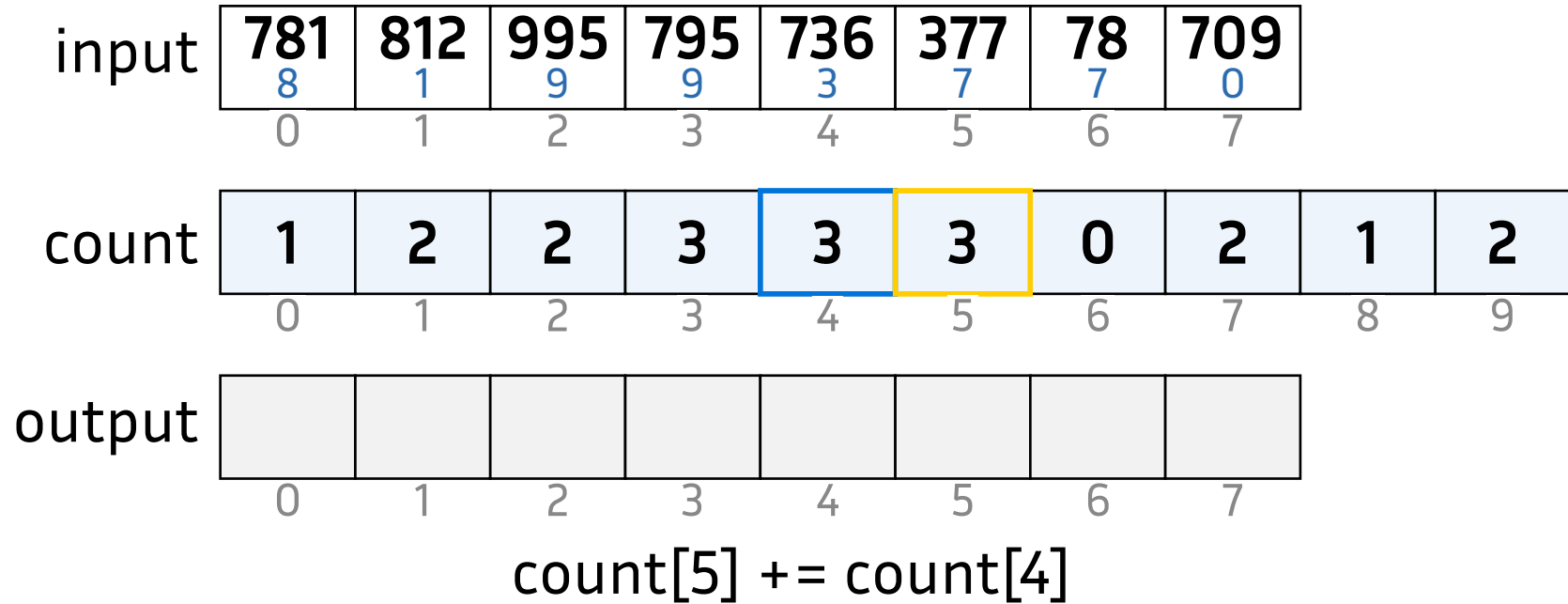
1	2	2	3	3	0	0	2	1	2
0	1	2	3	4	5	6	7	8	9

output

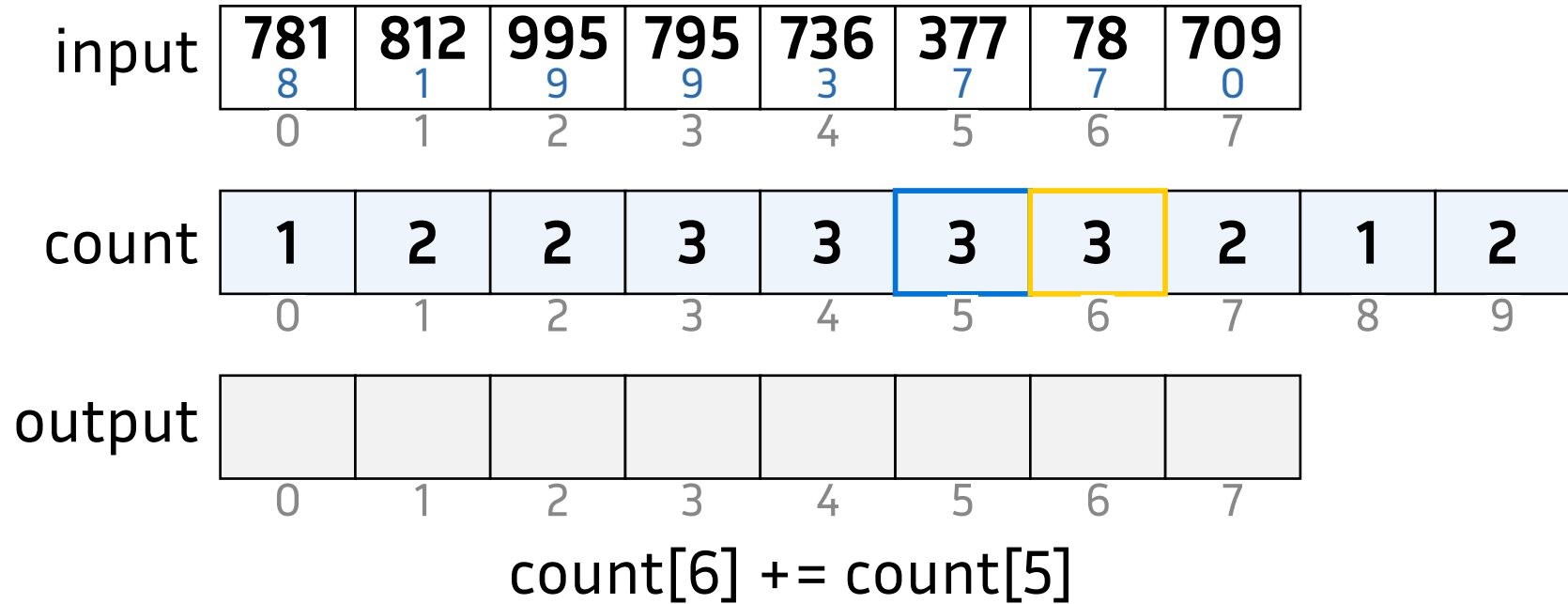
0	1	2	3	4	5	6	7

count[4] += count[3]

Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

input

781	812	995	795	736	377	78	709
8	1	9	9	3	7	7	0
0	1	2	3	4	5	6	7

count

1	2	2	3	3	3	3	5	1	2
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[7] += count[6]

Linear Sorts – Radix Example

input	781	812	995	795	736	377	78	709	
	8	1	9	9	3	7	7	0	
	0	1	2	3	4	5	6	7	
count	1	2	2	3	3	3	3	5	6
	0	1	2	3	4	5	6	7	8
output									
	0	1	2	3	4	5	6	7	

count[8] += count[7]

Linear Sorts – Radix Example

input

781	812	995	795	736	377	78	709
8	1	9	9	3	7	7	0
0	1	2	3	4	5	6	7

count

1	2	2	3	3	3	3	5	6	8
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

count[9] += count[8]

Linear Sorts – Radix Example

input

781	812	995	795	736	377	78	709
8	1	9	9	3	7	7	0
0	1	2	3	4	5	6	7

count

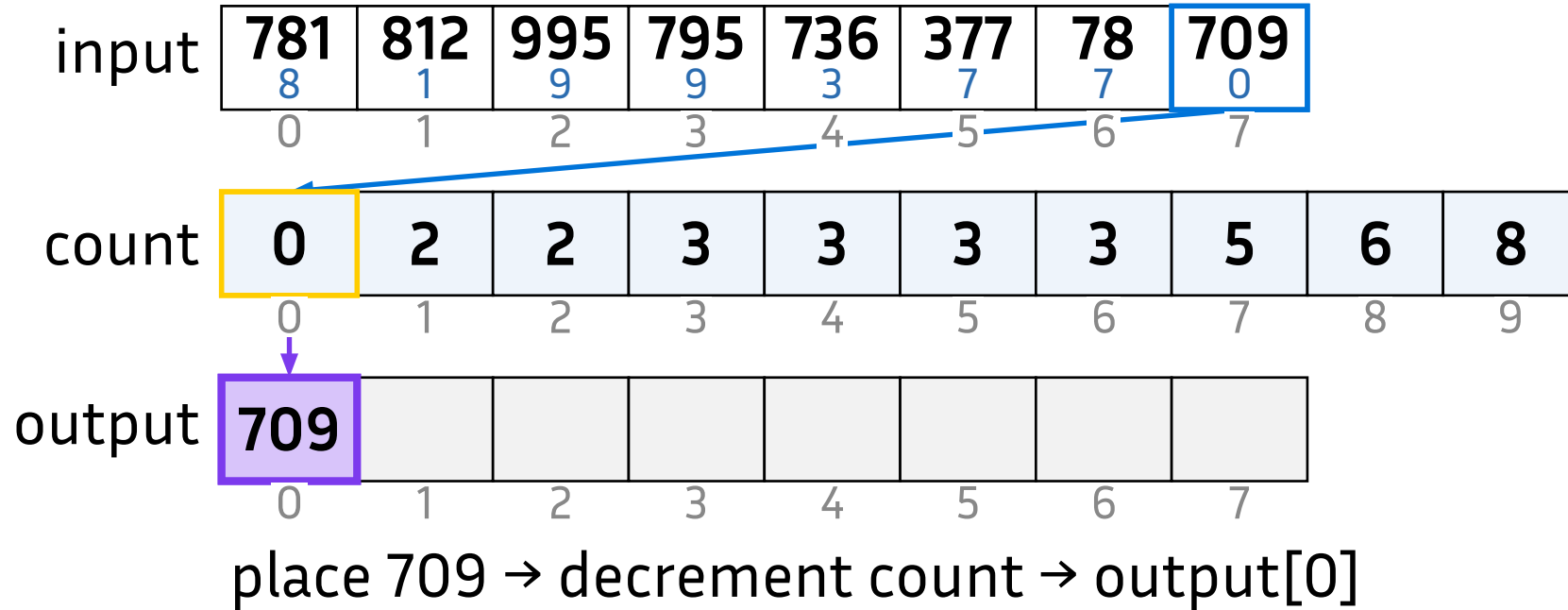
1	2	2	3	3	3	3	5	6	8
0	1	2	3	4	5	6	7	8	9

output

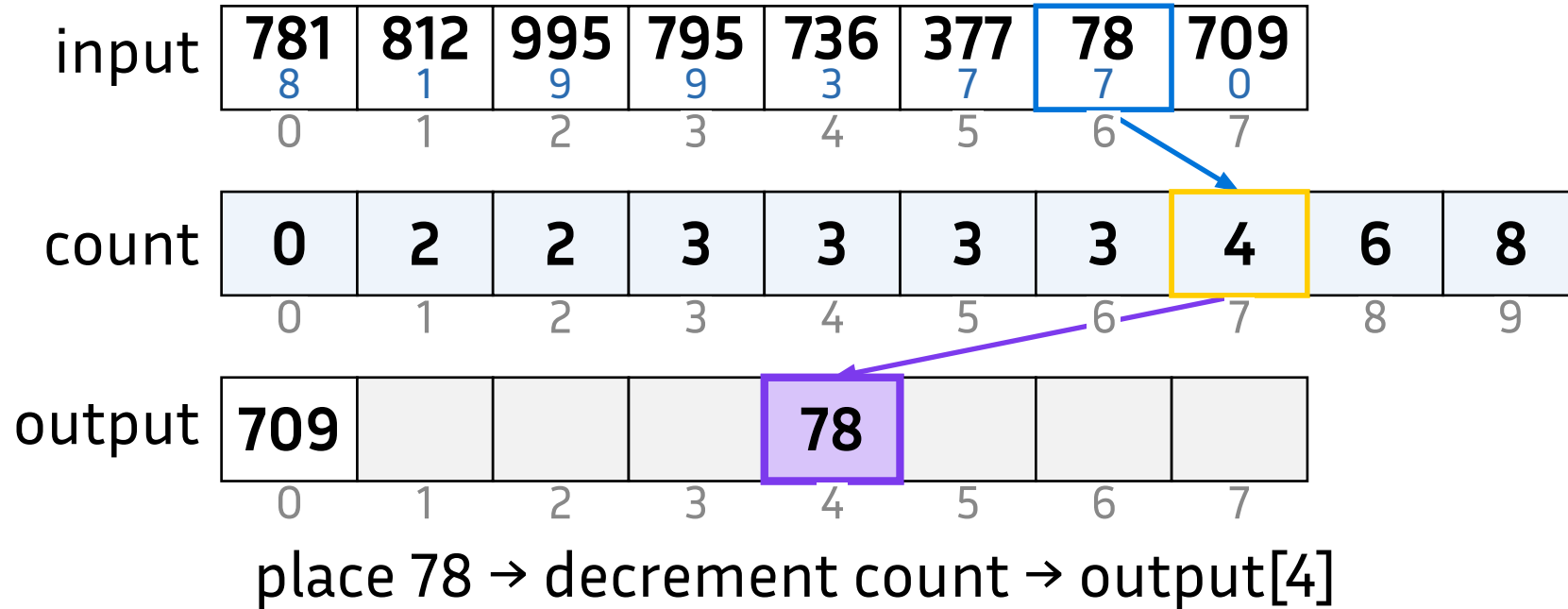
0	1	2	3	4	5	6	7

counts → end positions

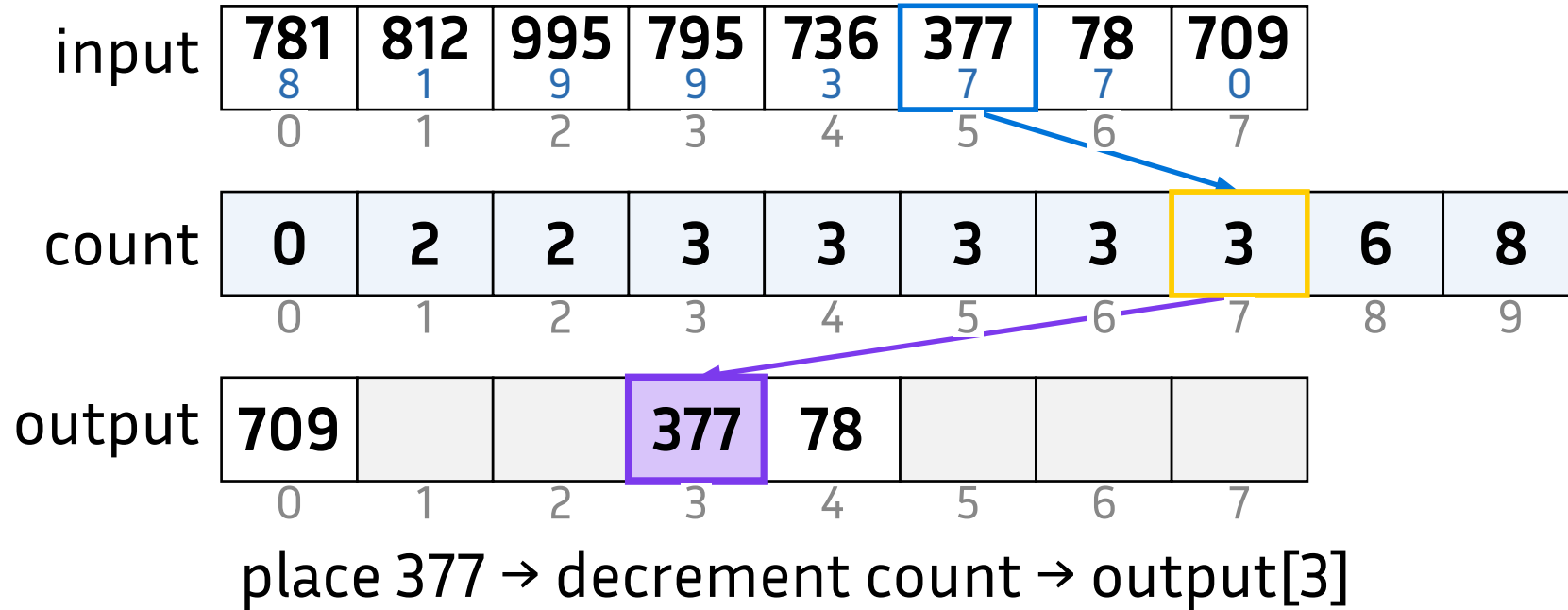
Linear Sorts – Radix Example



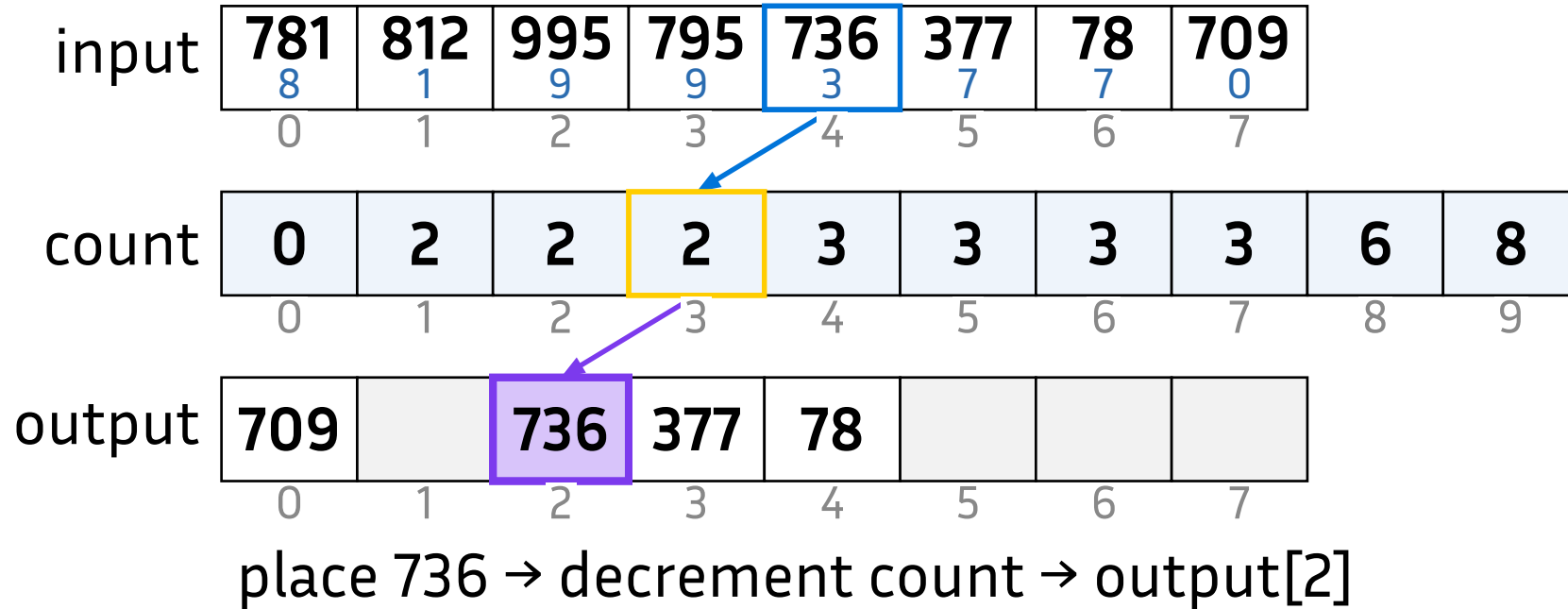
Linear Sorts – Radix Example



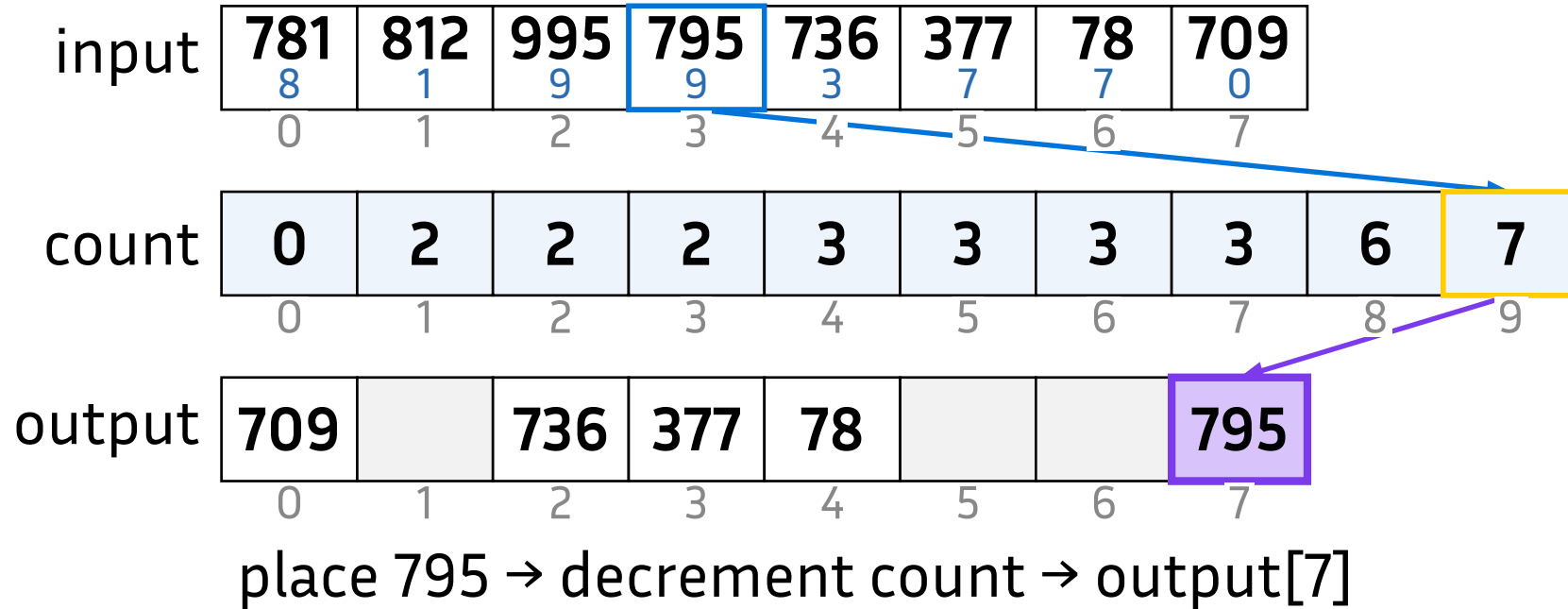
Linear Sorts – Radix Example



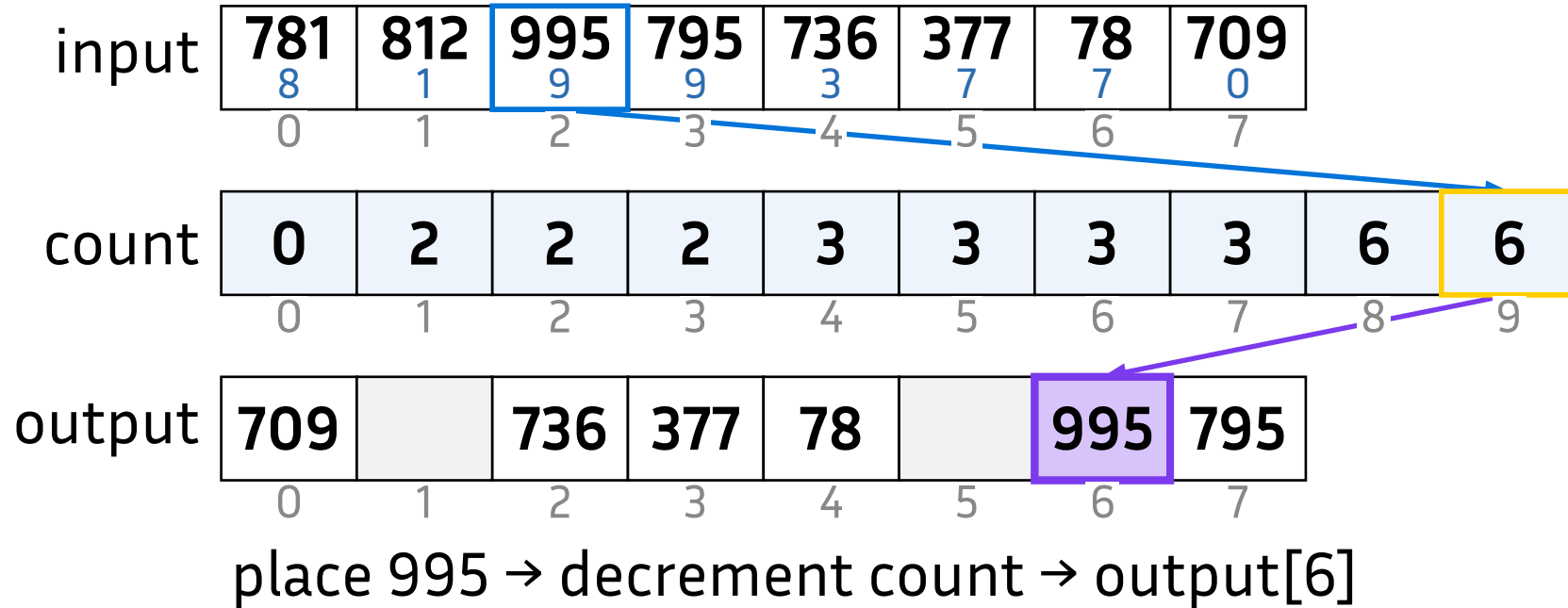
Linear Sorts – Radix Example



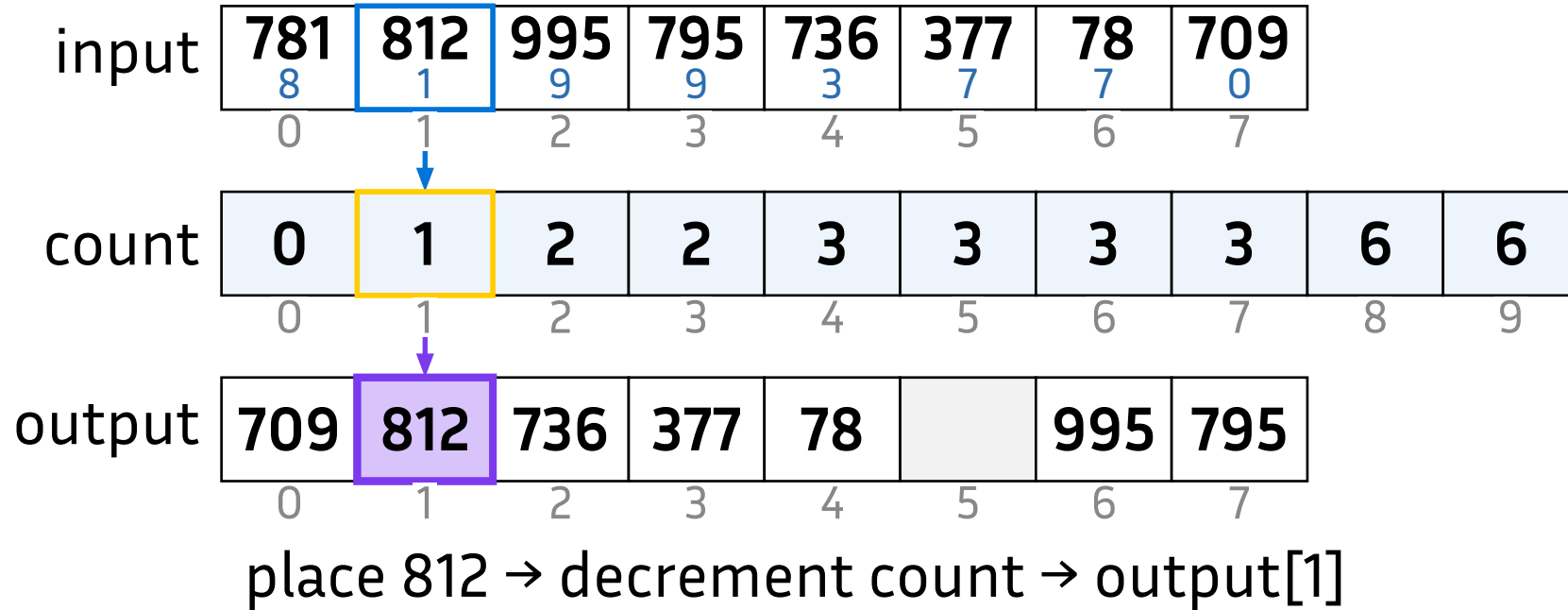
Linear Sorts – Radix Example



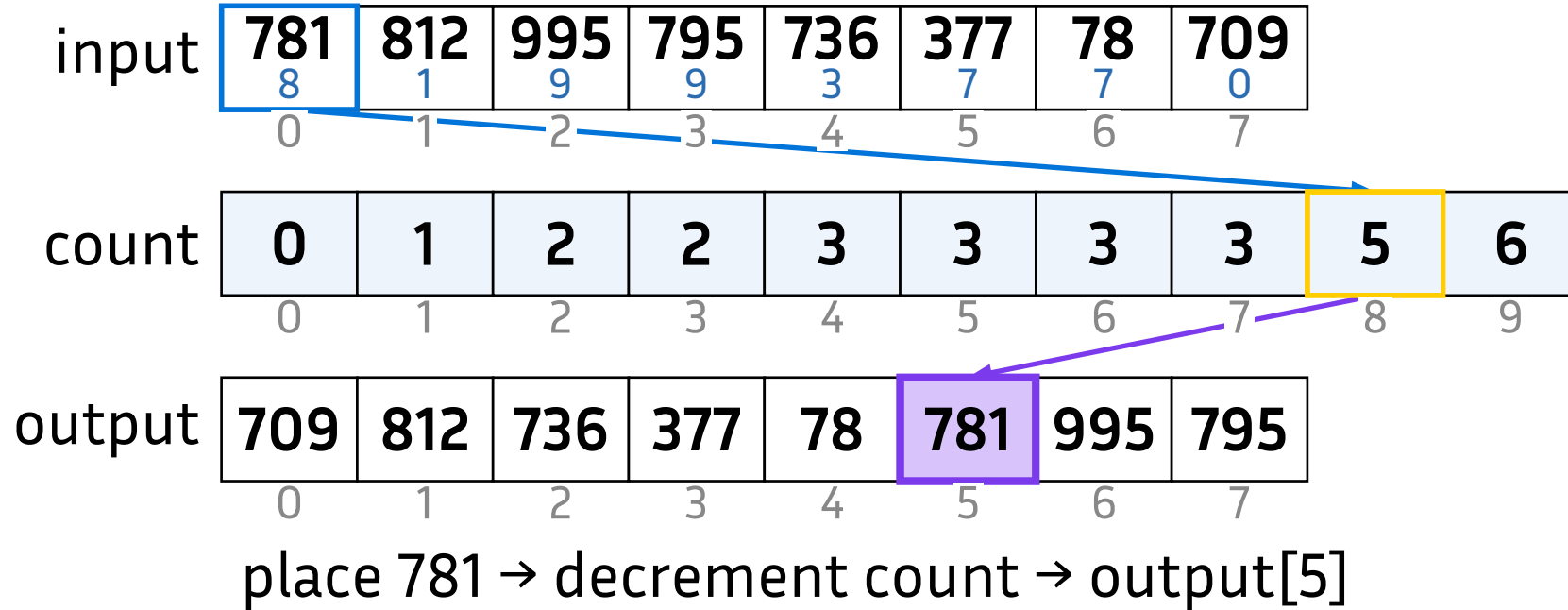
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

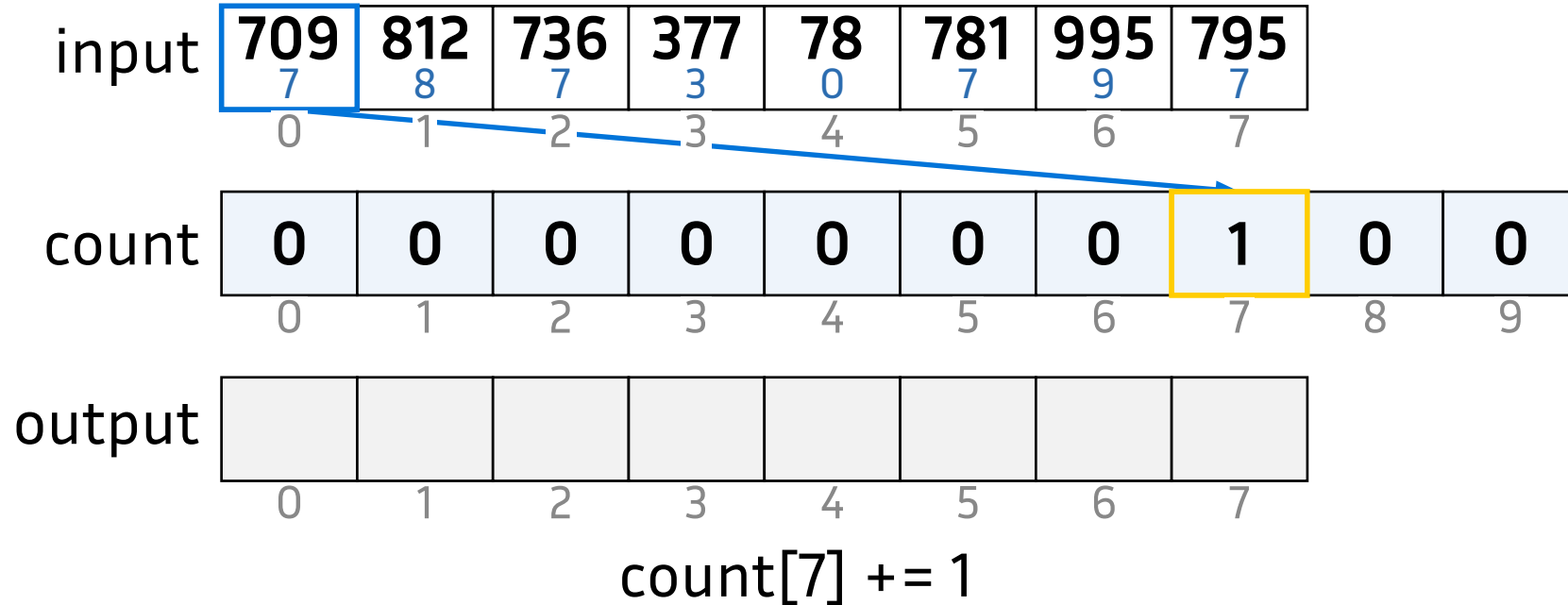


Linear Sorts – Radix Example

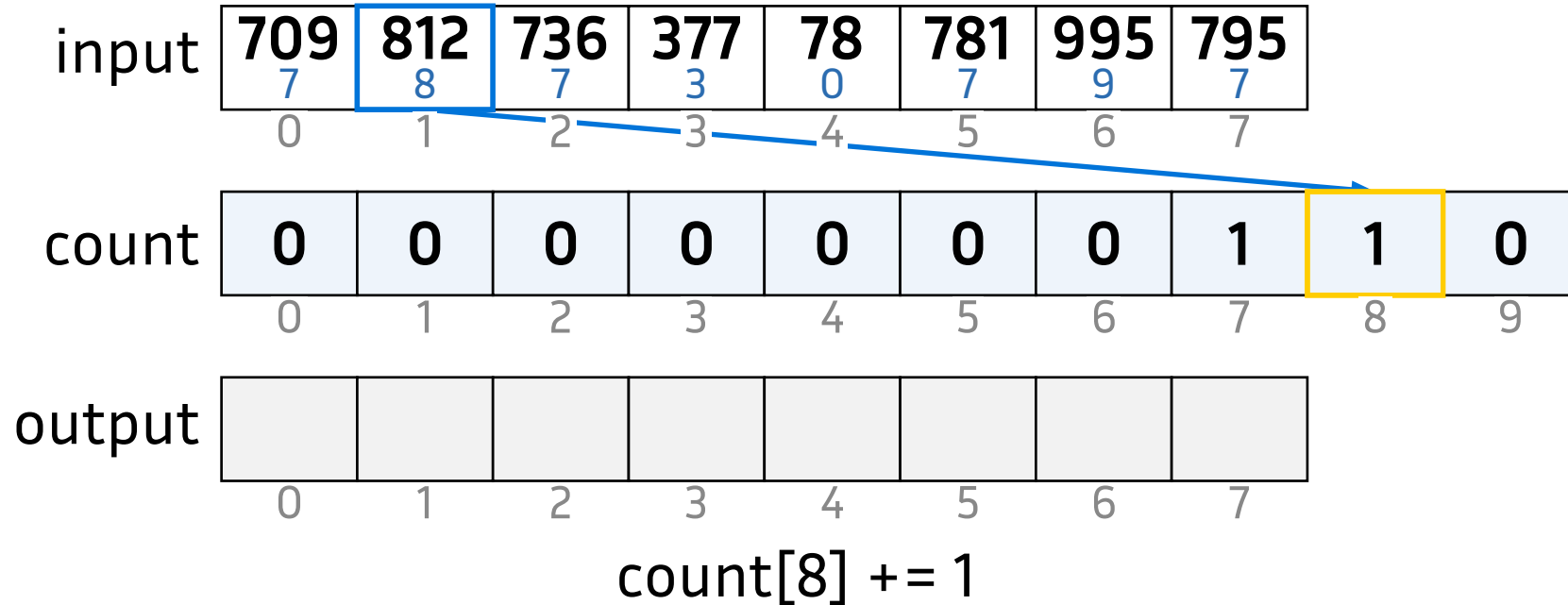
input	709	812	736	377	78	781	995	795		
	7	8	7	3	0	7	9	7		
	0	1	2	3	4	5	6	7		
count	0	0	0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9
output										
	0	1	2	3	4	5	6	7		

pass 3: hundreds digit

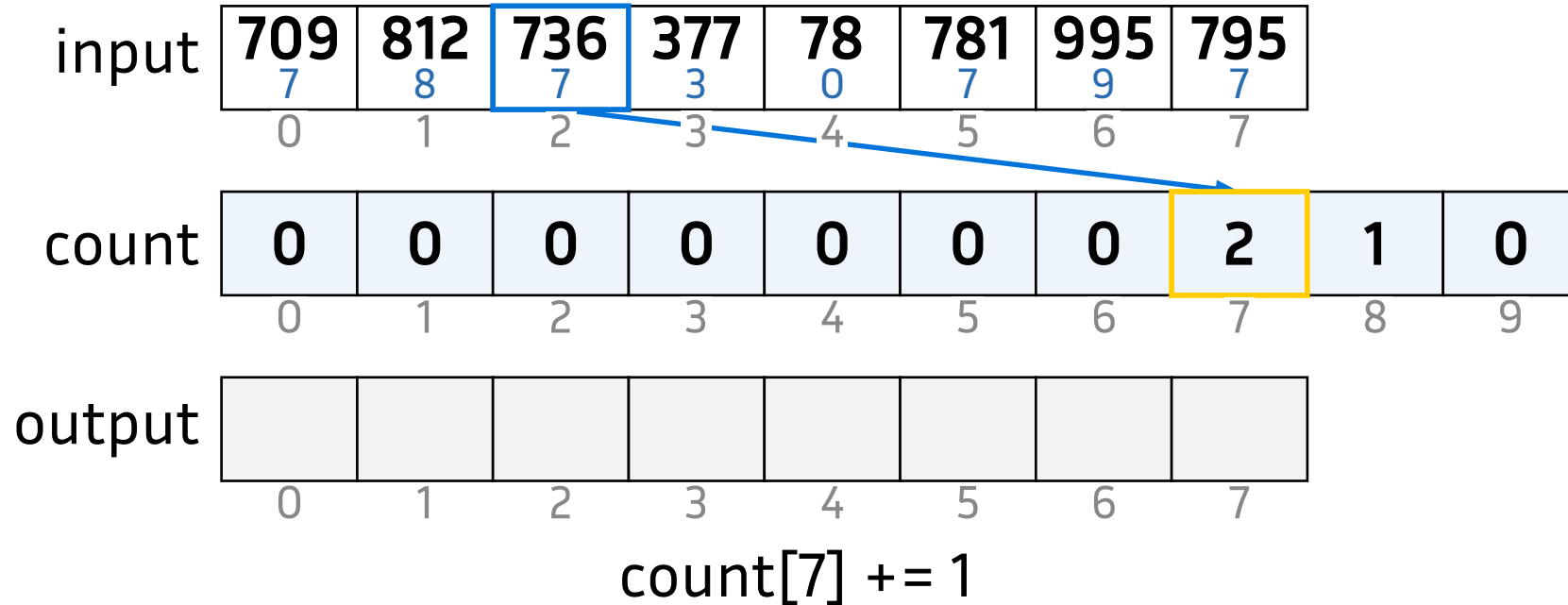
Linear Sorts – Radix Example



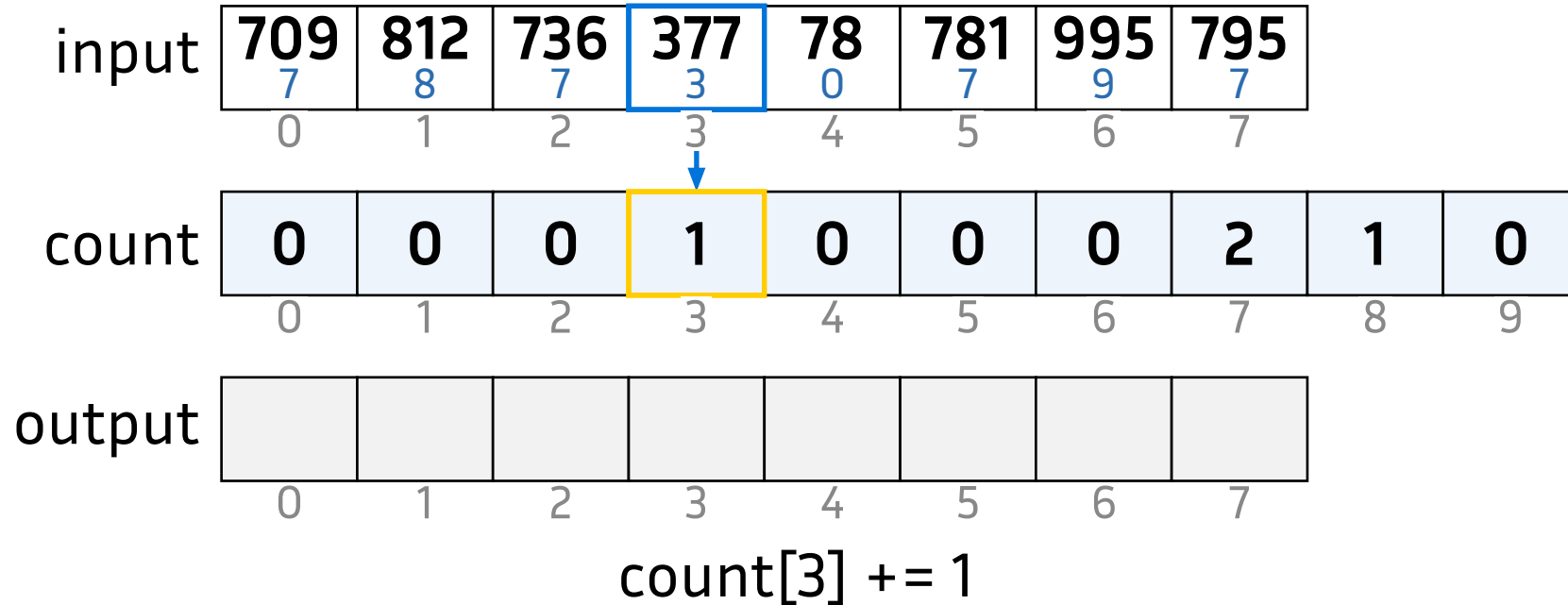
Linear Sorts – Radix Example



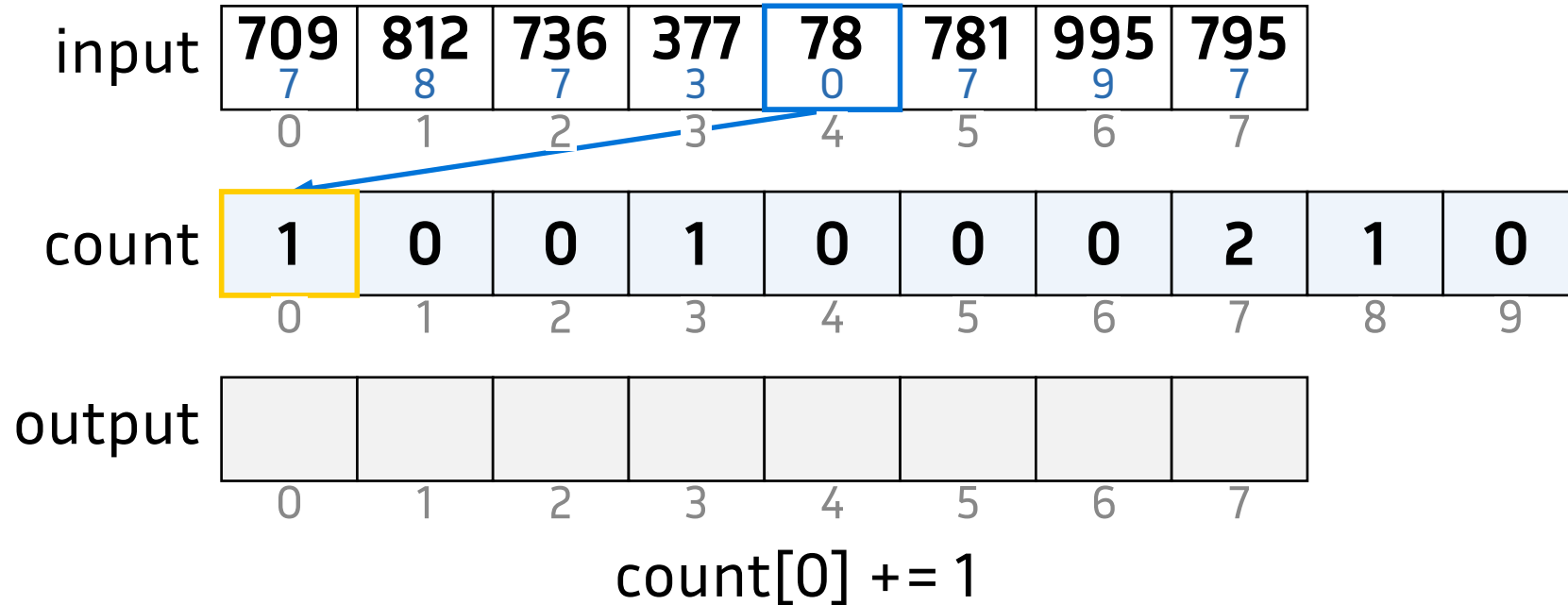
Linear Sorts – Radix Example



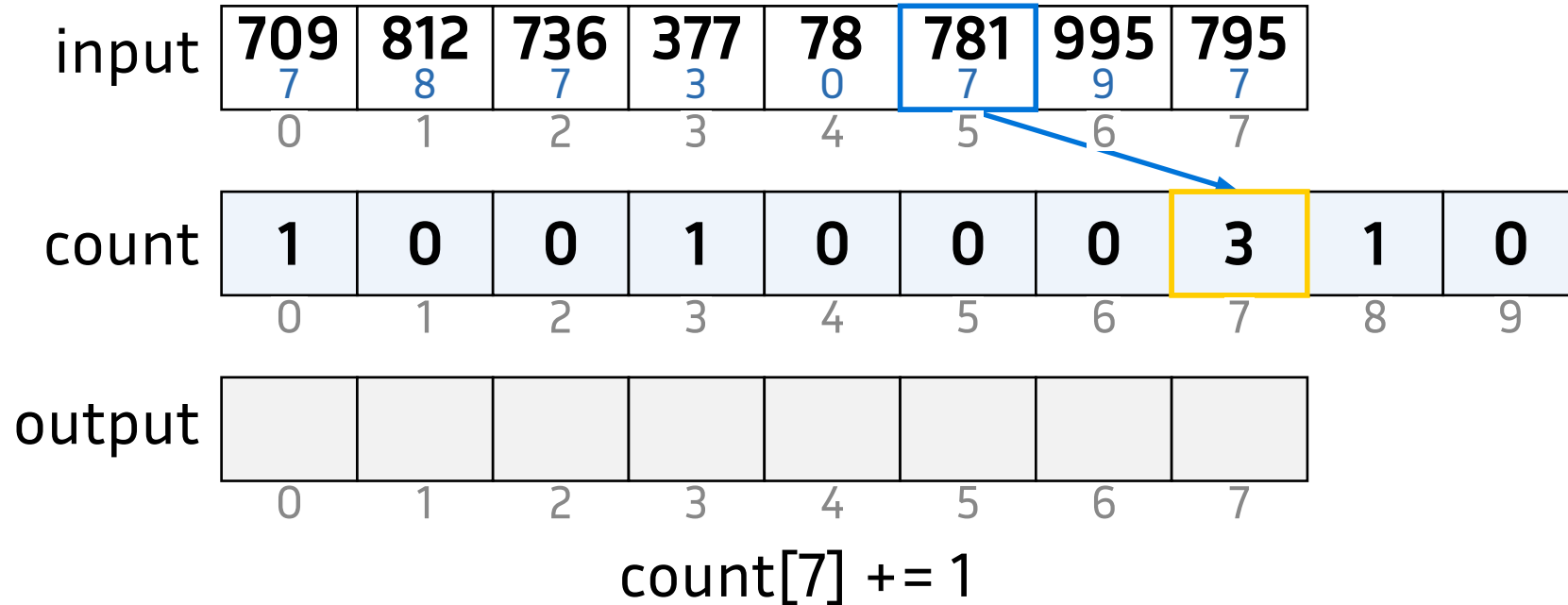
Linear Sorts – Radix Example



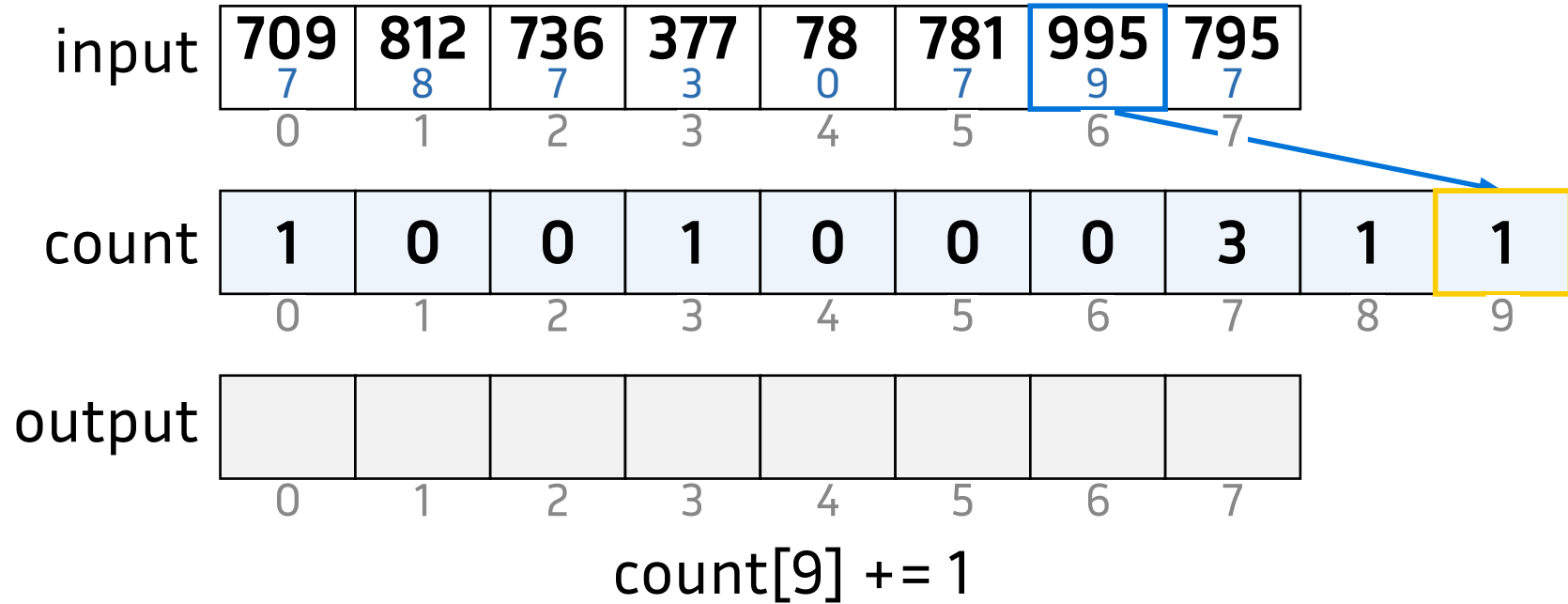
Linear Sorts – Radix Example



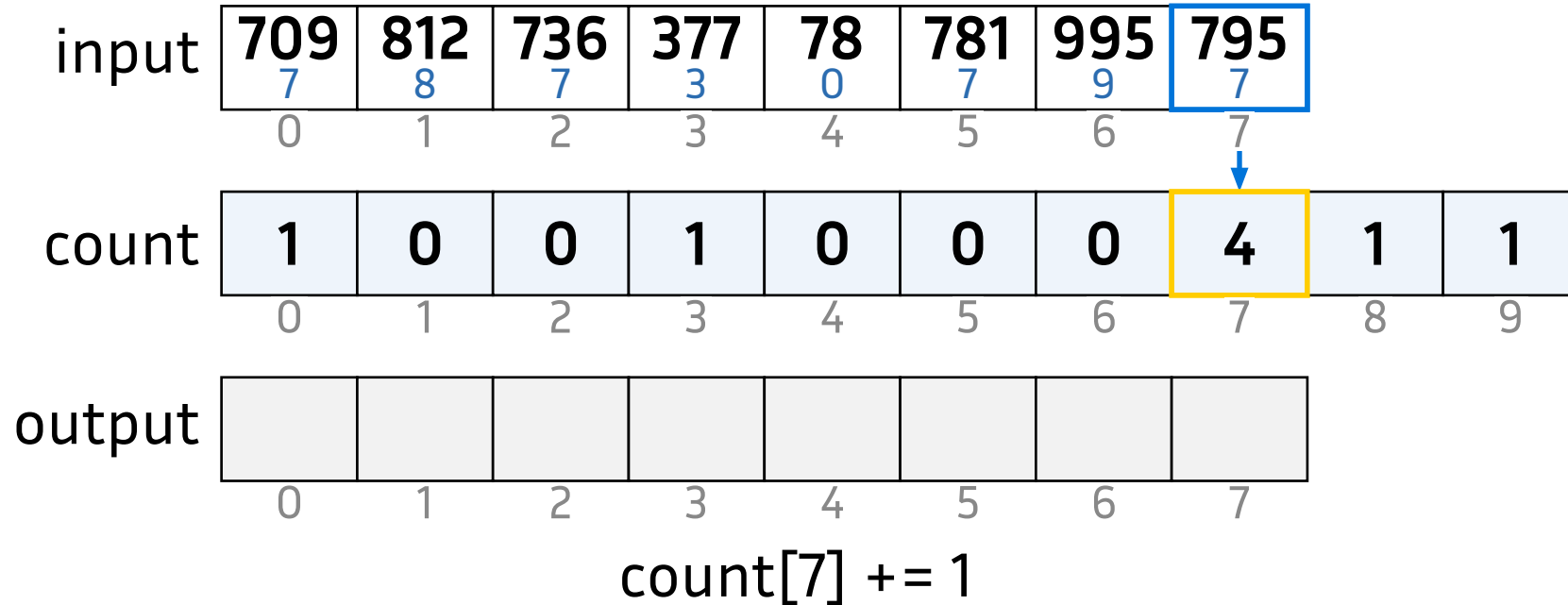
Linear Sorts – Radix Example



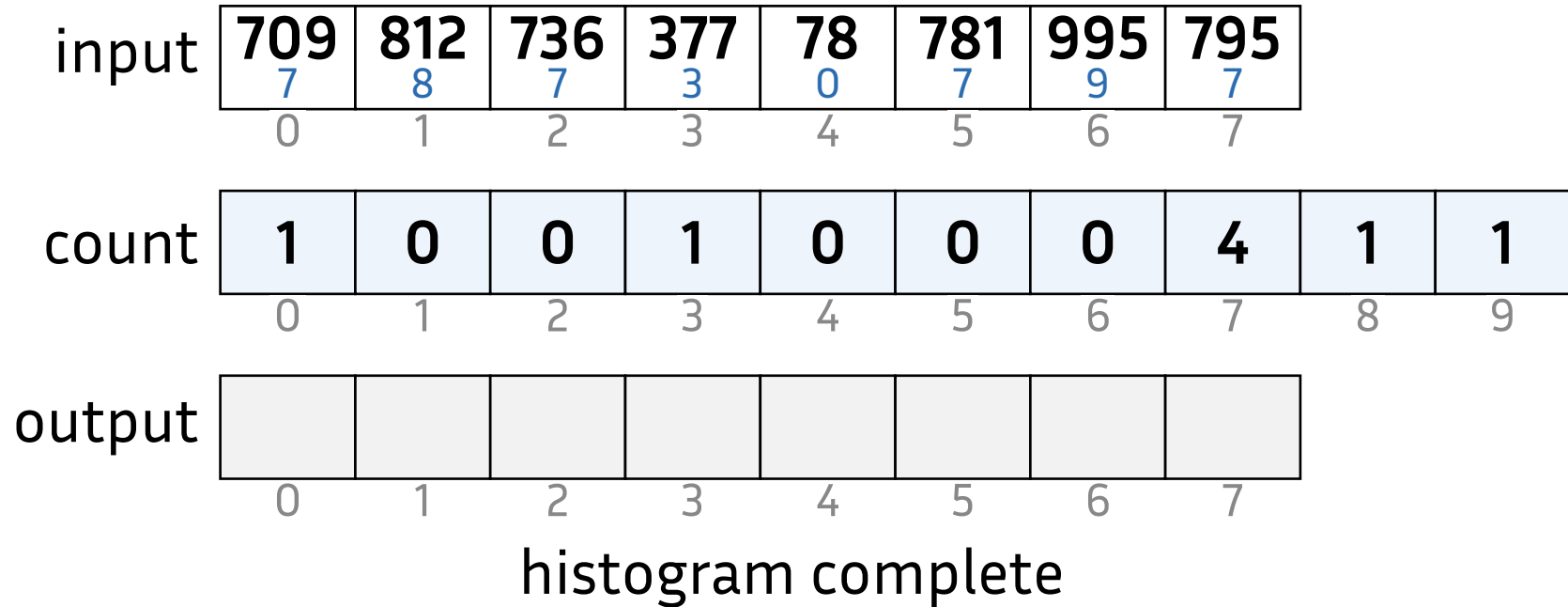
Linear Sorts – Radix Example



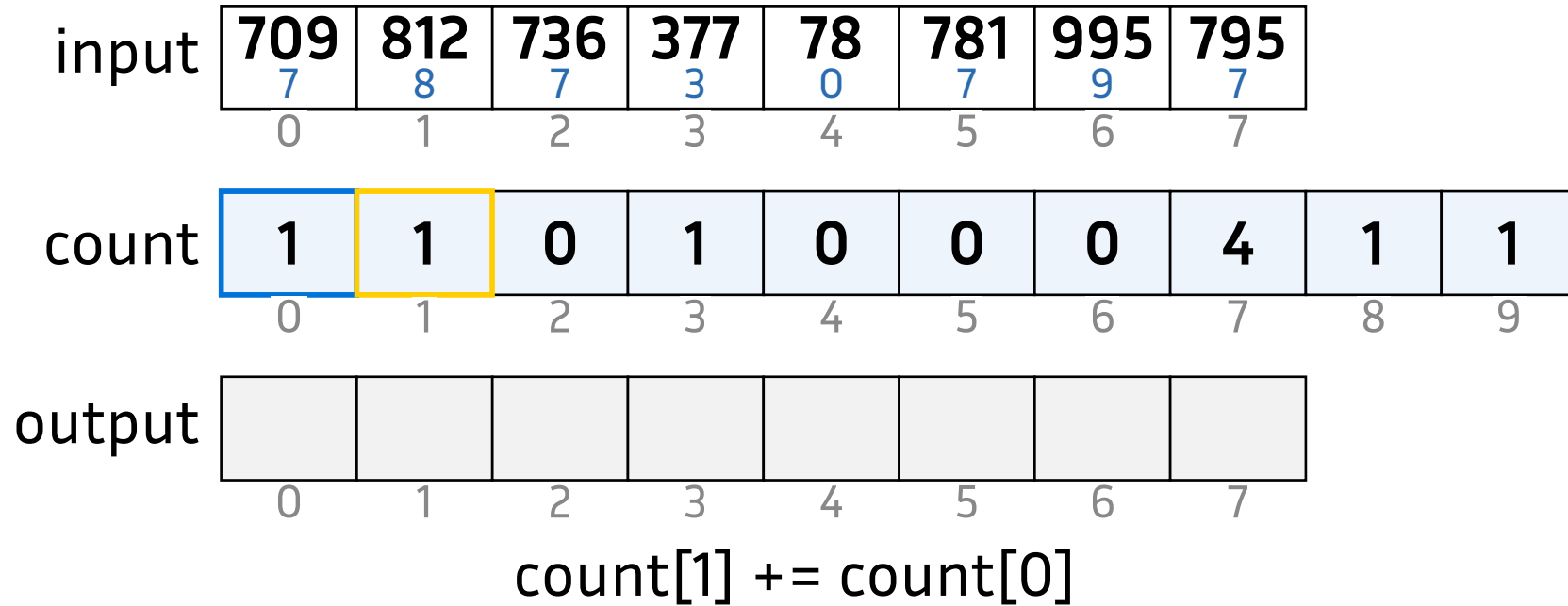
Linear Sorts – Radix Example



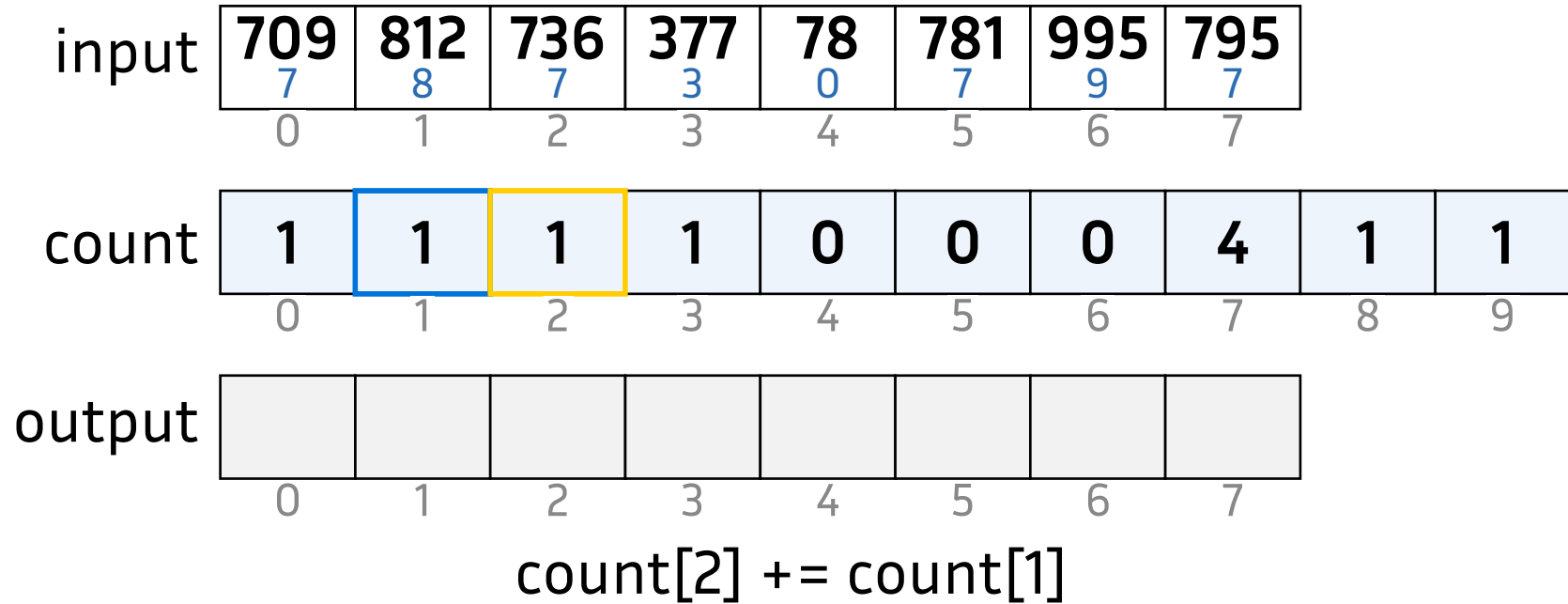
Linear Sorts – Radix Example



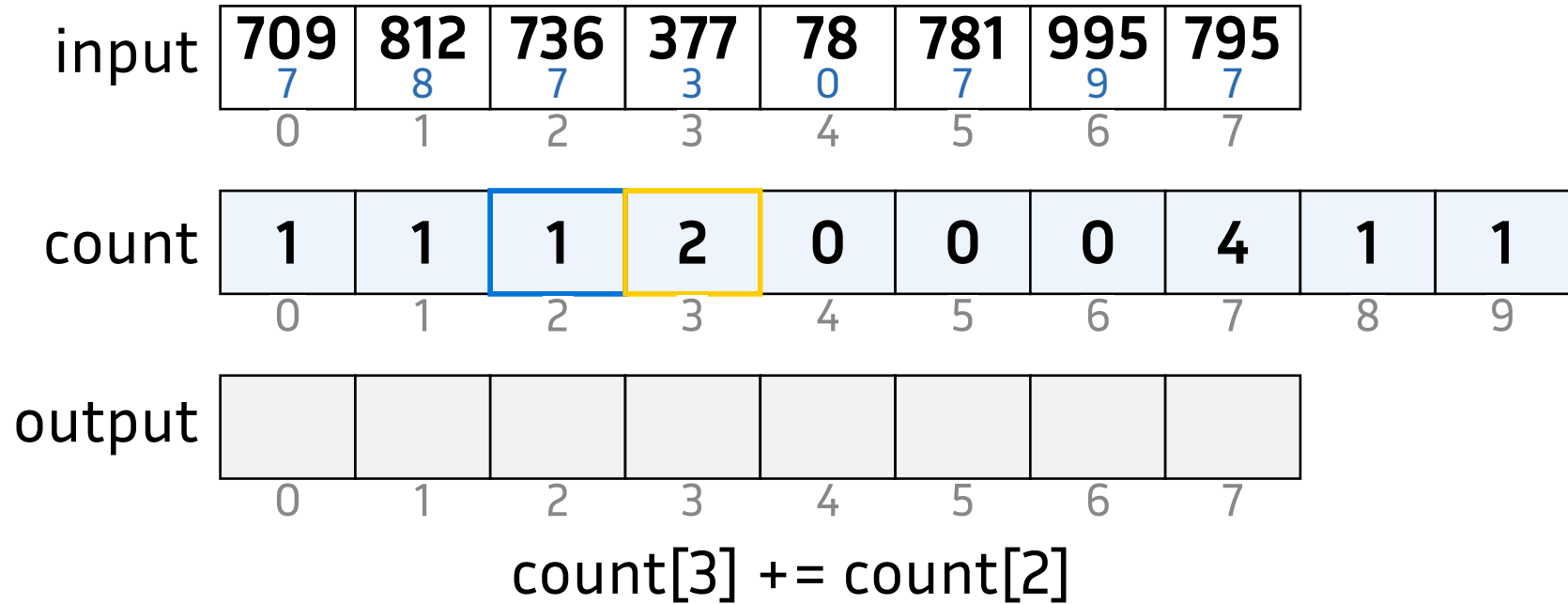
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

input

709	812	736	377	78	781	995	795
7	8	7	3	0	7	9	7
0	1	2	3	4	5	6	7

count

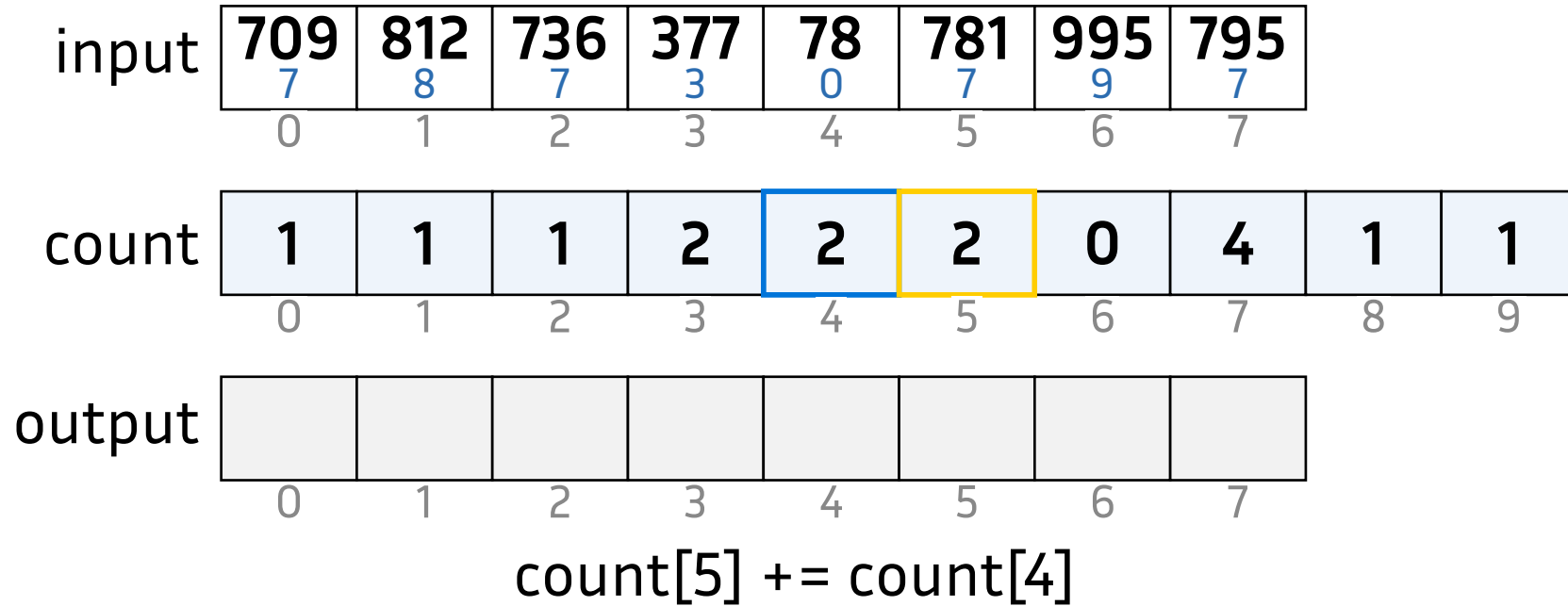
1	1	1	2	2	0	0	4	1	1
0	1	2	3	4	5	6	7	8	9

output

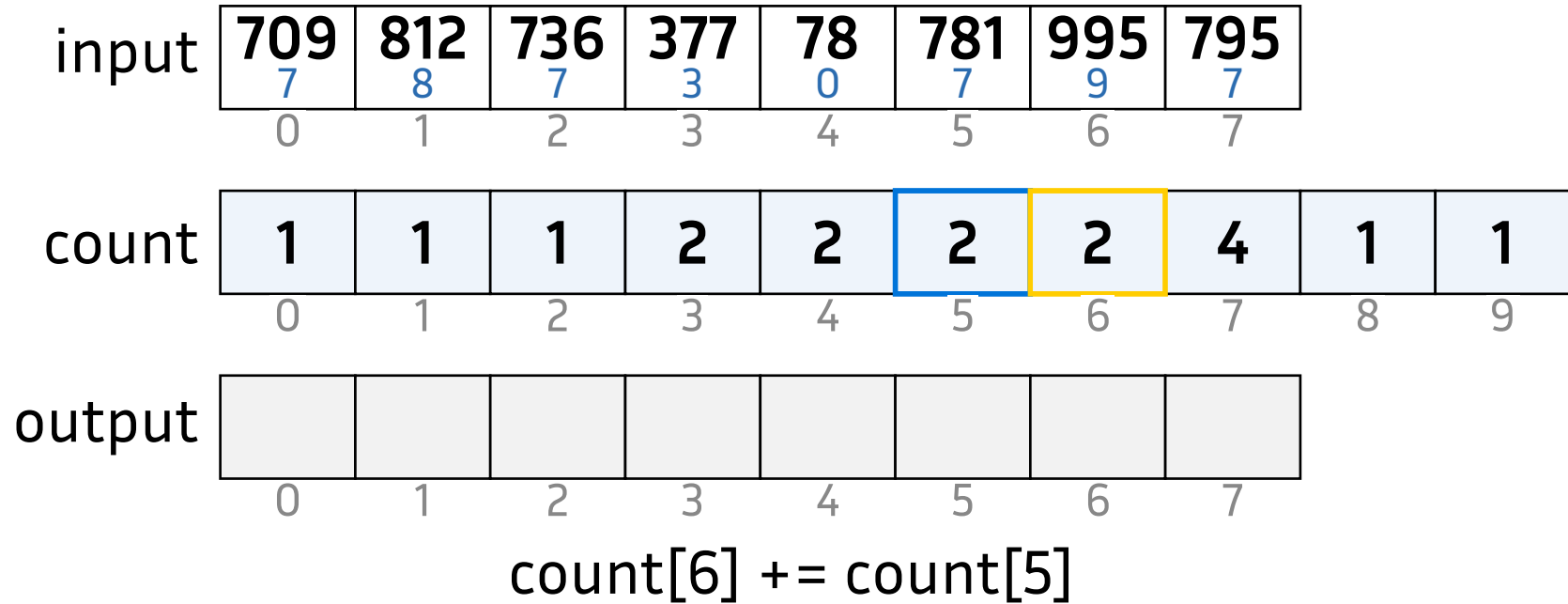
0	1	2	3	4	5	6	7

count[4] += count[3]

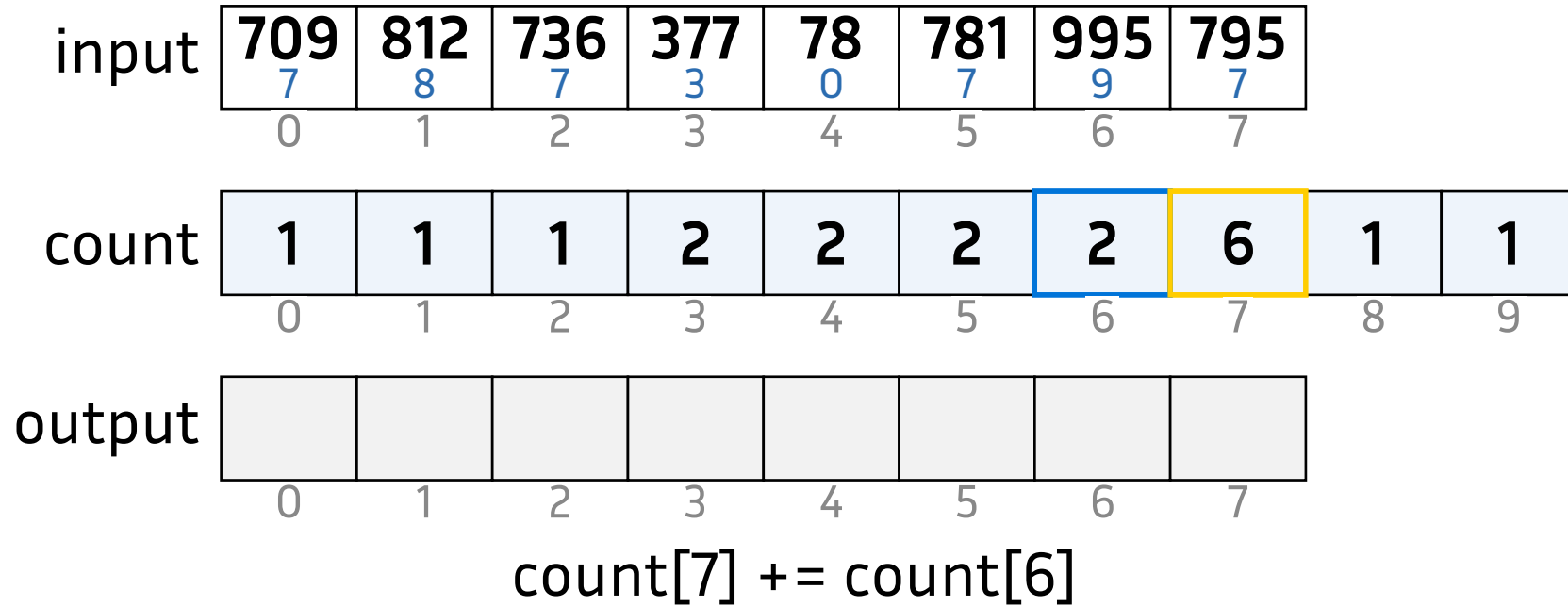
Linear Sorts – Radix Example



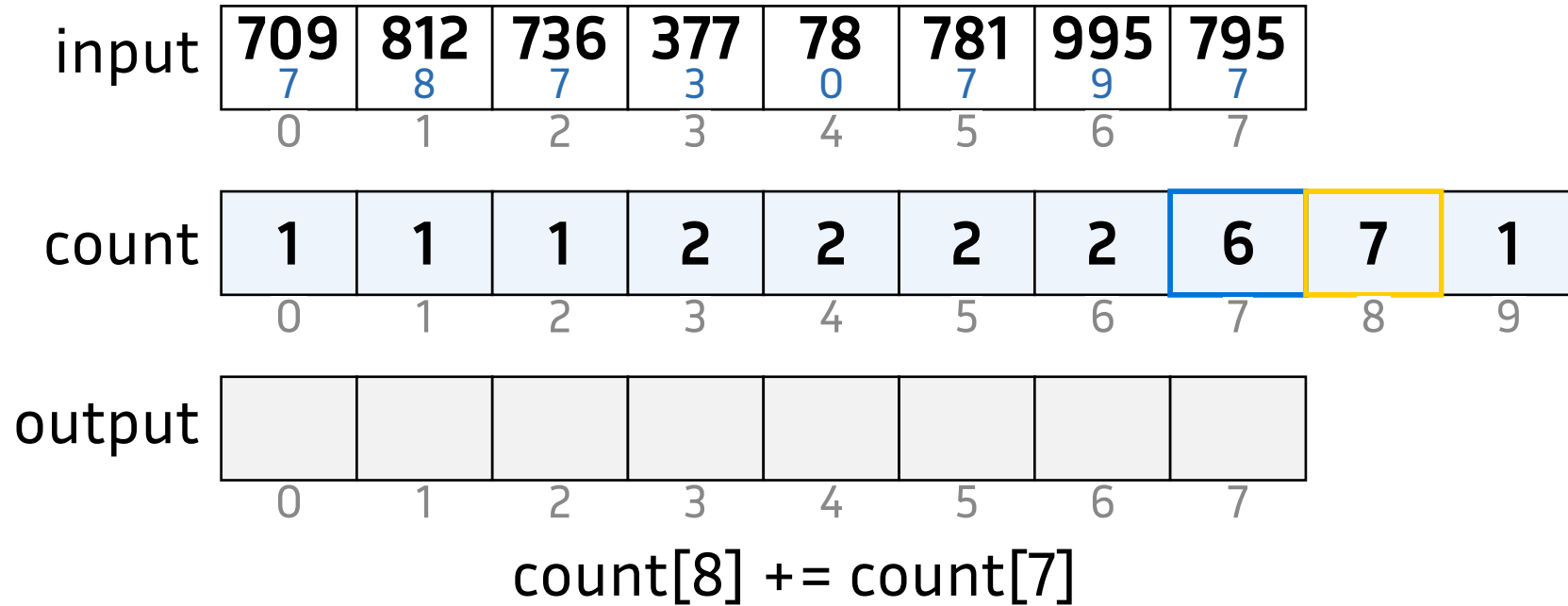
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

input

709	812	736	377	78	781	995	795
7	8	7	3	0	7	9	7
0	1	2	3	4	5	6	7

count

1	1	1	2	2	2	2	6	7	8
0	1	2	3	4	5	6	7	8	9

output

0	1	2	3	4	5	6	7

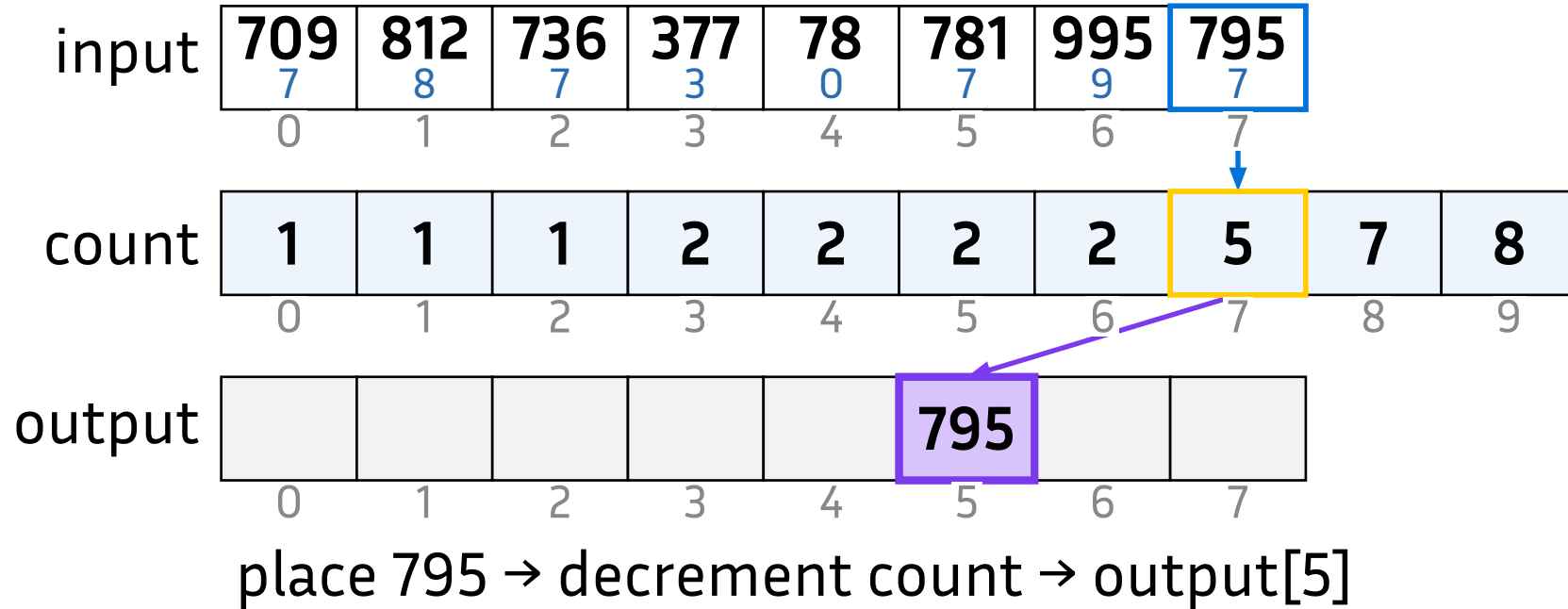
count[9] += count[8]

Linear Sorts – Radix Example

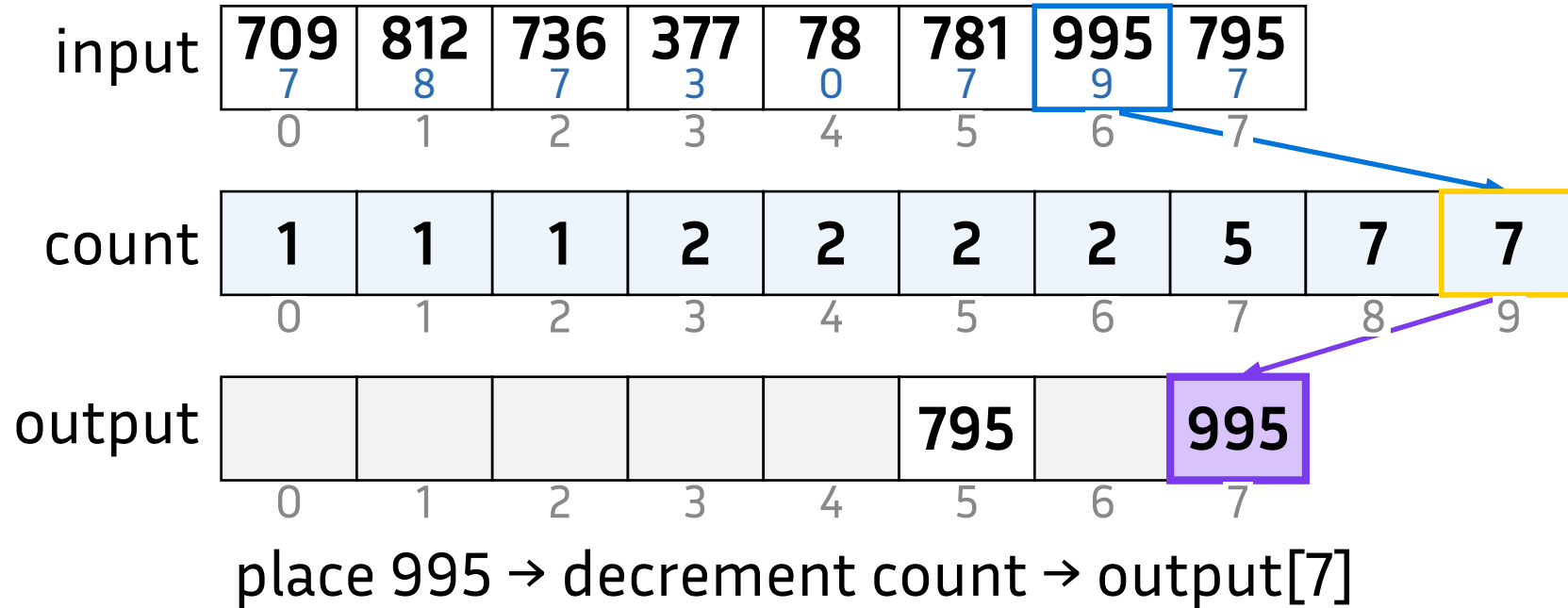
input	709	812	736	377	78	781	995	795		
	7	8	7	3	0	7	9	7		
	0	1	2	3	4	5	6	7		
count	1	1	1	2	2	2	2	6	7	8
	0	1	2	3	4	5	6	7	8	9
output										
	0	1	2	3	4	5	6	7		

counts → end positions

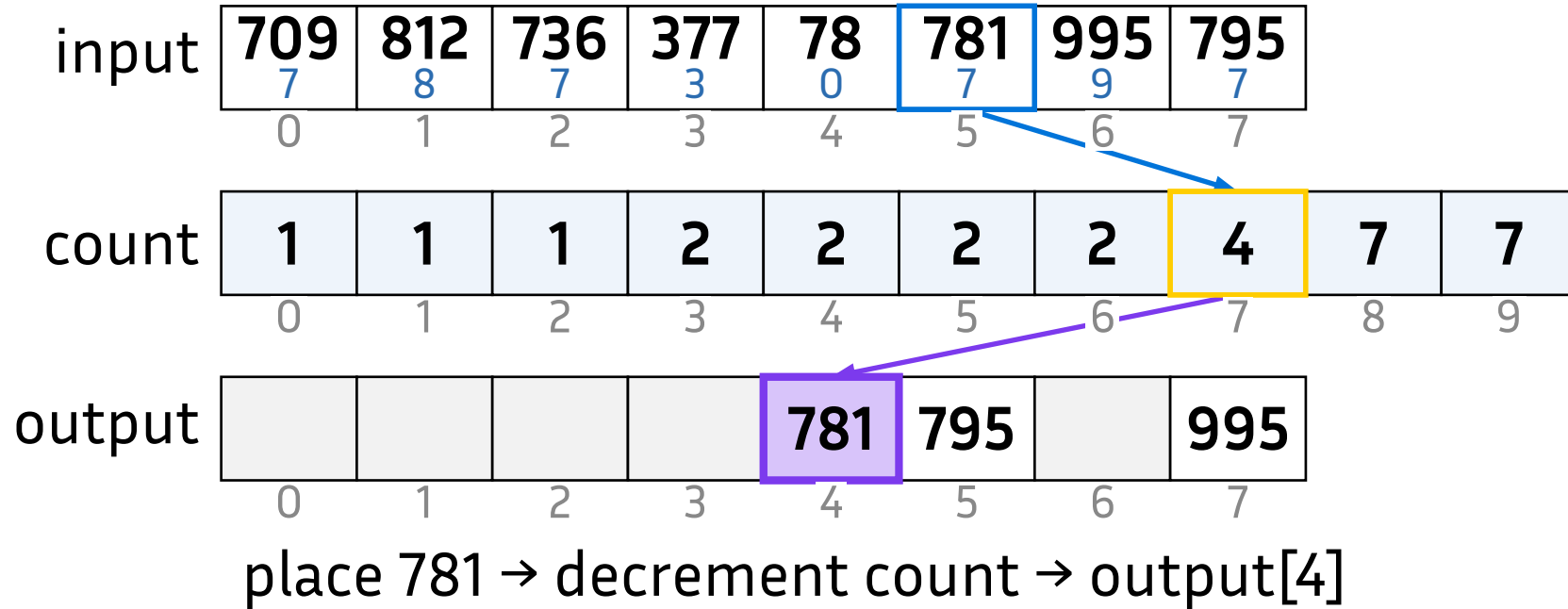
Linear Sorts – Radix Example



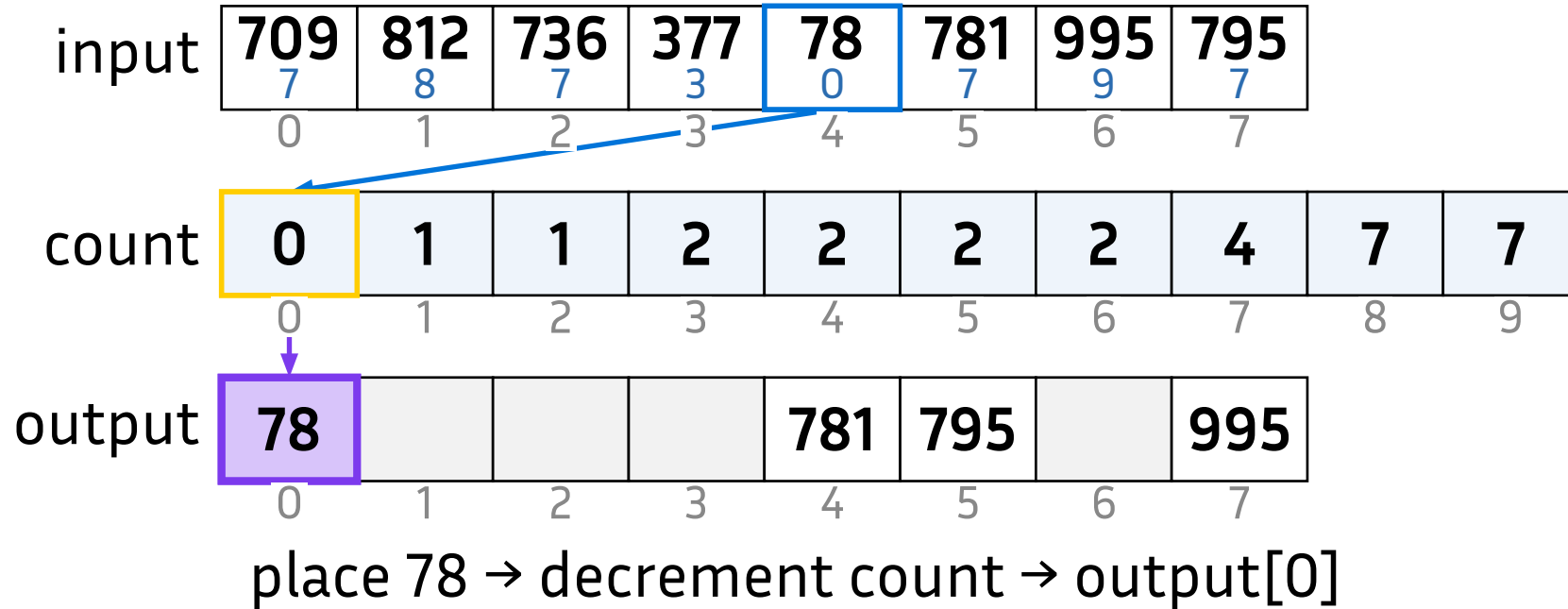
Linear Sorts – Radix Example



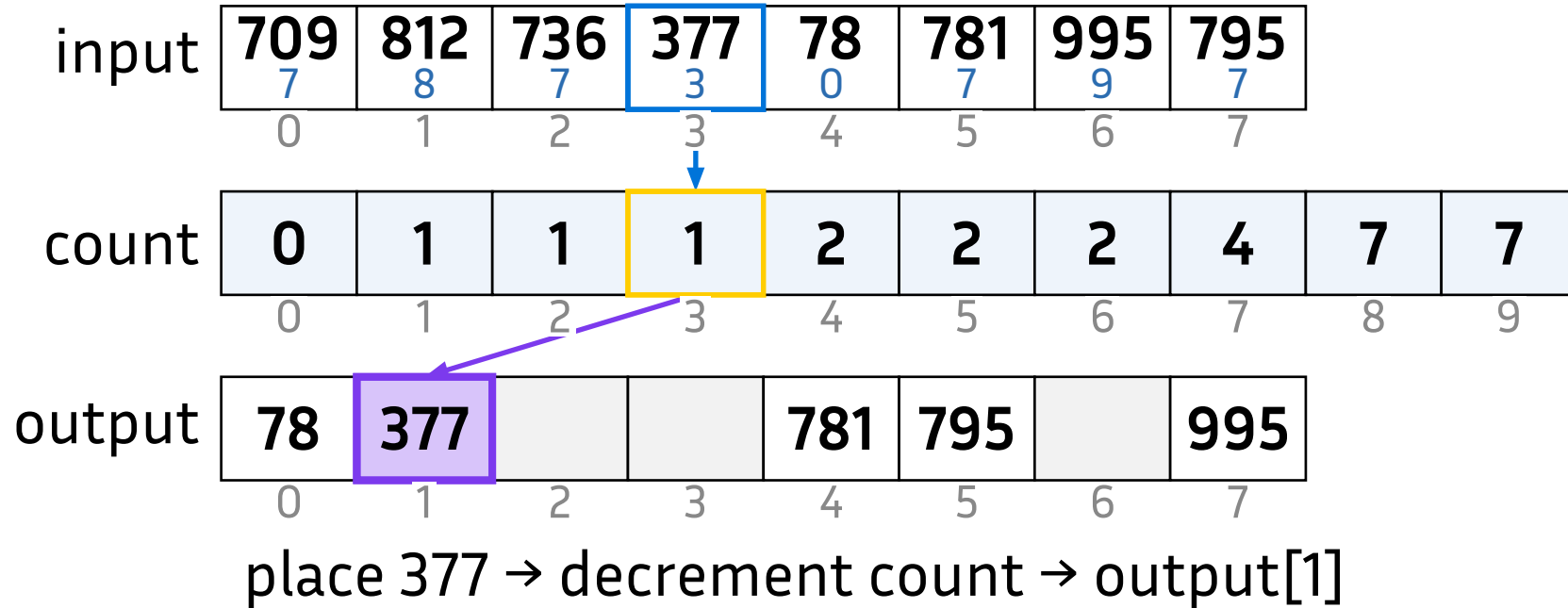
Linear Sorts – Radix Example



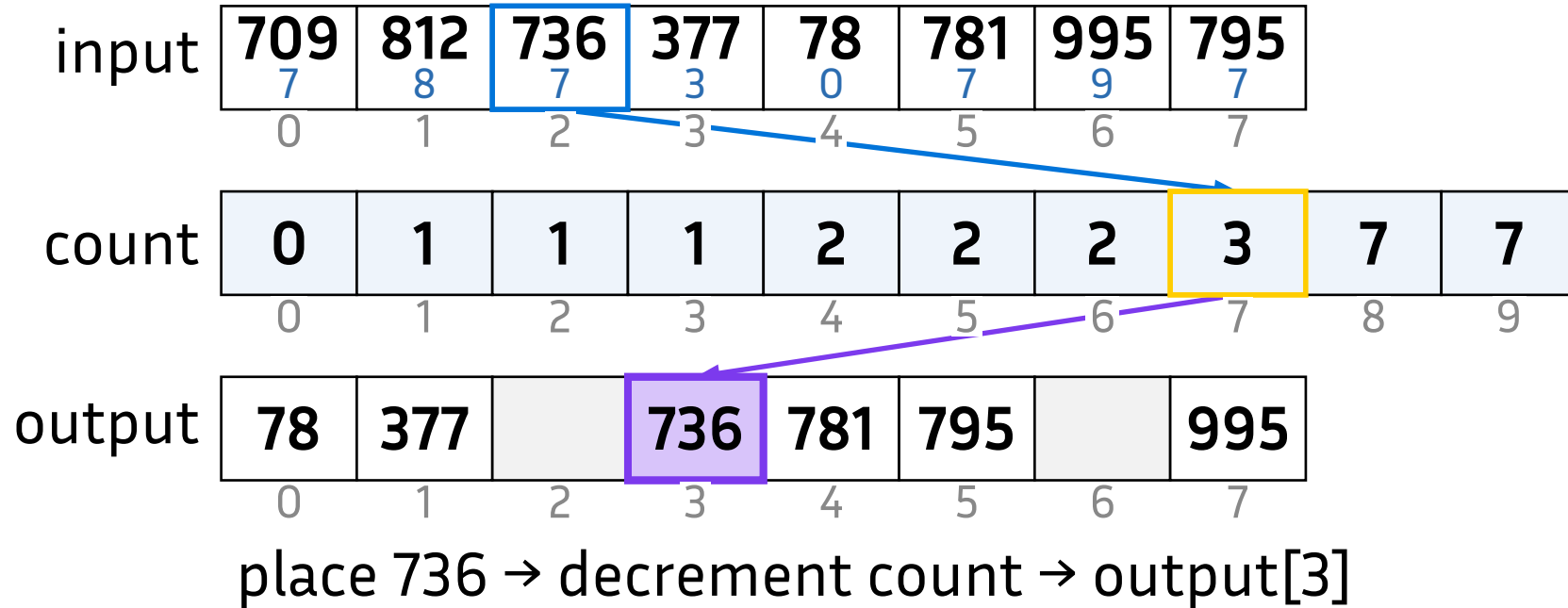
Linear Sorts – Radix Example



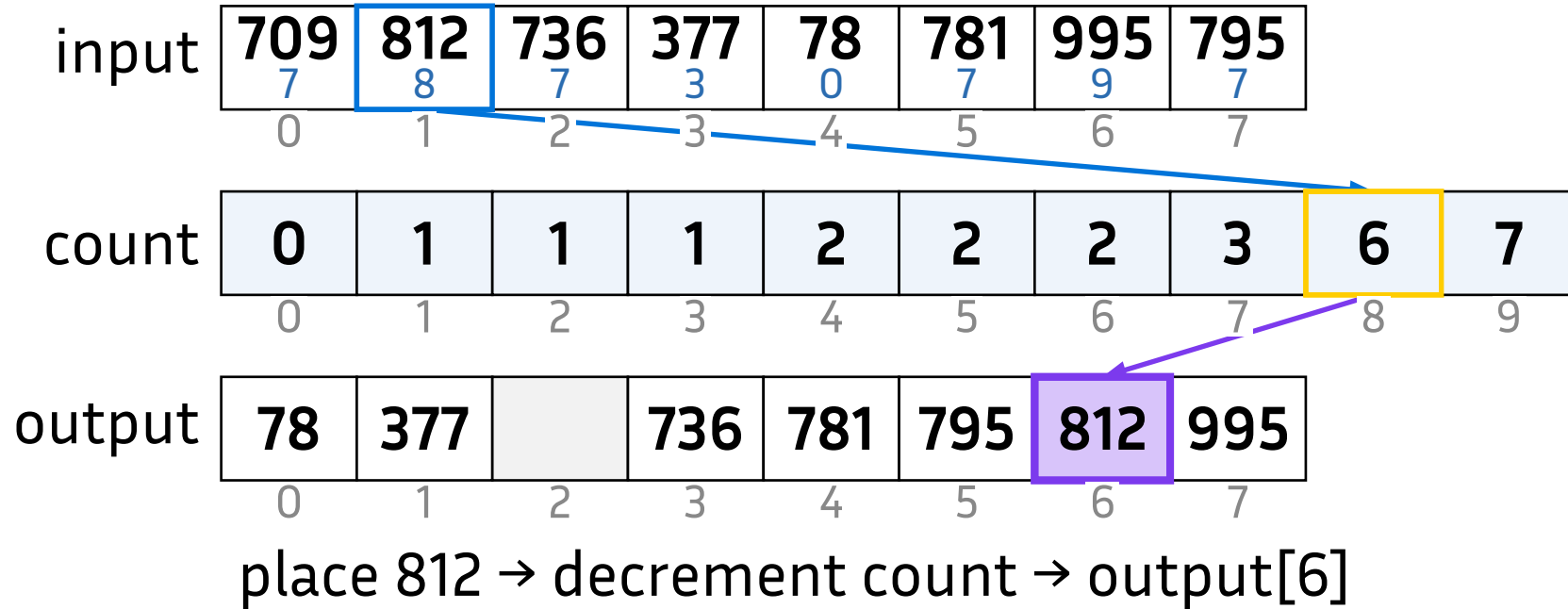
Linear Sorts – Radix Example



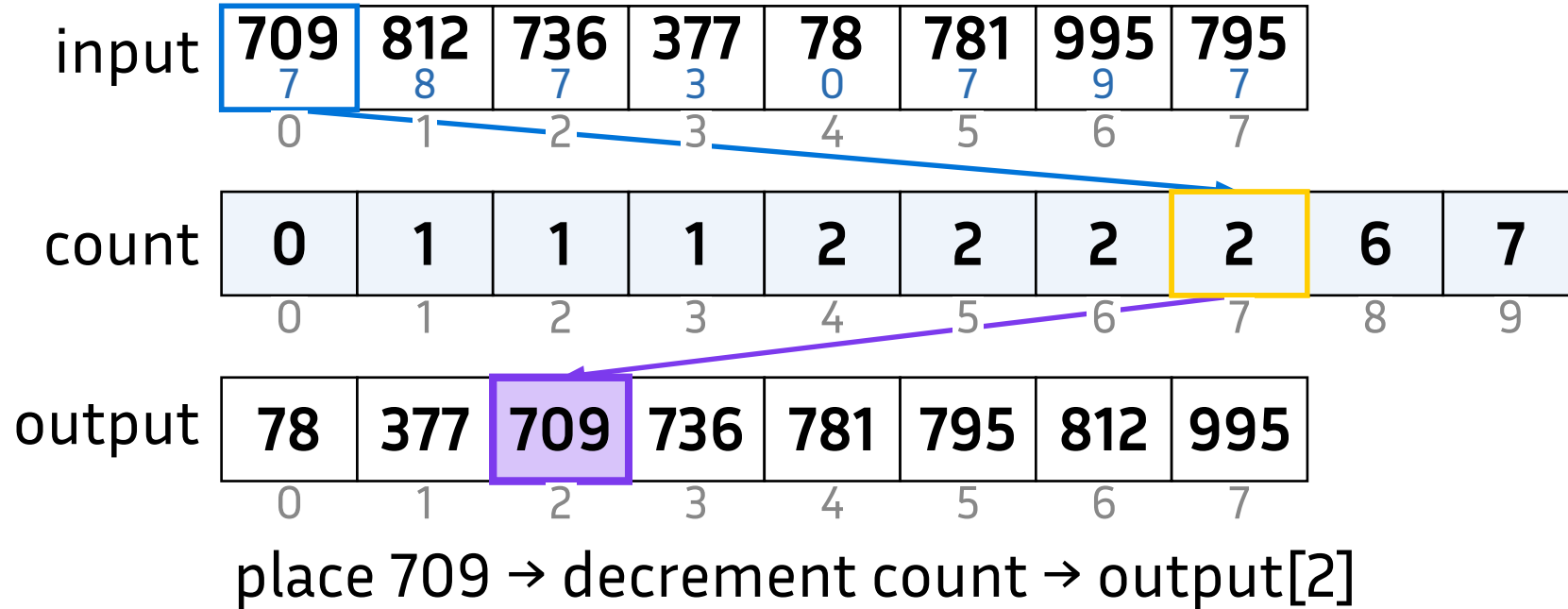
Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example



Linear Sorts – Radix Example

input

709	812	736	377	78	781	995	795
7	8	7	3	0	7	9	7
0	1	2	3	4	5	6	7

count

0	1	1	1	2	2	2	2	6	7
0	1	2	3	4	5	6	7	8	9

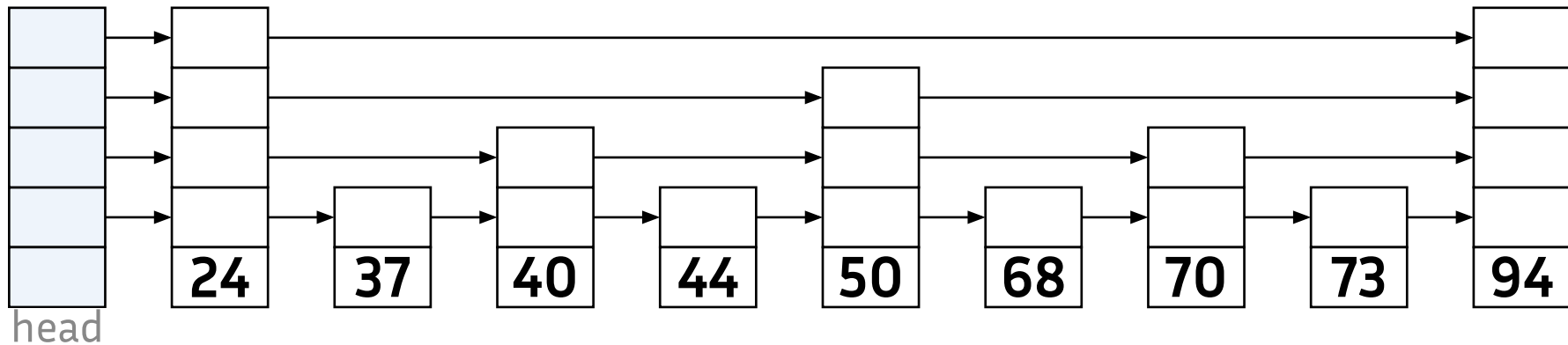
output

78	377	709	736	781	795	812	995
0	1	2	3	4	5	6	7

sorted

Skip Lists

Definition (Skip List) A linked-list like data structure where you can jump **more than one node** at a time by including **multiple next pointers**.

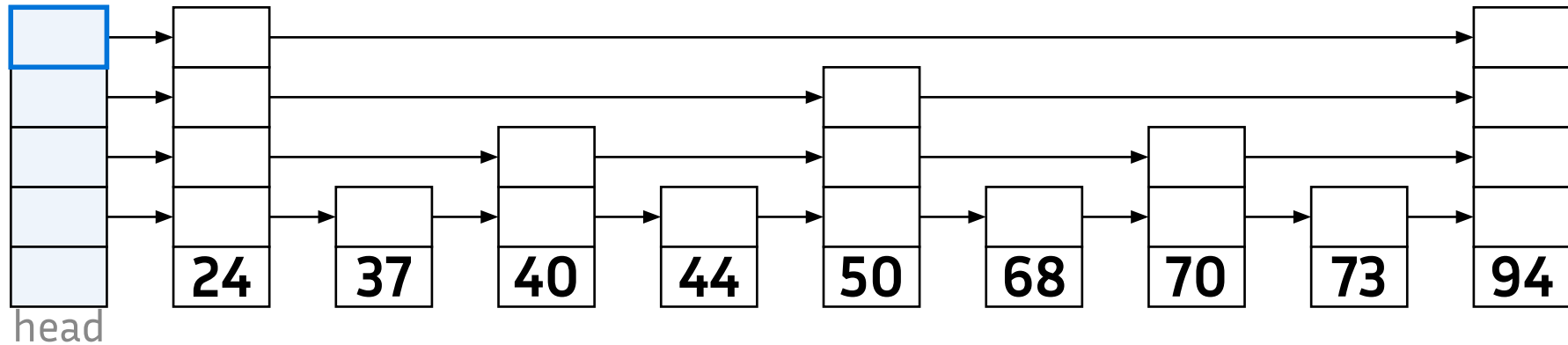


Skip List – Lookup

- 1 Start at the head node and the most express lane
- 2 While the **current node** is **not null**:
 - 3 **Peek** ahead at the next node
 - 4 If the lookup value is found, return **true**
 - 5 If the lookup value is **less** than the peeked value:
 - 6 | Move to the **next slower lane**
 - 7 | Otherwise take one step ahead, update **next node**
- 8 Return **false** if the node was not found

Skip List – Lookup Example

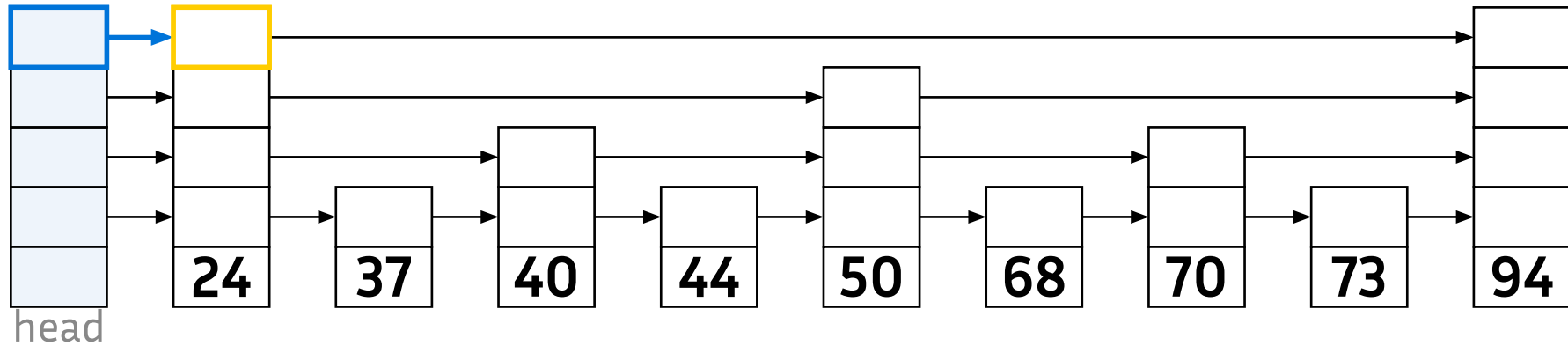
♦ Lookup **73**



search 73

Skip List – Lookup Example

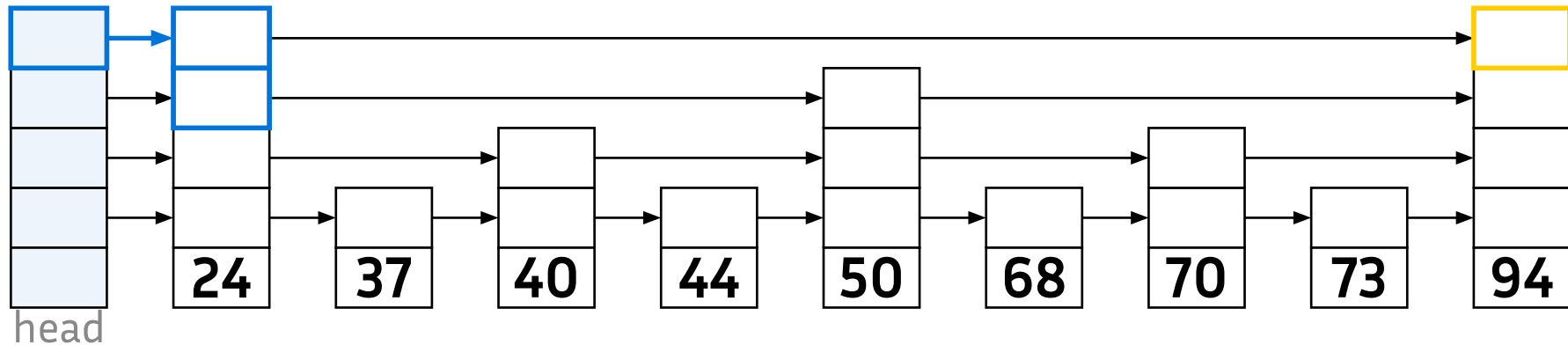
♦ Lookup **73**



$24 < 73 \rightarrow \text{right}$

Skip List – Lookup Example

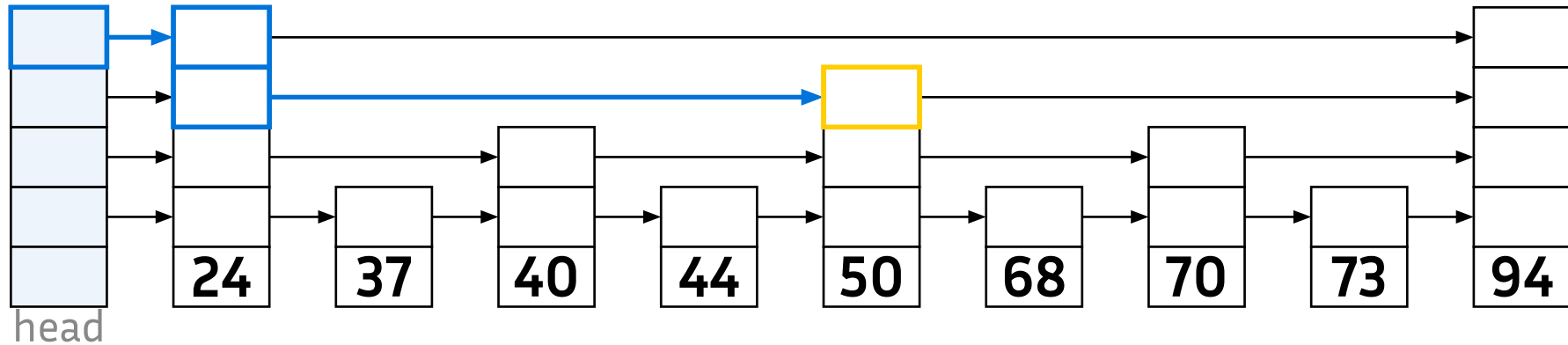
◆ Lookup **73**



$94 \geq 73 \rightarrow \text{drop}$

Skip List – Lookup Example

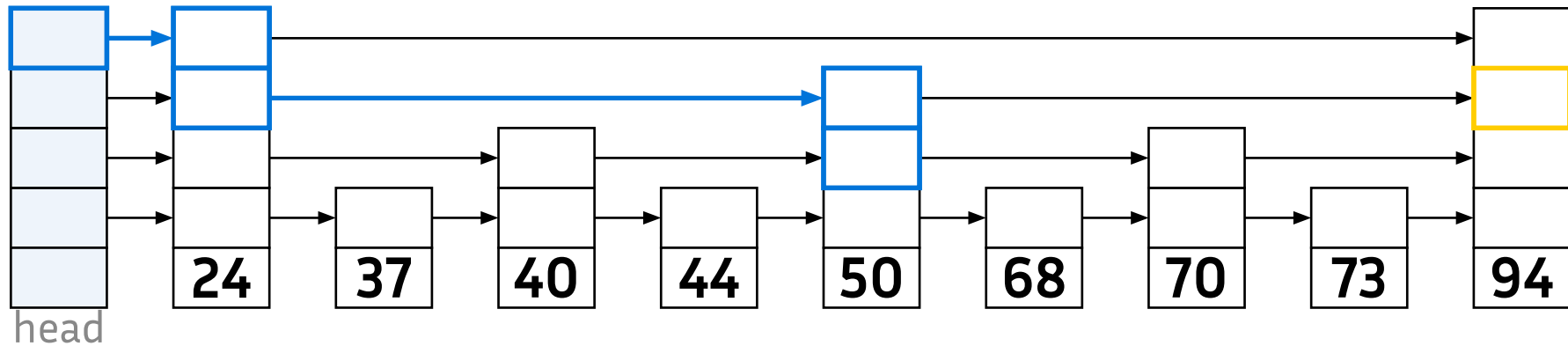
♦ Lookup **73**



$50 < 73 \rightarrow \text{right}$

Skip List – Lookup Example

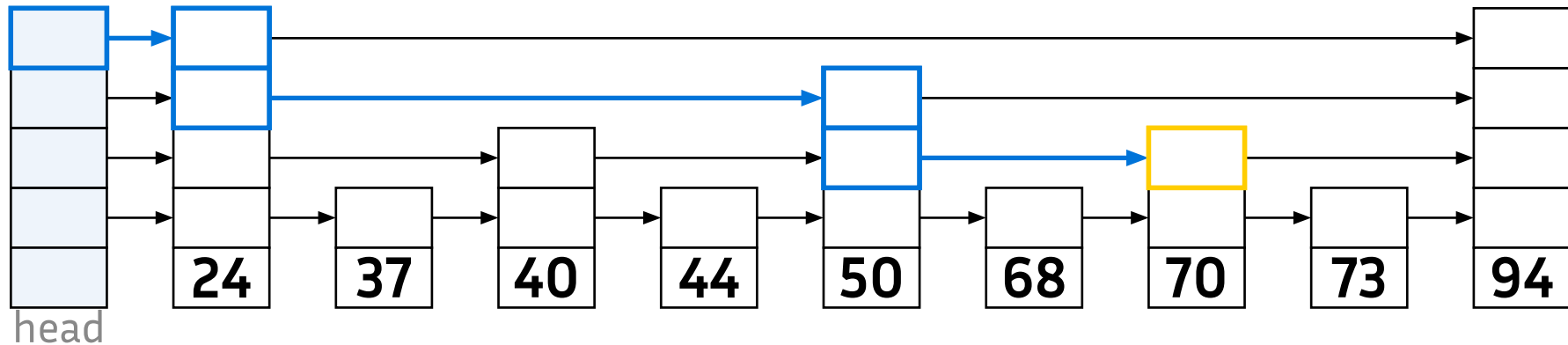
♦ Lookup **73**



$94 \geq 73 \rightarrow \text{drop}$

Skip List – Lookup Example

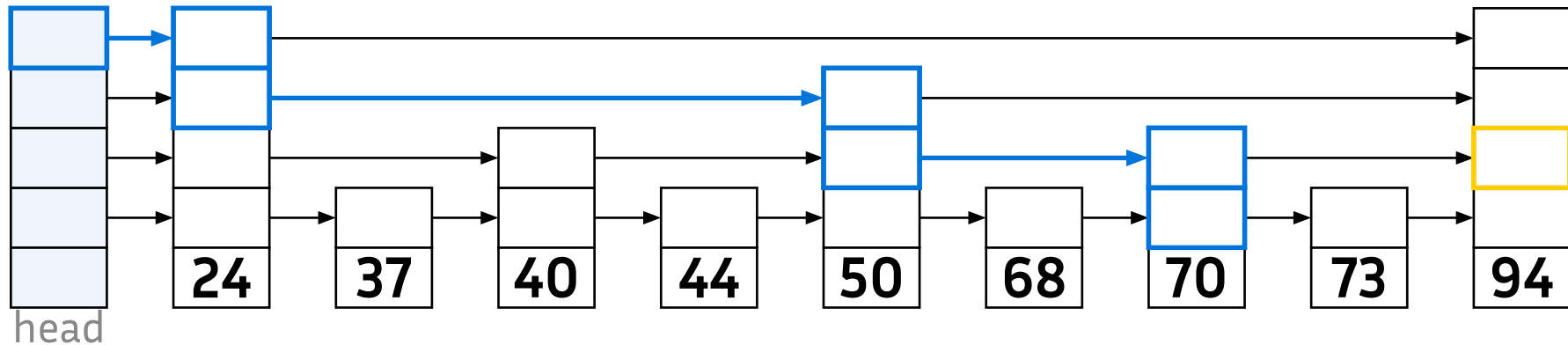
♦ Lookup **73**



$70 < 73 \rightarrow \text{right}$

Skip List – Lookup Example

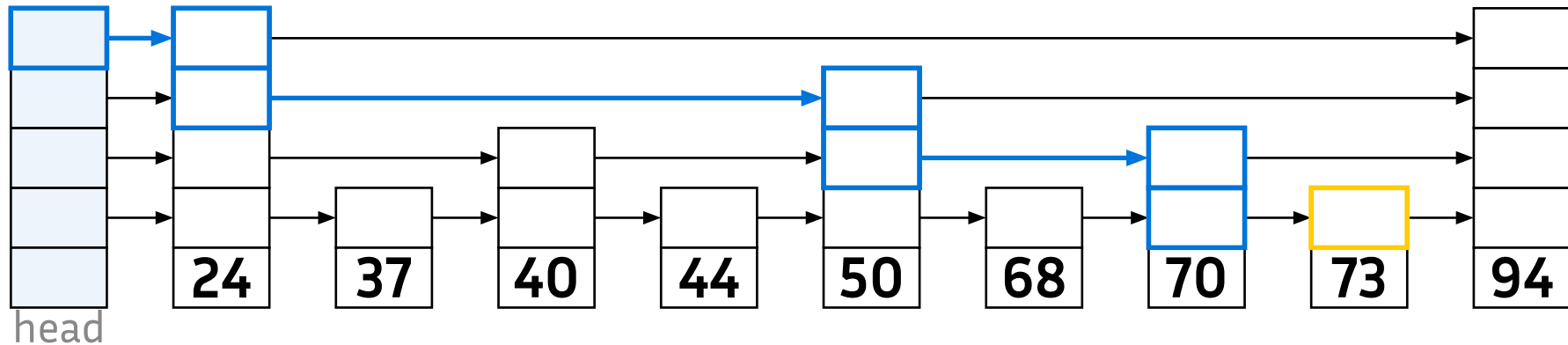
♦ Lookup **73**



$94 \geq 73 \rightarrow \text{drop}$

Skip List – Lookup Example

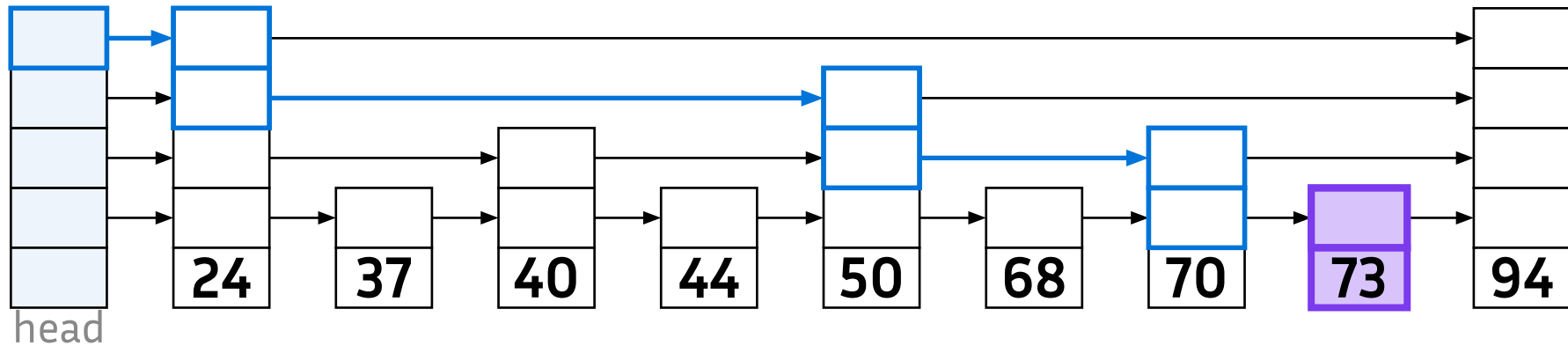
♦ Lookup **73**



$73 \geq 73 \rightarrow \text{stop}$

Skip List – Lookup Example

♦ Lookup **73**

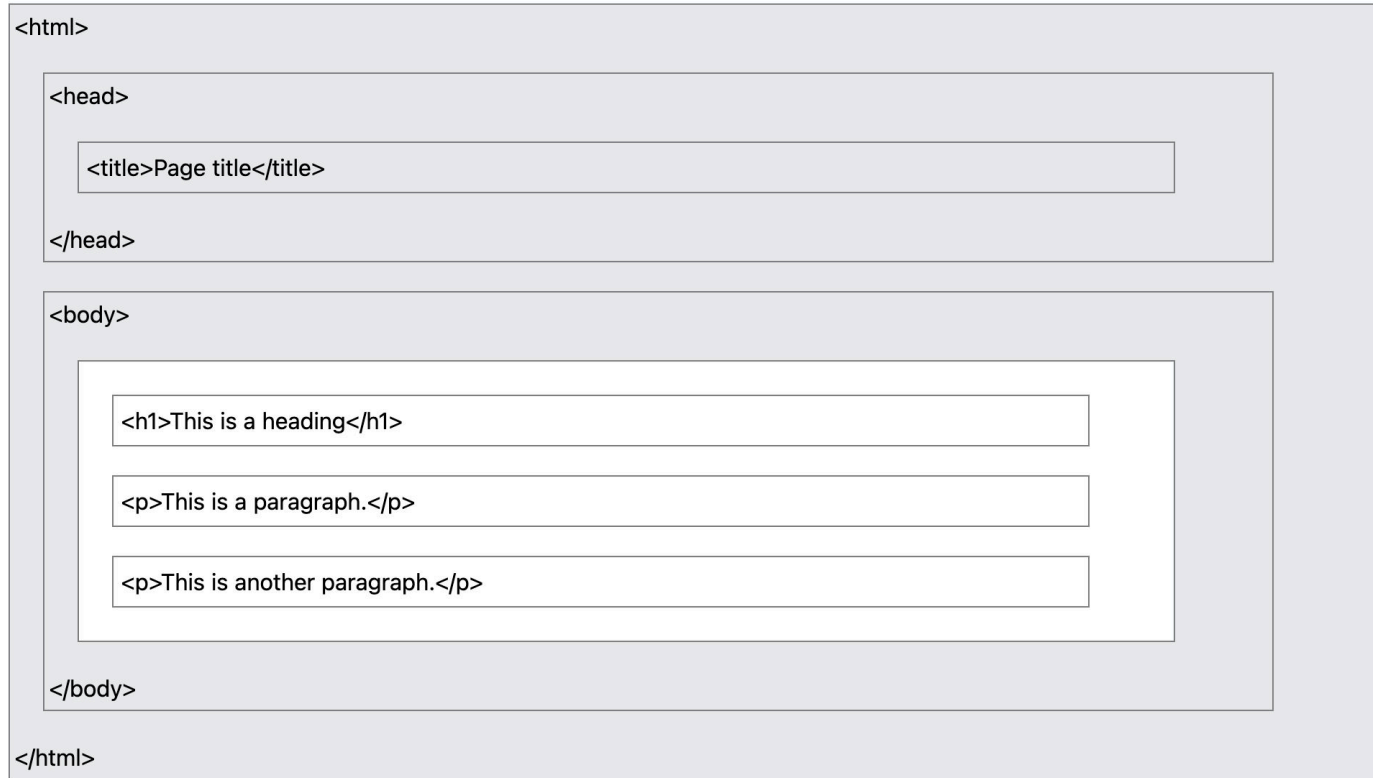


HTML + CSS + JavaScript

Some HTML tags. This is not an exhaustive list.

- ✦ `<html>` → Root of an HTML document.
- ✦ `<head>` → Page metadata.
- ✦ `<body>` → Page content.
- ✦ `<p>` → Paragraph of text.
- ✦ `<h1>`, `<h2>`, ..., `<h6>` → Section headings.
- ✦ `<a>` → Hyperlink / “anchor”.
- ✦ `<img>` → Image.
- ✦ `<div>` → Container to group other elements.

HTML + CSS + JavaScript



HTML + CSS + JavaScript

CSS

```
1 h2 { /* Apply to all h2 tags */
2     color: orange;
3     text-decoration: underline;
4 }
5 .bolded { /* Apply to elements with class="bolded" */
6     font-weight: bold;
7 }
8 #apple { /* Apply to elements with id="apple" */
9     color: red;
10    text-decoration: underline;
11 }
```

HTML + CSS + JavaScript

js

```
1 function init_ratings() {
2   for (const [_ , title_code] of Object.entries(titles)) {
3     fetch("http://127.0.0.1:8080/query?title=" + title_code)
4       .then(result => result.text())
5       .then(r => {
6         let elem = document.querySelector(
7           "#" + title_code + "-rating"
8         )
9         elem.innerHTML = "&diam;" .repeat(r)
10      })
11  }
12 }
```

GUI Programming

Definition (Declarative UI) A user interface paradigm where the developer describes what the UI should look like for the current data. The framework works out which DOM updates make that true.

- ✦ **Imperative** → find the node, then set its innerHTML, *everywhere, on every change*.
- ✦ **Declarative** → the span is the book's rating, and stays updated:

jsx

```
1 | <span>{"♦".repeat(rating())}</span>
```

GUI Programming – JSX

jsx

```
1 function App() {  
2   return (  
3     <>  
4       <h1>Book Ratings</h1>  
5       <img src={cover} width="150" />  
6     </>  
7   );  
8 }
```

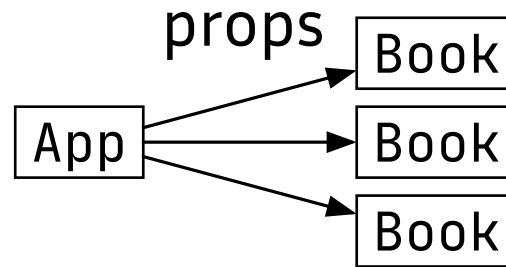
Must return one top-level element

Every tag must close

Embed JavaScript in HTML

GUI Programming – Components & Props

Data flows **down** into a component as **properties** or props.



Don't destructure props

jsx

```
1 function Book(props) {  
2   return <div id={props.code}>  
3     {props.title} by {props.author}</div>;  
4 }
```

GUI Programming – Signals

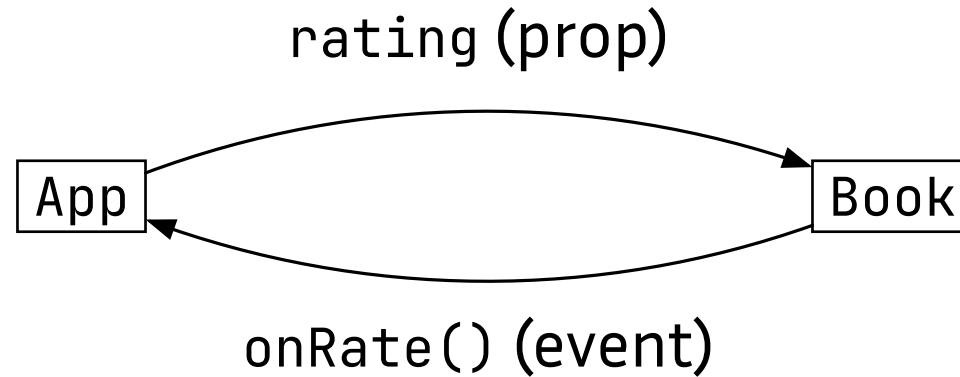
Definition (Signal) A reactive value. Read it with `s()`, write it with `setS(v)`. Solid re-runs only the spots that read `s()`.

jsx

```
1 | const [rating, setRating] = createSignal(5);  
2 | const diamonds = () => "♦".repeat(rating());
```

- ♦ `diamonds` is a **derived** value → a function of the signal.
- ♦ The component body runs **once**; there is no whole-component re-render.

GUI Programming – Callbacks



- ✦ Lift shared state to the common parent (App).
- ✦ Data flows **down** as props; a **callback** prop sends events **up**.
- ✦ One update site: `rate(code, value)`.

GUI Programming – Effects & Resources

Definition (Effect) Runs a side effect whenever the signals it **reads** change. Useful for when you need to interact with a server.

Definition (Resource) An asynchronous effect. Exposes more information about the state of the asynchronous request.



Practice Exam